SMART TRAFFIC FLOW OPTIMIZATION SYSTEM

A Thesis
Presented to the Faculty of
Information and Communications Technology Program
STI College Global City

In Partial Fulfilment
of the Requirements for the Degree
Bachelor of Science in Computer Engineering

Juliana Paula P. Baloy
Duane Benedict A. Mahinay
Harvey G. Malicse
Alvin Ken G. Modina

November 2025

ENDORSEMENT FORM FOR ORAL DEFENSE

TITLE OF RESEARCH: **Smart Traffic Flow Optimization System**

NAME OF PROPONENTS: Juliana Paula P. Baloy
Duane Benedict A. Mahinay
Harvey G. Malicse
Alvin Ken G. Modina

In Partial Fulfilment of the Requirements
for the degree Bachelor of Science in Computer Engineering
has been examined and is recommended for Oral Defense.

ENDORSED BY:

Sherwin M. Ostria
Thesis Adviser

APPROVED FOR ORAL DEFENSE:

Sherwin M. Ostria
Thesis Coordinator

NOTED BY:

Mark Frederick V. Salonga
Program Head

November 2025

APPROVAL SHEET

This thesis titled **Smart Traffic Flow Optimization System**, prepared and submitted by **Juliana Paula P. Baloy, Duane Benedict A. Mahinay, Harvey G. Malicse, and Alvin Ken G. Modina**, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Engineering, has been examined and is recommended for acceptance and approval.

Sherwin M. Ostria
Thesis Adviser

Accepted and approved by the Thesis Review Panel
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Engineering

Renia R. Matira
Panel Member

Joselito G. Oyao
Panel Member

Mark Frederick V. Salonga
Lead Panelist

APPROVED:

Sherwin M. Ostria
Thesis Coordinator

Mark Frederick V. Salonga
Program Head

November 2025

ACKNOWLEDGEMENTS

We, the researchers, would like to sincerely express our deepest gratitude to the people who have guided, supported, and inspired us throughout the journey of completing this thesis. Without their help, this study would not have been possible.

First and foremost, we would like to extend our heartfelt thanks to our thesis adviser and coordinator, Engr. Sherwin M. Ostria, for his continuous guidance, valuable advice, and patience in helping us understand and improve our work. His constant encouragement and reminders motivated us to stay focused and committed until the completion of this study.

We are also truly grateful to our Thesis Review Panel: Mr. Frederick V. Salonga, Mr. Joselito G. Oyao, and Mr. Eduardo B. Laurio Jr. for their time, effort, and insightful suggestions. Their comments and recommendations greatly contributed to the improvement and strengthening of this research.

Our special thanks also go to Mr. Joey Y. Bisnar and Mr. Angelo Bundang for their guidance, support, and contributions which became a big help to us in the process of finishing this thesis.

We would also like to give our sincerest appreciation to our parents and guardians for their unconditional love, financial assistance, and moral support. Their sacrifices, encouragement, and prayers inspired us to work harder and never give up despite challenges along the way.

To our classmates and friends, thank you for sharing ideas, giving encouragement, and providing positivity throughout the challenges of this journey. Your support has been a source of strength to us.

Lastly, above all, we thank God Almighty for granting us wisdom, knowledge, guidance, and strength. Without His grace, this accomplishment would not have been possible.

ABSTRACT

Title of research**:**	**Smart Traffic Flow Optimization System**
Researchers:	**Juliana Paula P. Baloy** **Duane Benedict A. Mahinay** **Harvey G. Malicse** **Alvin Ken G. Modina**
Degree:	**Bachelor of Science in Computer Engineering**
Date of Completion:	**June 2026**
Keywords:	Artificial Intelligence, Embedded System, Traffic Management, Internet of Things

Traffic congestion is one of the major problems in Metro Manila, resulting in longer travel times, accidents, and environmental pollution. Traditional traffic management systems, such as fixed-timer traffic lights and manual operations, have proven ineffective in addressing these challenges, particularly in prioritizing emergency vehicles and ensuring road safety. This study proposes a Smart Traffic Flow Optimization System that enhances traffic management through real-time optimization, ambulance prioritization, and automatic violation detection. The system integrates Artificial Intelligence (AI), Internet of Things (IoT), and computer vision to monitor traffic density, dynamically adjust traffic lights, detect ambulances, and identify violations such as red-light running, speeding, and motorcycle gear infractions. It combines hardware components like the Arduino Uno, Raspberry Pi 5, Arducam cameras, sensors, and relays with software tools such as Python, TensorFlow, YOLO, Supabase, and ReactJS to process real-time data and present it through web-based analytics. Using an evolutionary prototyping approach, the system was tested and refined under controlled conditions. While limited to selected intersections in Metro Manila and a few violation categories, results suggest that the system can significantly reduce congestion, improve emergency response times, and enhance road safety contributing to the development of smart city solutions in Philippines.

Table of Contents

	Page
Title Page	i
Endorsement Form for Proposal Defense	ii
Approval Sheet	iii
Acknowledgments	iv
Abstract	v
Table of Contents	vi
List of Tables	vii
List of Figures	viii
List of Appendices	ix
Introduction	12
Background of the problem	13
Overview of the current state of technology	17
Objectives of the study	18
Scope and limitations of the study	21
Review of related literature/studies/systems	28
Methodology	38
Overview of the Project	38
Collecting Components	39
Connecting of Components	39
Testing and Observation	39
Featured of Developed System/Project	40
Research Design	41
Research Settings	41
Participants	41
Sampling Procedure	42
Research Instrument	42
Data Collection	42
Data Analysis	43
HIPO	45

Hardware/Software

Hardware 53

Software 61

Results and Discussions 81

Conclusion and Recommendation 93

References 96

Appendices 98

LIST OF TABLES

Table		Page
1	Benchmarking on Other Made Prototypes	37
2	Ambulance Detection Response Time	85
3	Detection Accuracy Ai Performance	87
4	Traffic Violation Counts	87
5	Ambulance and Normal Light Switch	88
6	Testing Security of Website Signup page	89
7	Testing Data Fetching	90
8	Sound Sensor Module Response	91
9	List of Activities	115
10	Fixing Errors Guide	145
11	Functionality Results	161
12	Performance Result	162
13	Usability Result	164
14	Reliability Results	165
15	Portability Results	167
16	List of Materials bought	201

LIST OF FIGURES

Figure	Page
1 ERD	43
2 Process Flowchart	47
3 Block Diagram	50
4 Schematic Diagram	52
5 Arduino Uno	53
6 Power Supply	54
7 Relay	55
8 Arducam	56
9 Wire	56
10 Adaptor	57
11 Light Bulb	58
12 Raspberry Pi 5	59
13 Sound Sensor	60
14 Arduino IDE	61
15 Raspberry Pi IDE	62
16 Supabase	62
17 Tensorflow	63
18	
19	
20	
21	
22	
23	
24	
25	

26	3D Model of Assembled Stoplight (Isometric View)	60
27	3D Model of Assembled Stoplight (Top View)	61
28	3D Model of Assembled Stoplight (Back and Side View)	62
29	3D Model of the Stoplight (Bottom View)	63
30	3D Model of Assembled Stoplight (Front View)	64
31	3D Model of the Stoplight (Isometric View)	65
32	3D Model of the Stoplight (Front, Side, Back View)	66
33	3D Model of the Stoplight (Top View)	66
34	3D Model of the Stoplight (Bottom View)	67
35	3D Model of the Component Box (Isometric View)	68
36	3D Model of the Component Box (Side and Back View)	68
37	3D Model of the Component Box (Front View)	69
38	3D Model of the Component Box (Bottom View)	69
39	Physical Layout of Stoplight	70
40	Stoplight Light Functionality	71
41	Detection Line and Boxes Logic	72
42	Signup Page Website	72
43	Bar Graph of Ambulance Detection Response Time	74
44	Bar Graph of Detection Accuracy	75
45	Bar Graph of Traffic Violation Counts	76
46	Bar Graph of Light Switching Duration	77
47	Bar Graph of Data Fetching	79
48	Bar Graph of Sound Sensor Detection Strength	80

LIST OF APPENDICES

Appendix		Page
	A. Project Acceptance	99
	B. Transcript of Interview	102
	C. Involvement Form	107
	D. Schedule of Activities	112
	E. Screen Design	115
	F. Sample System Report	121
	G. User's Manual	123
	H. Database Schema and Tables	149
	I. Survey Questionnaire	151
	J. Survey Results and Tally	159
	K. Source Code	168
	L. Datasheet	188
	M. Budgetary Expenses	197
	N. Curriculum Vitae	202

INTRODUCTION

Traffic congestion was a problem in urban areas globally, which included the Philippines. There has been a significant increase in the volume of vehicles, which caused traffic in many lanes, leading to delays, accidents, and negative impacts on the environment. Due to the limitations of the existing traffic management systems, the issue grew, and the effects on emergency vehicles, such as ambulances and fire trucks, were particularly concerned, as they were often delayed, which put lives at risk. Furthermore, traffic violations were identified as a major contributor to road problems, as they disrupted the flow of traffic and compromised the safety of every commuter and driver, whether in a motorcycle, car, or bicycle, creating a negative impact on the transportation system.

To address this problem, the development of a Smart Traffic Flow Optimization System was proposed. This system was designed to leverage real-time optimization, respondent prioritization, and automated violation detection. Using advanced technologies such as sensors, IoT, and machine learning, the system aimed to provide prioritization for emergency respondents. Additionally, a violation detection feature was integrated to assist traffic marshals in determining driver infractions of road rules, thereby enhancing safety for all commuters and drivers.

The implementation of this research project was intended to help reduce traffic congestion on roads, improve the response time of emergency vehicles, enhance citizen safety, and mitigate the challenges of urban traffic. The project aimed to demonstrate the capabilities of smart technologies in managing a traffic system by optimizing each lane, thereby addressing the Philippines' growing traffic challenges.

Background of the Problem

Frequent changes have enhanced the increase in the number of vehicles on city roads, thereby leading to more traffic jams, longer times to get to and from work or other places, and pollution of the environment. When the population density increased, the capacity of classical traffic control systems to respond to the increasing flow of transport seemed to be insufficient, and thus, certain inefficiencies developed. This research relied on recent United Nations data which showed that more than 55% of the world population already lived in urban areas and that this percentage was likely to reach 68% by the year 2050. With the growth of cities, people faced new tests in traffic management, which highlighted the need for the development of new ideas in this sphere. Addressing these problems was considered very important to reduce traffic, improve emergency response times (Abdullahi Chowdhury et al. 2023), and enhance road safety. Ignoring them would have been detrimental and would have led to more delays in traffic, longer emergency response times, accidents, and inefficiency in urban transportation systems. The consequences of not addressing these issues in a Smart Traffic Flow Optimization System were identified as delayed emergency vehicle response, worsened congestion and violations, and public dissatisfaction due to frustration from traffic.

The existing traffic signals that utilized fixed timers and manual checks were unable to cope with contemporary traffic flow. These systems did not exhibit real-time sensitivity to allow for changes whenever traffic conditions arose, and the traffic regulation systems often failed, with traffic jams and road accidents regularly being recorded and a lot of time being wasted on the roads. They also discouraged efficiency, such that vehicles clogging up the highways burned fuel inefficiently, adding to the emission of greenhouse gases. This has been a significant concern, especially in crowded urban areas, pushing up pollution levels and contributing to global warming. The Capstone project aimed to solve traffic congestion by optimizing the effectiveness of CCTV cameras and signal timings in real time. It was designed to improve emergency respondents' response time through the prioritization of emergency vehicles at lanes, especially at intersections. It also sought to enhance road safety with a violation detection system, creating a public-friendly and efficient way of managing traffic in urban areas (Pang, J.L. 2023). In addition, automated

violation detection systems such as red-light cameras were limited because they could only respond through visual monitoring and could not intervene in real time to immediately fix problems. These limitations demonstrated the need for an advanced traffic management system that optimized traffic, prioritized emergency vehicles, and actively prevented violations to avoid accidents.

Key contemporary challenges demonstrated that cities were unable to manage population growth, unemployment, needs, and other urban challenges, which shaped the notion of Smart Cities – an idea focused on different advanced technologies for developing effective and sustainable cities. Central to this approach was the Internet of Things (IoT), which was defined as the integration of different objects to gather and share data. It was recognized that when traffic was managed through IoT, cities could implement smart traffic solutions that would study, track, and control traffic with incredible precision. This led to improved traffic flow, better management of traffic density, and positive implications for road safety. The proposed IoT-based Smart Traffic Flow Optimization System was envisioned to enhance the facilitation of traffic flow in urban centers by responding to real-time information from embedded devices such as cameras, and Arduino to control lights' operations according to traffic patterns. The research focused on the field of traffic management. Its aim was to enhance traffic management, speed limit enforcement, and emergency response by using advanced technologies like sensors, with a specific focus on prioritizing ambulance vehicles and violation detection (Aditya Pawar et al. 2024). The key trends that were identified included the increasing use of IoT, AI, and smart sensors in traffic systems. The challenges addressed were traffic congestion, emergency vehicle delays, low emergency response time, and violation detection. The opportunities included developing real-time, automated systems that improved road safety and efficiency. The impact of the project on the development of the field arose from the use of IoT technology coupled with a Facility Management System for real-time traffic monitoring and controlling in traffic-congested zones.

The intended audience of the study comprised urban planners, local governments, and citizens. The conclusions drawn from the study were designed to offer guidance to urban planners and city officials in utilizing Embedded-IoT technology for the efficiency and

sustainability of traffic and for the reduction of infrastructural costs. The public was set to benefit from shorter travel times, a better atmosphere, and safer routes. Also, businesses were expected to benefit from improved revenue and enhanced efficiency of work and transportation for deliveries. The study primarily targeted areas like BGC Global, especially lanes coming from McKinley Hills, where traffic led to higher emissions and longer estimated arrival times for vehicles than usual. With real-time traffic optimization, the systems aim to dynamically adjust each stoplight timing using administrative controls. This was intended to minimize traffic congestion and allow the flow of traffic to proceed smoothly, leading to an effective traffic management system. The capstone project was designed to help the urban populace experience less traffic on the roads and safer roads, enable city governments to achieve improved mobility and sustainability, assist businesses through reduced time in transporting goods, and be environmentally friendly by reducing emissions on the roads.

This research fulfilled significant research needs by investigating the potential of IoT technology for developing reliable and efficient smart traffic management systems in smart cities. As it was revealed that several cities had already embarked on the implementation of Embedded IoT-based solutions, this research sought to elaborate on the efficiency of such systems, the emerging risks, and new ideas that could be adopted by other cities globally. Hence, the study had important implications for the development of new urban traffic control and transport strategies to address traffic congestion and its effects on the natural environment. The conventional traffic systems at the time lacked the capacity to handle these issues effectively, hence the need for IoT, which provided timely and sustainable solutions to traffic circulation problems. The consequences of not addressing these issues were identified as delayed emergency vehicle response, worsened congestion and violations, and public dissatisfaction.

Statement of the Problem

The Metro Manila areas still have issues of extreme traffic jams, delayed emergency services, and road offences, which result in inefficient transportation movement and poor safety of commuters. The current traffic control systems that heavily depend on fixed-timer

stoplights and human intervention by the enforcers of the law cannot respond to the actual situation on the road. These delays cause long vehicle queues, irregular traffic movement, and incompetence to prioritize emergency vehicles like ambulances, which in most cases contributes to delay in medical response and higher risks to human life. Also, the existing violation monitoring systems are not automated or accurate, and so law enforcement finds it hard to monitor and catalogue violations like beating the red light, over speeding and use of improper motorcycle gears. There is the increased need to have an intelligent system that will streamline the traffic movement, enhance the response time to emergencies, and effectively identify the road offenses because of the lack of an integrated, data-driven, and automated traffic management system.

General Problem

The inability of conventional traffic systems to manage increasing urban traffic congestion effectively led to delays, environmental pollution, and compromised road safety in rapidly urbanizing areas.

Specific Problem

Delayed Emergency Response

Conventional systems fail to prioritize emergency vehicles at intersections, leading to longer response times.

Violation Detection Inefficiency

Current violation detection systems lacked real-time intervention capabilities, which resulted in unaddressed or delayed actions against traffic violations.

Traffic Congestion Management

Traditional traffic management systems lack real-time monitoring and data analytics capabilities, making it difficult to effectively observe and manage traffic congestion.

Overview of the Current State of the Technology

For decades, metropolitan regions have struggled with traffic, emergency vehicle delays, and traffic offenses. Traditional management systems, which relied on fixed traffic lights and human action, failed to adapt to real-time traffic situations. These systems were not designed to solve the problem of prioritizing emergency vehicles, which resulted in delayed responses and other critical situations. Lastly, identifying traffic offenses such as red light running and speeding had traditionally relied on manual enforcement or static cameras, which were not always reliable or successful.

The first solutions in traffic management involve using Adaptive Traffic Signal Control Systems (ATCS) to reduce traffic. They used sensors on the road to detect vehicles and adjust the timing of traffic lights. While this improvement provided some ease from traffic, these systems were limited by their reliance on local data and a lack of capacity to optimize traffic flow on a city-wide scale. Likewise, Adaptive Traffic Signal Control Systems failed to handle the unique demands of emergency vehicles, which require priority passage through traffic.

Since the earlier solutions were limited by local data and a lack of optimization capacity, the next approach involved using IoT. This solution consisted of sensors and cameras connected via a network and placed throughout the city to monitor traffic in real time. The data from each device was transmitted to a central system that could change or adjust the traffic lights and reroute vehicles. This solution led to more efficient traffic flow. However, while this system improved general traffic management, it did not fully address the critical issue of ambulance prioritization.

Next, image processing techniques were applied to solve the problem of traffic violation detection. This modern approach used high-resolution cameras with AI algorithms to detect violations like speeding and illegal parking. It operated 24/7 and provided instant feedback to the authorities. However, there were challenges in accurately identifying violations under different weather conditions or in low-light environments.

The recent development in smart traffic management was the use of Artificial Intelligence

(AI). This improvement provided an additional enhancement in managing traffic. Artificial Intelligence (AI) could analyze a broad amount of traffic data, predict congestion patterns, and optimize traffic signals to reduce delays. It could also detect emergency vehicles approaching an intersection and automatically adjust the traffic light. Despite these improvements, implementing these kinds of advancements requires large-scale data processing capabilities.

This research intended to address the gap by developing a Smart Traffic Flow Optimization that focused not only on real-time traffic flow but also on prioritizing ambulances. Other solutions had either focused on general traffic management or specific features like violation detection. The project's goal was to enhance urban mobility while ensuring faster responses for emergency vehicles and improving road safety.

Objectives of the Study

General Objective

The overall aim of the research is to design and develop Smart Traffic Flow Optimization System, which will enhance traffic management by using real-time monitoring of traffic, prioritizing emergency vehicles, and automated identification of violation, thus providing better road safety, reduction of traffic congestion, and assist in the efficient movement of people in the city.

Specific Objective

This study aims to design a Smart Traffic Flow Optimization System that improved the traffic lanes and reduced traffic congestion, particularly during rush hour. The module or device also detected road violations when drivers violated the rules and regulations of the law within the road, ensuring the safety of not just the drivers and passengers of the vehicles, but also the safety of the public. More importantly, the system or device prioritized emergency vehicles by clearing their path and improving their response time to accident locations. By enabling faster emergency responses, the system significantly reduced the severity of accidents and minimized the likelihood of further incidents. By reducing traffic congestion, the study also lessened the travel time of individuals and

ensured smoother traffic flow for everyone. By creating web-based analytics, the system projected data in real time for observation using charts and report logs.

According to Enforcer Empalmado, Julius, an officer of the Bonifacio Global City, the traffic stoplight is set in various techniques that it was programmed to. Firstly, according to the enforcer the stoplight changes depending on the time of day, it is said that in the morning it changes sequence by 4 AM in the morning, afternoon 4 PM there is a change in sequence and the evening the change of stop light interval is ever changing depending on the time. Since the traffic works at different times, the enforcer knows what time the traffic will flow and the stoplight changes according to it.

The enforcer also added that sometimes they interevent in traffic to assist the Vehicles when the stoplight can't handle since the nature of the Stoplight doesn't rely on the programmed to prioritize the most vehicle lane, but instead the stoplight works based on the routes they came from said by the enforcer, According to the enforcer in the BGC he explained that in the lane that came from the EDSA has a time of 3 minutes time sequence to let all the cars pass because EDSA has cars incoming from different area and from Aura to BGC has a time interval of 1 minute and 25 seconds time of sequence to let all the cars pass because of the traffic light in the area of Aura that will handle another set of intersection that also has the same set of rules.

The enforcer added that they do Hand signal a type of technique for each incoming lane for 60 seconds for cars to move forward to have a flow in traffic. It is a way for them to intervene if the flow of traffic cannot be resolved by the stoplight. The way the enforcer demo is that they do a signal that all the cars will go in his direction, each time the hand moves in a forward direction 1 second is counted in to his head. After 60 seconds of counting that lane will be stopped and the other lane will be done the same ensuring that the traffic flow is being handled to reduce the traffic congestion.

In conclusion, the Enforcers don't have any clickers of some sort for them to measure the time interval. instead, The Traffic enforcer from their management sets the time interval according to the route each lane will come from, the enforcer uses a Hand signal technique in the Forward signal to let cars pass through for 60 seconds to ensure traffic flow is being

handled when the legacy stoplight can't handle it.

To achieved this, the Developers focused on the following specific objectives.

The Developers Designed and developed a feature that detected ambulances and adjusted traffic lights to give them a clear path.

The system improved their response time to the location of the accidents. It included a Camera to capture images and utilized the use of OpenCV to analyze those images for detected emergency vehicles.

The Developers Designed and developed a Feature that determined a vehicle has violated a rule or law to ensure both the driver and public safety.

The system made roads safer and more efficient than before. It included a Camera to capture images, and a Raspberry Pi to handle complex tasks such as processing camera data.

The Developers Designed and developed a Web Platform that uses AI-processed data to monitor congestion and analyze violation trends, enabling administrators to effectively oversee traffic management.

The platform received and processed real-time AI-interpreted data from the Arducam Raspberry Pi camera, enabling the system to monitor congestion and detect violations. The processed information was displayed through analytics dashboards, providing visual representations that allowed administrators to continuously oversee traffic flow and manage the system effectively.

Scope and Limitations of the Study

Scope

This study focused on the development of a Smart Traffic Flow Optimization System. The system was created in a bid to solve the traffic congestion problem in urban Metro Manila. It involved equipment and analytical parts that automatically regulated traffic lights to ease the movement of vehicles. It was also the priority of the ambulances, as the system was able to identify their presence with a special device and clear their way. The system included the detection of traffic violators through road cameras to give the authorities the right information. In particular, the system was intended to operate at a two-lane intersection where two opposite lanes are running simultaneously.

Control Unit

Arduino Uno

The Arduino Uno is the microcontroller used in the Smart Traffic Flow Optimization System. It controlled input from the relays to the lightbulbs and interfaced with other modules to coordinate traffic signals as well as relay systems. Its availability of many I/O pins and its capabilities allowed it to control and read several sensors and appliances at once, so data could be instantly processed, and decisions could be made.

Raspberry Pi 5

The Raspberry Pi 5 served as the central processor for the AI-based violation detection system. It ran the algorithms on which Artificial Intelligence had been trained. It processed the camera feed, recognized license plates from images, and communicated with the central control manager via Wi-Fi.

Sensor

Arducam Raspberry Pi

These were used to capture images and to identify violations. The system could sort live

video feeds to detect violations such as a vehicle running a red light, driving over the allowed speed, or motorcycle violations such as incomplete gear equipment. Object Dectetion used the images in real time to identify vehicles, recognize the license plates, and send signals about the violation to an administrator's database for further actions.

Communication Devices

Wires Cabling

This was used to interface all the electronic parts, including the Arduino, relays, lightbulbs, and cameras. It provided the link between different components of the system for both communication and power supply. Wiring was a significant factor in the effectiveness of the traffic management system, as it allowed for accurate data flow.

Wi-Fi Connection

This served as another medium for connecting the device to the system, which enabled the user to access the control panel.

Light Bulb

These were used for displaying signals—Red, Yellow, and Green—to vehicles as well as pedestrians.

Power

Relay

For traffic light control, the relays were used on the circuit to switch the traffic lights based on signals from the Arduino. Whenever the Arduino received data that indicated a change in the signal (e.g., an ambulance or high traffic density), it signaled the relay to change the traffic signal. They translated the low-power output from the Arduino to control the high-power traffic lights.

Generator

This served as the main power source for the machine to ensure uninterrupted processing of the entire system.

Adaptor

This functioned as a power provider to each part of the hardware that was used, including the Arduino, the Raspberry Pi, and other components.

Software IDE

Arduino IDE

This was the software used to program the Arduino UNO. It provided the environment for writing and uploading code that defined the Arduino's interaction with sensors and its traffic light management functionality. It also supported multiple libraries for input sensors and communication technologies.

Raspberry Pi

The Raspberry Pi functioned as a more complex microprocessor that complemented the Arduino. It ran complex algorithms and managed tasks like video processing with the Arducam and database interactions with Supabase. It also acted as a server for the traffic management system, processing input from the multiple cameras, analyzing the data, and efficiently regulating the overall system. It was responsible for inputting data and images into the Supabase database tables.

Supabase PostgreSQL

Supabase served as the centralized database for storing both traffic and violation logs. It stored violation records, including license plates, images, and date/time stamps; traffic flow data such as vehicle counts and signal timings; and reference strings such as violation types and vehicle types. It also provided APIs for the ReactJS website analytics dashboard that was created for data visualization and fetching.

TensorFlow

This was utilized to implement machine learning algorithms that analyzed traffic patterns. By leveraging TensorFlow's real-time processing capabilities, the system ensured efficient traffic flow, prioritized emergency vehicles, and enhanced overall urban mobility.

Visual Studio Code

This software IDE was used to develop the website for the web-based analytics platform, where the database connections were implemented.

ReactJs

For the website analytics, ReactJs was used for website creation and was responsible for projecting data that came from Supabase in real-time onto charts, bar graphs, violation records, and camera feeds.

YOLO

It was an algorithm designed to process images and videos in real-time. It was used as a testing ground for the algorithms that were embedded for the camera, allowing it to perform detections at first glance.

Python

It served as the main language for backend processing and A.I. development. It functioned as the core component that connected the camera to the website for monitoring, violation detection, and data processing for analysis and reporting.

Google Colab

It allowed the team to write and execute Python code in a browser. It served as a test ground for algorithms before they were embedded into the system, enabling development without dependency on local hardware.

RoboFlow

It was used to label and annotate different types of violations and vehicles by drawing

bounding boxes around them. Each box was named and labeled during the training process of the Artificial Intelligence model.

Digital Sound Detector Sensor Module

This was used to detect high-decibel audio signals, specifically the sirens of approaching ambulances. It operated by monitoring environmental noise levels and sending a digital trigger to the microcontroller when sound intensity exceeded a calibrated threshold. This component served as a secondary detection layer, allowing the system to anticipate the arrival of an ambulance and initiate ambulance prioritization protocols even before the vehicle was visually identified by the camera.

Limitations

In terms of the limitations of our study, it was limited in various possible ways.

The study was conducted in a small area. We wanted to start small with our study because implementing our device caused traffic jams in many lanes, as it consumed time and the resources were limited. Due to the issue at hand, we were unable to test our thesis in large-scale areas.

In addition to the limitation, Although the system was developed to improve traffic flow, its real-world implementation was restricted to controlled testing scenarios to avoid potential safety risks to commuters and drivers. This limitation ensured that accident risks during testing were minimized while validating the system's functions.

For our violation detection system, this did not cover all violations but was limited to a minimum to calibrate the reliability of our system in detecting violations committed by cars or motorcycles. Violations such as speeding, red light violations, no helmet, no visors, and incomplete equipment were used for our system only to calibrate whether our algorithms were reliable.

Latency was also observed in the Raspberry Pi due to the frame rate and processing speed during image analysis, which affected real-time responsiveness.

The detection of emergency vehicles was limited to only ambulances. Due to the scope of detecting only the ambulance for better testing and reliability.

Additionally, the system to properly detect vehicles and violations, proper camera angle, adequate lighting conditions, and appropriate pole height were necessary. These factors ensured that the objects were clearly visible and accurately identified by the detection system.

The system was designed to operate on a two-phase intersection, where two opposite lanes share the same traffic light timer (e.g., lanes A and C run together, and lanes B and D run together). This design was chosen because the system does not use dedicated arrow lights for turning. Each direction has two lanes that only allow going straight or turning right, while left turns are not allowed to avoid conflicts with oncoming traffic. Because of this, the intersection can operate safely and efficiently using a two-phase traffic light setup. This setup reduces confusion for drivers and simplifies the traffic flow, making it the most suitable choice for the implemented design.

One of the limitations of the study is the limited power duration of the stoplight system. The prototype uses three Philips LED Downlights rated at 14W (1200 lumens) and nine Philips LED Downlights rated at 10W (1550 lumens), powered by a Portable Power Station rated at 200W/220V. Although the LEDs consume less power than regular bulbs, the portable power station can only supply electricity for about two to three hours of continuous use. This limits the system's operation time during long demonstrations and tests. In future improvements, a higher-capacity power source or a direct power connection may be used to extend the system's runtime.

The system is architected as a local host for deployment rather than utilizing cloud services. This design decision is based on the requirement for real-time performance and reliability for an on-site user. The local host system is very fast because the data does not have to travel over the internet. This gets rid of the waiting time (delay) that the cloud would cause. Also, the local host always works and is very dependable because it does not need a stable internet connection. The cloud system requires good internet and will stop working without it. Because of this, the local host is the best choice for our specific project.

During the testing of the Digital Sound Detector, the researchers used a recorded ambulance siren played through a mobile phone speaker. A major limitation arose due to the low sound intensity produced by mobile devices, which is significantly weaker compared to high-decibel ambulance sirens used in real emergency situations. As a result, the system only achieved consistent sound detection within 1 to 3 meters, with partial detection at 4 to 6 meters, and complete detection failure beyond 7 meters.

Another key limitation is the LM386 sound sensor module used in the prototype. While functional for basic sound-trigger applications, it is not optimized for long-range or low-volume detection. Its sensitivity and amplification range are limited, making the module unable to detect faint or distant siren sounds reliably. This significantly reduced the effective detection range during testing.

Therefore, the combination of low-volume audio from the mobile device and the restricted sensitivity of the LM386 module limited the system's performance. The researchers expect that using an actual high-decibel ambulance siren and a more advanced sensor module would greatly improve real-world detection range.

Review of Related Literature/Studies/Systems

Foreign Literature, Studies, or Systems

According to Nguyen, Le, Phan, and Lai (2022), the researchers developed a video-based red-light violation detection system using deep learning techniques such as object detection, tracking, and Recurrent Neural Networks (RNN). The system was tested on real-world traffic footage to identify vehicles that crossed intersections during red-light phases. Their findings showed that the proposed model achieved high accuracy in detecting violations, even under varying lighting and traffic conditions. The integration of RNN reduced false detections by analyzing vehicle movement patterns across time, ensuring that only true red-light violations were identified. This model demonstrated improved efficiency in monitoring and enforcing traffic laws automatically.

The proposed model focused on automating red-light violation detection through frame-by-frame vehicle tracking and temporal sequence learning. It analyzed each vehicle's trajectory to determine whether it crossed the intersection during a red-light phase. The model's evaluation indicated superior precision and recall compared to traditional video analysis methods.

Synthesis

The study by Nguyen et al. (2022) highlighted how deep learning and temporal tracking can improve red-light violation detection accuracy. That can help our goal in detecting beating the red light.

According to Ghaleb et al. (2023), a very simple and very affordable system explained how to help solve many traffic problems in many cities using IoT technology. The authors designed a system that used sensors to gather real-time traffic information. The data was used to adjust the traffic lights and guide the cars to less crowded routes or roads which reduced waiting times and traffic jams or accidents on the road. The system made decisions based on current traffic conditions, making it much faster, easier, and smarter than any traditional methods. By improving traffic flow on the roads, this IoT system also helped

lower pollution. With fewer cars idling and getting stuck in traffic, carbon emissions decreased, making the city cleaner, clearer, and healthier. The system was flexible and could be used in much larger cities because it was very cost effective and could handle varying traffic conditions. The authors emphasized that using IoT technology could save time, reduce frustration for drivers, and benefit the environment, making it a great tool or device for many cities.

Synthesis

This study aimed to explore how IoT technology could address traffic problems by utilizing a simple and cost-effective system, as highlighted by Ghaleb et al. (2023). Sensors collected real-time traffic data to optimize traffic lights and suggest less congested routes, reducing delays, accidents, and congestion. The system's dynamic approach not only improved traffic flow but also lowered carbon emissions, contributing to environmental sustainability. Its flexibility made it applicable to cities of varying sizes, enhancing urban mobility. This research supported the development of smart traffic optimzation system focused on efficiency, sustainability, and better driver experiences.

According to Agorku, Agbobli, Chowdhury, Amankwah-Nkyi, Ogungbire, Ankamah Lartey, and Aboah (2023), the study developed a real-time violation detection model for motorcycle helmet usage using a deep learning architecture (YOLOv5) and ensemble learning. They trained the model on video recordings and used data augmentation to enhance robustness under varying lighting and weather conditions. Their model achieved a mAP score of 0.5267 on test videos and ranked 11th on the AI City Track 5 public leaderboard.

The proposed model focused on monitoring riders and passengers on motorbikes, classifying whether helmets were worn, and flagging violators in real-time. The emphasis was on deploying in smart-city environments for continuous traffic regulation enforcement.

Synthesis

This research illustrates how real-time computer vision systems, specifically those using YOLOv5 and ensemble methods, can effectively assist traffic law enforcement in this case

by detecting helmet-violations. It complements our own work on violation detection (red-light running, speed violations, helmet usage) by providing evidence that deep learning frameworks are viable and improving accuracy in real-world traffic monitoring.

According to Ben Laoula, Elfahim, El Midaoui, Youssfi, and Bouattane (2023), road traffic accidents caused by traffic violations are a major public health issue that results in loss of lives and economic costs. Their study suggested an incremental road safety strategy to identify high-risk areas and common traffic violations. By analyzing traffic violation data across different districts using statistical tests (Kolmogorov-Smirnov test) and clustering methods (k-means), they identified risky areas and the most frequent violations, such as speeding, improper registration, license issues, not wearing seat belts, phone use, and driving under influence. Their findings showed that some districts had significantly higher violation rates, and that k-means clustering provided the best results for optimizing enforcement strategies.

Synthesis

This research supports the importance of identifying high-risk areas and common violations to improve road safety. It complements our system because by detecting violations like red-light running, helmet use, and speeding, we can provide real-time alerts and enforce traffic rules more efficiently.

According to Pang, J.L. (2023), traditional traffic monitoring systems were often inadequate in processing the extensive volume of traffic image data efficiently and accurately. In this context, intelligent traffic monitoring systems, leveraging advanced computer vision technology, have been developed. These research efforts not only addressed the technical gaps in the field of intelligent traffic monitoring but also provided robust technical support and a theoretical foundation for the development of more efficient and intelligent urban traffic management systems. This framework aimed to enrich the model's comprehension of intricate traffic scenarios while expediting the processing speed to fulfill the requirements of real-time monitoring.

Synthesis

This study aimed to address the limitations of traditional traffic monitoring systems by utilizing intelligent systems, as discussed by Pang (2023). Advanced computer vision technology enhances the system's ability to process large volumes of traffic image data efficiently and accurately. These efforts bridged technical gaps, providing strong support and a theoretical basis for developing smarter urban traffic management systems. The framework improved the understanding of complex traffic scenarios while increasing processing speed to meet real-time monitoring needs. This research aligned with the goal of creating efficient and intelligent traffic solutions for modern urban environments.

Based on Chowdhury et al. (2023) studies about IoT-Based Emergency Vehicle Services in Intelligent Transportation System. The Emergency Management System was a vital component in terms of transportation systems; its primary objective was to Prioritize the Emergency Vehicles in urban areas to the area(s) where the accident was reported. But due to the congestion in traffic in lanes, the prioritization of Emergency Respondents was delayed and could come to a time when the Victim was already Dead. There were many occasions why the EV's were often delayed or late; the main cause was during rush hours when there were many vehicles on the roads which caused traffic jamming mostly in Intersection lanes. According to lead studies, this commonly led to higher death rates. Property damage was also affected and there was a high range of traffic congestion. Based on their use of Related Studies, solutions such as giving high attention by changing the traffic signals automatically when they were spotted by the CCTV or by any enforcers on duty were suggested. Other findings also suggested changing their routes depending on the Traffic information given such as the number of vehicles, flow, and the clearance time when they were about to go to the said place of accident. Researchers addressed this problem by providing a Model of an "Unmanned Aerial Vehicle" which could aid the EV's without causing traffic congestion and being disrupted by other non-emergency vehicles. This model, according to their study, would help EV's select an optimal path by controlling the signal phasing time, ensuring that EV's could reach where the accident happened at a minimum injury before it turned major. This device model would minimize traffic disruption since it was airborne. Their Simulations results indicated that the proposed model would achieve an 8% lower response rate, while the clearance time to achieve that surrounding the area of accident was improved by 12%.

They had proposed an approach in which there was a controller that would receive a notification that there had been an incident in an area. It would dispatch to the said location based on the type of incident that was happening (Highly critical, medium, non-critical). It also provided what type of Emergency vehicle was needed to reach the accident location (Ambulance, police car, fire fighters etc.). It also provided a lot of Codes where each vehicle was prioritized depending on the Accident (CODE A, CODE B, CODE C); it was arranged by the administrator to prioritize what responder was needed the most and what was the least needed. After the location was pinpointed, it would direct through the system and would give the Emergency responders an Optimal Passage in which they would arrive in the shortest time possible (Karmakar et al., 2020). An approach that was needed for this to optimize was turning all the stoplights where the said Emergency vehicle was into Green, allowing all vehicles to go until the Emergency vehicle was out of the lane continuing the flow back to its original flow.

Synthesis

This study focused on using IoT technology to help ambulances get through traffic faster, as discussed by Chowdhury et al. (2023). By automatically adjusting traffic lights and providing the best route, the system reduced delays. It also used drones to guide ambulances, which cut response times by 8% and improved traffic flow by 12%. This approach helped create smarter traffic systems that prioritize ambulances in emergencies.

According to the results acquired after the researcher's simulation, the impact of drone usage showed that EV's response time was faster and more efficient. For short distances, it showed a 30% occupancy rate, while the EV's without the aid of Drones showed a reduction from 408 to 317 and 295 to 285 and 267 seconds. According to their studies, it showed a pattern where they observed 50%, 70%, and 100% occupancy rates in terms of short and long-distance accidents. The lower time travel for EV's with drone support to cross a lane without causing traffic congestion or any delays showed a result of 3%, a minimum for EV to travel across all locations of accidents. It was also said that drones could save up to 5 seconds of EV travel time, which made an impact on assessing all incidents.

The proposed model focused on minimizing the Emergency Vehicles' response time while minimizing the traffic congestion with Non-Emergency Vehicles. It was adjusted depending on the number of interventions with traffic lights. They experimented with the device, and the clearance time was reduced when vehicles were increased, while also monitoring the flow, speed, and density of the road. The results were obtained for a 50% rate of occupancy.

Synthesis

This study explored how drones could help ambulances reach their destinations faster. Drones provided real-time information on traffic, allowing ambulances to avoid congested roads. Results showed that using drones could reduce response times by up to 30%. This research highlighted the potential of drones to make emergency responses quicker and smoother, which improved traffic flow and reduced delays.

According to Pawar, A., Sutar, Y., & N. K. (2024), an AI-based system could be employed to tackle one of the biggest challenges facing civilization: traffic signal delays for emergency vehicles like ambulances and fire trucks. These delays were very dangerous, especially in life-threatening situations where every second counted. The system utilized sophisticated artificial intelligence and video processing capabilities to identify the approach of emergency vehicles in real time. After detection, an emergency vehicle triggered changes to signal timings at junctions to allow for its smooth passage. This entailed changing traffic lights to green for the road in front of the emergency vehicle to enable it to clear the road with ease. In doing so, the system greatly minimized response time and helped emergency responders get to a hospital or an accident scene much quicker. This technology merely needed to connect into a community's existing traffic control system, thus there was no need for extensive additions or expensive equipment. This technology could easily be scaled to a large level. Furthermore, the AI system synchronously observed the traffic density level and adjusted the response depending on the current situation, so the proposed solution was not only effective but also could be utilized by different cities with various traffic intensities. Based on technology integration with municipalities and transportation authorities, this kind of AI-based system could

increase the efficiency of emergency response capacity, thus improve overall public safety and save people's lives.

Synthesis

This study looked at how AI could improve ambulance response times by controlling traffic lights, as described by Pawar et al. (2024). The system used AI to detect ambulances in real time and changed traffic lights to clear the way. It connected easily with existing systems and adjusted to traffic conditions, which helped ambulances get through faster and made traffic management more efficient.

According to Rakesh Bhakre et al. (2025) Traffic management and road safety are significant challenges in urban environments, where violations such as signal jumping, over speeding, and wrong-lane driving contribute to congestion and accidents. Traditional monitoring systems relying on manual surveillance are inefficient and prone to human error. To overcome these limitations, this study proposes an AI-based traffic violation detection system that leverages computer vision and deep learning techniques for real-time monitoring and violation identification. The model utilizes object detection algorithms to accurately recognize vehicles, track their movements, and detect violations such as red-light jumping, helmet absence, and lane indiscipline. Integration with video feed analysis enables automated evidence to capture and alert generations, reducing the need for human intervention. The proposed system ensures high accuracy, real-time processing, and scalability, making it suitable for deployment in smart cities and traffic control systems. This paper discusses the methodology, implementation, and evaluation of the AI-based model, demonstrating its effectiveness in enhancing road safety and enforcing traffic regulations efficiently.

Synthesis

The study by Bhakre et al. (2025) addresses the critical need for an AI-based traffic violation detection system to combat major challenges in urban environments, specifically traffic congestion and accidents caused by violations like signal jumping, over speeding, and wrong-lane driving.

Local Literature, Studies, or Systems

According to Buniel, G. G., & Tantoy, C. P. (2024), the research presented the implementation and evaluation of an adaptive traffic light system prototype in Barangay Luna, Surigao City, Philippines. The system utilized Arduino Mega boards and sensors to detect vehicle presence across three lanes, dynamically adjusting traffic light sequences to optimize traffic flow. Data collected over multiple trials assessed various vehicle detection scenarios, demonstrating that the adaptive system significantly reduced wait times and improved traffic efficiency compared to conventional fixed-time systems. Key findings highlighted the system's ability to prioritize lanes based on real-time traffic conditions, effectively managing situations with varying vehicle detection. Despite certain limitations, such as sensor calibration challenges, the study confirmed the potential of adaptive traffic management systems to enhance urban mobility.

Addressing challenges associated with rapid urbanization and increasing vehicle ownership, the research provided valuable insights into adaptive traffic control solutions for small urban areas. It filled a gap in the practical applications of real-time adaptive traffic signal systems in smaller settings, where traditional static systems often failed to adapt to varying traffic conditions, resulting in inefficiencies like unnecessary delays and increased vehicle emissions. The study offered a scalable solution for other urban areas facing similar traffic management issues, aiming to improve traffic flow and pedestrian safety.

Synthesis

This study aimed to implement an adaptive traffic light system in Barangay Luna, Surigao City, using Arduino Mega boards and sensors to optimize traffic flow by dynamically adjusting signal timings based on real-time vehicle detection. The adaptive system significantly improved traffic efficiency and reduced wait times compared to traditional fixed-time signals, demonstrating its potential to enhance urban mobility despite challenges like sensor calibration. By addressing the limitations of static traffic systems, this research offered a scalable solution for small urban areas experiencing rapid urbanization and traffic issues, with future recommendations for improving sensor accuracy and integrating pedestrian data.

According to De Ocampo et al. (2021), traffic congestion was discussed as a global issue typically seen as a road capacity problem, with traditional solutions like expanding roads and improving public transport often proving inadequate. A UN SDG 2030 report indicated that the persistence of congestion issues was due to fragmented approaches. The study proposed a comprehensive traffic management framework that integrated technical and social elements. Adapted from Johannsen's (2007) six-level control systems, the framework progressed from basic traffic monitoring to integrated governance, emphasizing technology, human interaction, and cross-sector collaboration.

Traffic congestion in cities like Manila remained severe due to infrastructure limitations and behavioral factors, such as inadequate traffic rule education and cultural influences. De Ocampo et al. stressed the need for "whole-of-society" approaches, advocating for multi-stakeholder involvement in planning sustainable transport systems. The Sustainable Urban Mobility Plan (SUMP) aligned with this by prioritizing people and incorporating active transport modes for a more sustainable and socially viable solution.

Synthesis

The study particularly focused on cities like Manila, where infrastructure limitations and behavioral factors such as inadequate traffic rule education and cultural influences—exacerbated congestion. De Ocampo et al. advocated for "whole-of-society" approaches, calling for multi-stakeholder involvement in planning sustainable transport systems. The Sustainable Urban Mobility Plan (SUMP) aligned with this approach by prioritizing people and incorporating active transport modes to create more sustainable, socially acceptable solutions.

This study aimed to tackle traffic congestion by proposing an integrated framework that combines technical and social elements, moving beyond traditional infrastructure solutions. By promoting multi-stakeholder engagement and technology integration, this research sought to provide scalable and sustainable traffic management solutions that could be adapted to cities facing severe congestion challenges, such as Manila.

Benchmarking

FEATURES	Smart traffic flow Optimization system	Emergency management system	AI-based traffic system	Adaptive Traffic light	Traditional traffic monitoring system
Minor Violation	/	X	/	/	/
Responsive time	/	/	/	/	X
Reliability	/	/	/	/	X
Ambulance prioritization	/	/	X	X	X
Traffic Optimization	/	/	/	/	/
Road Safety	/	X	/	/	/

Table 1. *Benchmarking on other made prototypes*

Table 1 compared the prototype with existing traffic systems based on six features: Minor Violation, Response Time, Reliability, Emergency Prioritization, Traffic Optimization, and Road Safety. Traditional systems ranked lowest due to poor response time and low reliability, which caused congestion. AI-based adaptive traffic lights performed best among competitors, effectively managing traffic. The proposed system surpassed both by excelling in all features, prioritizing safety, reducing congestion, and ensuring faster emergency response.

METHODOLOGY

SMART TRAFFIC FLOW OPTIMIZATION SYSTEM

Overview of the Project

The Smart Traffic Flow Optimization System was developed to modernize traditional traffic signal operations by integrating AI-based detection, sensor-driven inputs, and automated prioritization logic. The project focused on replicating real-world traffic scenarios observed during the interview with BGC traffic enforcers, where stoplight timing depends heavily on the route, congestion level, and time of day. To simulate these conditions, the system used a combination of hardware components—such as the Raspberry Pi, Arducam camera, digital sound sensor, and Arduino-controlled relay modules—to gather and process realtime traffic data. The methodology involved training an AI model capable of detecting vehicles, ambulances, and violations, synchronizing the processing between Raspberry Pi and Arduino, and refining the traffic light timing logic based on actual traffic flow behaviors. Through iterative testing, troubleshooting, and adjustments to camera positioning, sound detection range, system delays, and light sequence timing, the project aimed to create a functional prototype that demonstrates how automated systems can handle traffic flow, enforce rules, and prioritize emergency vehicles with greater consistency and responsiveness than legacy stoplights.

System Design Specification

Preparing

The project was developed in a systematic and organized design approach with the intention of making the hardware, software, and AI-based components work together to create one intelligent traffic management platform. The system design commenced with the preparation stage, during which the researchers examined the system requirements, found the required technologies, and resolved the general architecture in terms of the integration of Arduino-powered traffic lights control, Raspberry Pi-powered image processing, and the web-based analytics dashboard. At this step, the team identified the functional requirements of the system, reviewed the literature, created first sketches of the

wiring and hardware plan, and the specifications of the embedded and AI model.

Collecting of Components

Hardware and electronic equipment needed in the system were collected. This entailed the purchase of the Arduino Uno to do microcontroller functions, the Raspberry Pi 5 to process AI and take real-time images, the Arducam camera to monitor the traffic, relay modules to switch the stoplights, LED light bulbs to use in the stoplight prototype, the digital sound sensor to detect the presence of an ambulance, and the wiring, adaptors, and power supplies needed to do it. All the parts were chosen regarding compatibility, functionality, and suitability in the target prototype environment. These components were available, and therefore the researchers were able to proceed with hardware assembly and system integration.

Connecting the Components

In this stage, we assembled the physical prototype, wiring the Arduino to the relay modules, the Raspberry Pi to the Arducam, and the digital sound sensor into the system. The relays were connected to the traffic lights to enable the automated switching, and all the image-processing operations were performed by the Raspberry Pi, which had communication with the Supabase database. The handling of the wires at this phase was very important to prevent short circuiting, unreliable electrical connections, and disruption of testing procedures. The software side, such as Python scripts, the integration of the YOLO model, Arduino code, Supabase database, and API communication with the React dashboard, was also set up by the team. When the components were assembled, they were then housed in a physical structure that is a stable environment to realize a realistic situation of a road traffic intersection.

Testing and Observation

We connected components were switched on and tested in terms of performance. The initial experiment involved the testing of a simple traffic light sequence that was managed by Arduino to ensure the accuracy of timing and the reactiveness of the relay. The AIs of the Raspberry Pi were then experimented on different vehicles, lighting, and sizes to guarantee

successful recognition of ambulances, motorcycles, and cars and proper identification of violations, including speeding and red-light running. They were observed to correct such problems as camera color calibration, delay of frame rates, power variation, and synchronization between Raspberry Pi and Arduino. Several experiments were implemented to check ambulance prioritization, real-time traffic optimization, and proper recording of the violation in the Supabase database. Any discrepancies found during testing resulted in a modification of the code, rewiring, or adjustment of sensor settings and camera angles. The system did this by means of constant monitoring and fine-tuning and reached an optimal functionality level and was able to enhance traffic flow, give priority to emergency cars, and effectively enforce road laws.

Features of Developed System/Project

There are various features that are integrated in the Smart Traffic Flow Optimization System, and which collaborate to enhance traffic management in the urban areas. Real-time traffic flow optimization is one of the main characteristics because the system considers the presence of vehicles and their movement and automatically changes the traffic signal timing to relieve congestion and enhance mobility. The other important characteristic is ambulance priority; the system recognizes emergency vehicles by detecting images and sensing sound. The system can pause the traffic light sequence to allow ambulance free passage through the intersection. This attribute leads to quicker emergency response time and road safety. This system has a completely automated violation detection mechanism besides traffic control, which can identify the common types of violations, including speeding, rolling the red light, and an inappropriate way of dressing the motorcycle. The system logs the information on violations, takes photographs of the offending vehicles, and stores the data in the database to be reviewed by the administration. The logs could be viewed and analyzed on a special web-based dashboard that displays real-time camera footage, graphical analytics, and system alerts. The dashboard gives the administrators a centralized view to follow the traffic behavior and gauge the performance of the system. The system was meant to be used with low latency by deploying it on local hosts so that fast processing and reactivity can be achieved even without an internet connection. The architecture is scalable and modular, so that the system

can be adopted or extended to a larger intersection as required, or other functionalities. All in all, the characteristics of the system prove that it can increase the efficiency of traffic, promote enforcement, and make roads smarter and safer.

In this section, it included the hardware components, software, costing, and activities that were used in our project, Smart Traffic Flow Optimization System. These elements are critical to the system's design, implementation, and successful operation. The software improved traffic flow gave priority to emergency vehicles, and identified traffic offenses, while the hardware components did enable real-time traffic monitoring and control. The project's development was guided from the beginning to end by the specified tasks, and detailed costing will guarantee that the project stays within budget.

Research Design

The research design is a combination of quantitative and qualitative methodology. To seek quantifiable information about the issues and requirements in the traffic, the quantitative section was devoted to survey questionnaires that were filled out by commuters. A qualitative section was comprised of short interviews and open-ended survey questions, to collect opinions and suggestions of the respondents. Meanwhile, the Smart Traffic Flow Optimization System designed with developmental design was developed and improved with continuous testing and modification. With such approaches combined, the research could gather both statistics and valuable information on how to improve the system.

Research Settings

The survey and interviews were conducted in various locations within Taguig City, one of the highly urbanized areas in Metro Manila known for its frequent traffic congestion. This location was selected because it reflects real-world traffic conditions that the proposed system aims to address. The responses gathered from commuters provided credible insights into the daily challenges they face while navigating Taguig's busy roads and intersections.

Participants

The participants in this study were commuters who regularly travel through major roads

and intersections within Taguig City. Their firsthand experiences with daily traffic congestion provided valuable insights into the current traffic conditions and the challenges faced by road users. The information they shared was essential in identifying existing issues in traffic flow and in developing possible solutions that the proposed Smart Traffic Flow Optimization System can address.

Sampling Procedure

Appropriate respondents were identified through purposive sampling. A face-to-face survey was conducted with selected commuters in various areas of Taguig City. The surveys were administered in public waiting places such as bus stops, terminals, and pedestrian zones to ensure that the participants were individuals who regularly experience traffic congestion. This approach ensured that the responses came directly from people who are affected by daily traffic conditions and could provide relevant insights for the study.

Research Instrument

Survey questionnaires and structured interview guides were the primary research tools. The questionnaires were divided into close-ended questions to be analyzed quantitatively, and open-ended questions to be analyzed qualitatively. The interviews were also done face-to-face, which enabled the researchers to seek clarification and obtain more in-depth information. The use of written survey forms was also a guarantee of the accessibility of participants who might not be comfortable with electronic modes of survey.

Data Collection

The surveys and interviews were conducted in Taguig to gather the data. Questionnaires were issued in printed form, with commuters being hand delivered in the questionnaires, which were later collected in person after giving them enough time to answer. Also, short-face-to-face interviews involving selected respondents. Based on the responses received, the researchers could determine how traffic issues commuter. This was what was taken as the basis in enhancing the attributes of the Smart Traffic Flow Optimization System so that it could suit even in real-life traffic scenarios.

Data Analysis

The information gathered from the surveys was analyzed using both Quantitative and Qualitative. The close-ended questions provided numerical data to show common patterns, while the open-ended questions were grouped and interpreted based on common themes. The results showed recurring problems such as slow response during emergencies, ineffective manual traffic control, and the lack of automated violation monitoring. The different opinions shared by the respondents highlighted the need for a smart traffic system and gave useful feedback that helped improve the prototype.

ERD

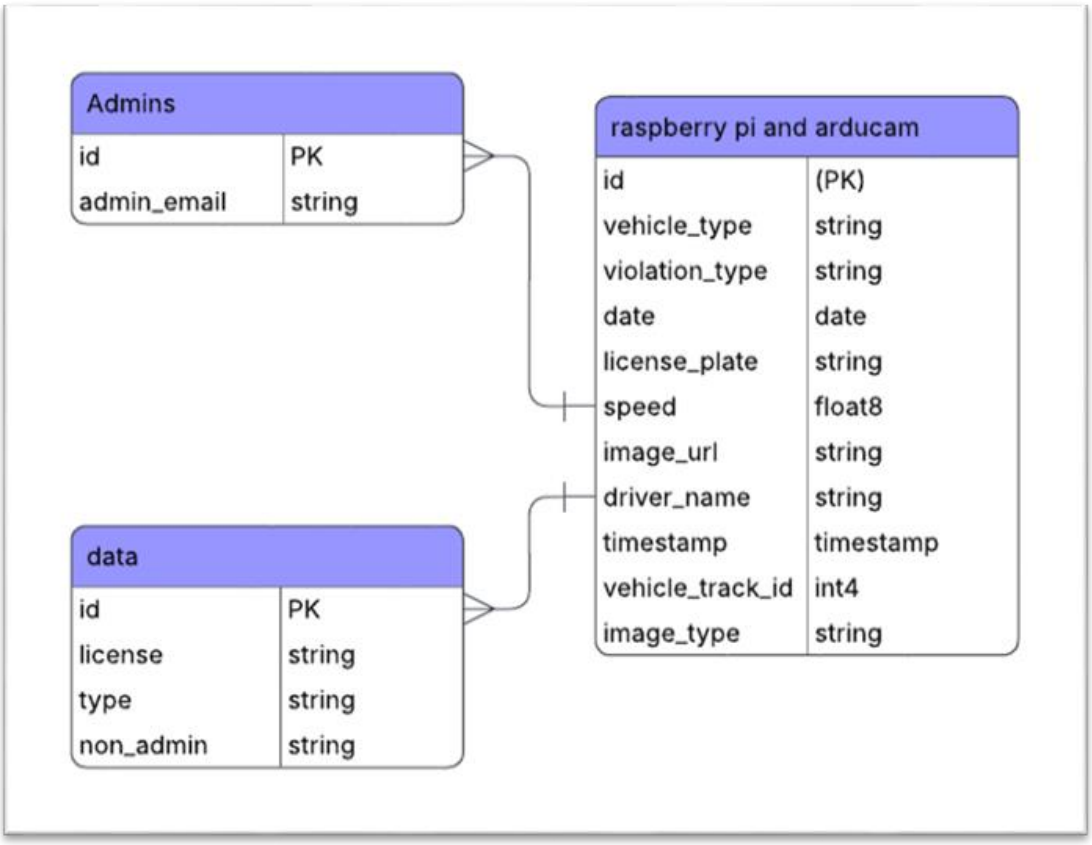


Figure 1. *ER Diagram of the Smart Traffic Flow Optimization System*

This Entity-Relationship Diagram illustrates the database architecture for a traffic violation monitoring and reporting system built using Supabase as the backend database with its

built-in authentication system, and React.js as the frontend framework for displaying and managing the data.

The diagram shows how the different components of your traffic monitoring system are connected. It contains three main tables: Admins, data, and reportlogs.

The admins table manages administrative authentication and control for the system. It stores the administrator's unique identifier and email address, leveraging Supabase's built-in authentication features to handle secure login and administrative privileges. This table is connected to the report logs table, meaning that administrators have oversight of all violation records and can manage the entire system's data.

The data table serves as the registered user repository for the system. When new users register through the React.js interface, their information is stored here, including their license plate number, vehicle type, and the non_admin field which identifies them as regular users. This table functions purely as a user profile database. Registered users stored here have read-only access to view violation reports but cannot submit or modify any data. Each registered user can view multiple reports in the system, but their role is limited to observation only.

The report logs table is the central data storage component where all traffic violation records are logged and stored. This is where the Arducam sends and stores its captured data directly. When the Arducam detects a violation, it transmits comprehensive information to this table including the vehicle type, violation type, date of occurrence, license plate number, speed measurements, driver name identification, and most importantly, the photographic evidence captured by the camera through the image_url and image_type fields. The timestamp records exactly when the Arducam captured each incident, and the vehicle_track_id helps track repeat violations. Because Arducam continuously monitors and captures violations, this table accumulates many records over time as new incidents are detected and logged. These stored logs are then accessible to both administrators (who can manage them) and registered users in the data table (who can only view them through the React.js interface)**.**

In summary, the diagram shows how the system manages administrative control, maintains registered user accounts with view-only access, and stores violation data captured and transmitted by the Arducam. The Arducam hardware sends real-time violation data to the reportlogs table in Supabase, which is then displayed through React.js frontend based on user roles administrators can manage all aspects while registered users can only browse and view the violation records captured by the camera system. All tables are connected logically so that the system can maintain proper access control and efficiently store complete traffic violation information from the Arducam

HIPO

Input

The system collects several inputs to determine the appropriate traffic light behavior. These include real-time data from the Arducam camera for detecting vehicles, violations, and ambulances, as well as audio signals from the digital sound sensor that identifies high-decibel ambulance sirens. Additional inputs come from pre-programmed timing intervals based on the route where vehicles arrive, like how traffic enforcers determine timing depending on lane congestion and time of day. The system also incorporates environmental factors observed during testing, such as camera distance, light conditions, and siren loudness. Together, these inputs allow the system to receive accurate and timely information about the traffic situation at the intersection.

Process

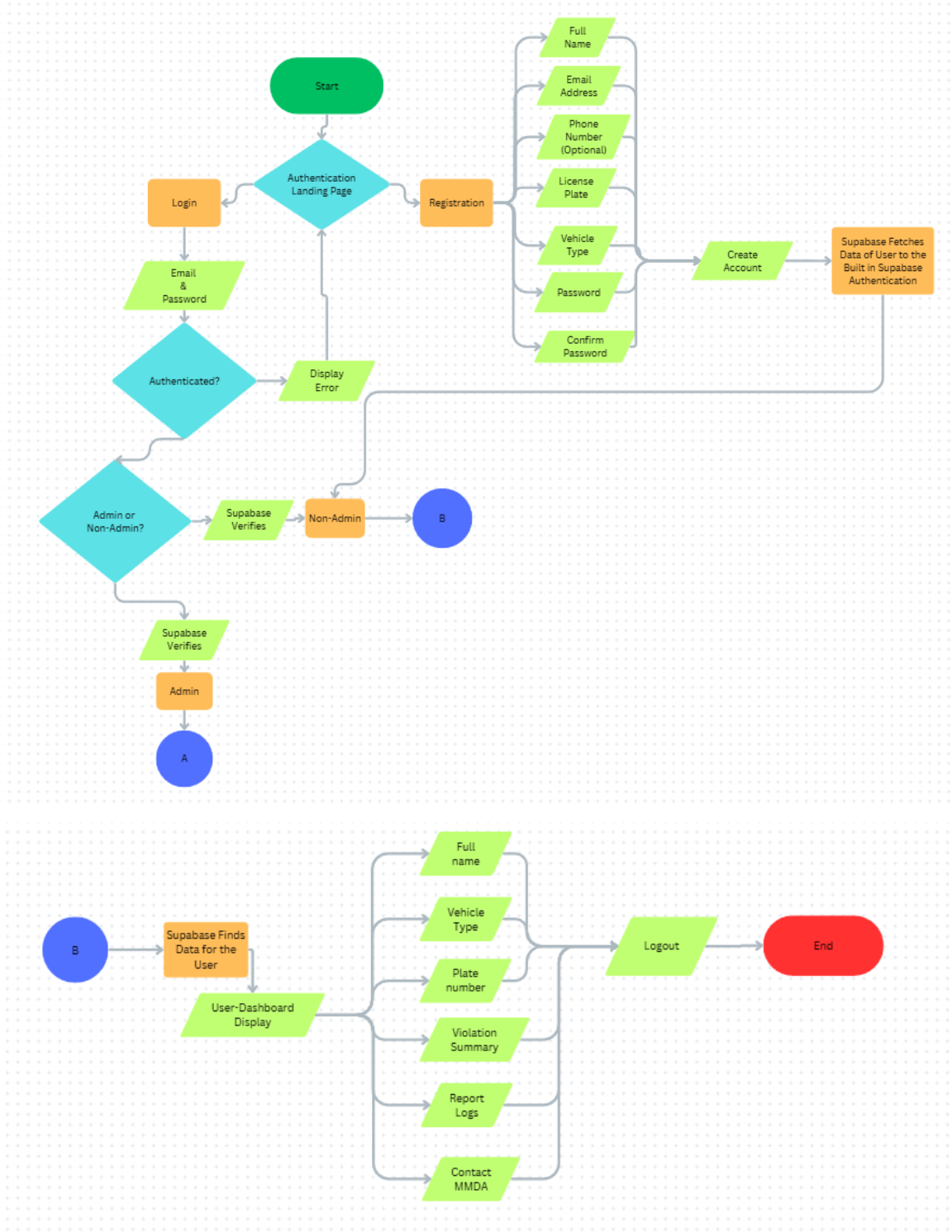
After collecting the inputs, the system processes the data through a combination of AI detection, timing logic, and prioritization rules. The Raspberry Pi analyzes camera frames using the YOLO-based detection model to identify vehicles, violations, and ambulances in real time. Meanwhile, the Arduino interprets signals from the sound sensor and manages the relay controlled traffic lights. The system then evaluates congestion patterns, similar to how enforcers use hand-counting and route-based judgment and determines which lane should receive a longer green light. When an ambulance is detected visually or through sound, the system overrides the normal sequence and extends the green light for the

ambulance lane. Throughout this process, the system synchronizes data between hardware components and ensures timing consistency for accurate stoplight transitions.

Output

Based on the processed data, the system produces several outputs that replicate real-world traffic management actions. The primary output is the automatic switching of the stoplight signals, where green, yellow, and red intervals adjust according to traffic density or ambulance priority. During ambulance detection, the system outputs an extended green signal for the emergency lane while restricting other directions, mirroring how human enforcers intervene to clear the road. Additional outputs include recorded logs stored in Supabase, such as detected vehicles, violation snapshots, timestamps, and system response times. These outputs allow the system to manage traffic flow more efficiently, document violations accurately, and demonstrate near real-time reactions suitable for real intersection scenarios.

Process Flowchart



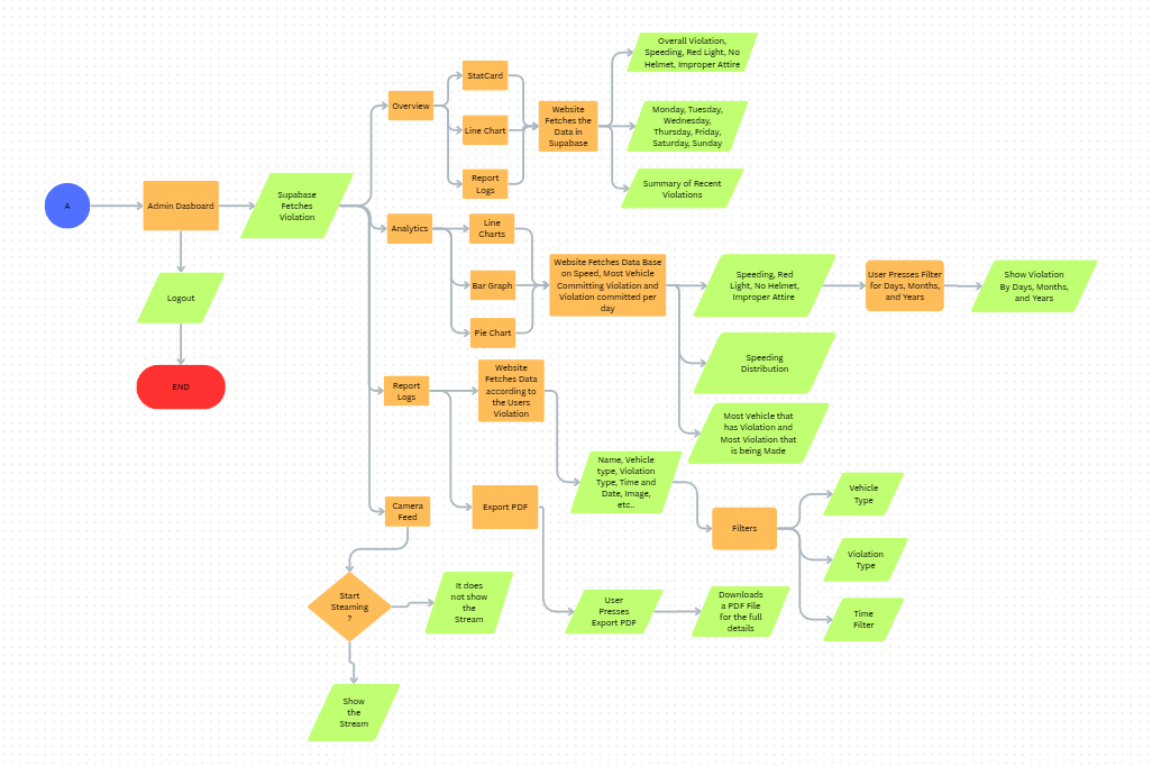


Figure 2. *Process Flowchart of the Smart Traffic Flow Optimization System*

This diagram illustrates a Traffic Violation Management System

The Traffic Violation Management System for MMDA is a web-based platform that authenticates users through Supabase and directs them to role-specific dashboards—administrators access comprehensive management tools while regular users view their personal violation history. The system handles user registration with vehicle information, processes login credentials, and maintains real-time violation data including speeding, red light violations, and helmet infractions. Administrators can analyze violations through multiple visualization tools (charts, graphs), monitor live camera feeds, filter records by various criteria (date, type, location), and export detailed PDF reports. All user sessions are securely managed with proper authentication and logout functionality, while Supabase provides backend support for data storage, retrieval, and synchronization throughout the system.

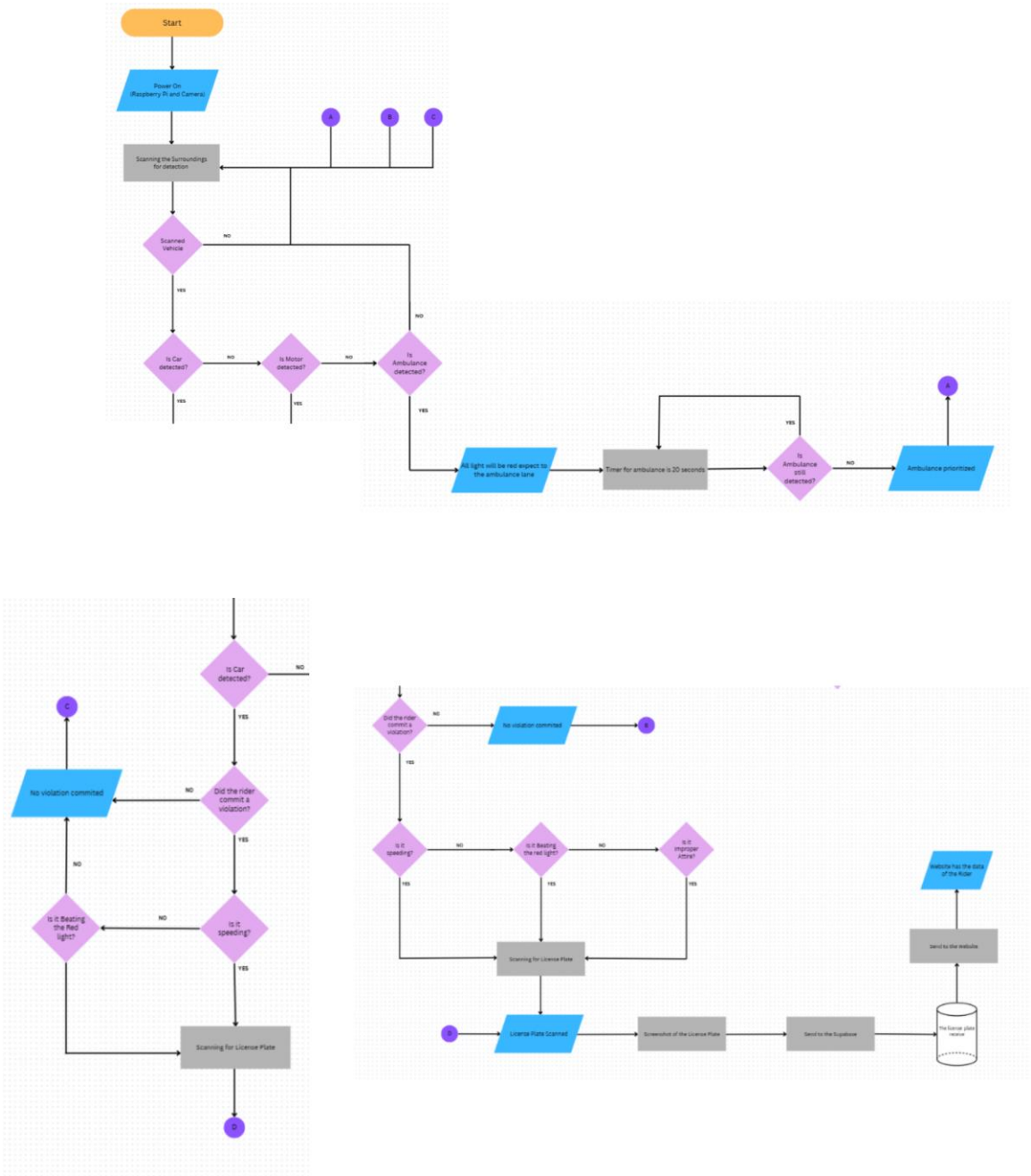


Figure 3. *Process Flowchart of the Stoplight of the Smart Traffic Flow Optimization System*

This flowchart illustrates an automated traffic enforcement system utilizing a Raspberry Pi microcomputer equipped with camera technology to monitor the raodway and detect vehicles. Upon vehicle detection, the system executes a series of verification to classify the

vehicle type whether it is automobile or motorcycle and identify potential traffic violations, including failure to wear protective helmets, excessive speed, red light infractions, and non-compliance with proper attire regulations. When violations are identified, the system gives a character recognition to capture license plate information, documents the incident through photographic evidence, and transmits comprehensive violation data to both the Supabase database and the web-based management platform for official processing and enforcement actions. Additionally, the system incorporates an ambulance vehicle detection protocol that activates traffic signal priority controls, providing ambulances with a 20-second clearance window to facilitate unimpeded passage through intersections during emergency response operations.

Block Diagram

Hardware

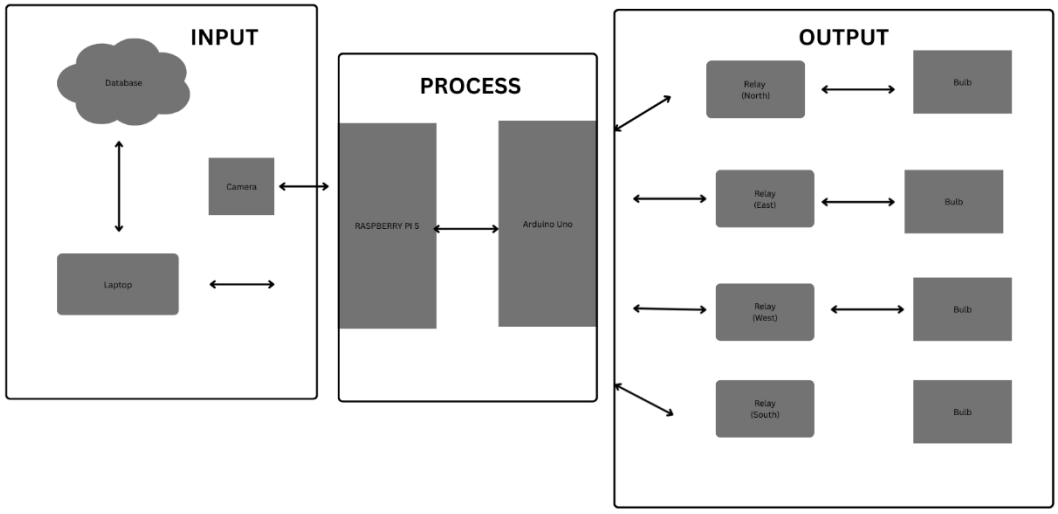


Figure 4. *Block Diagram of the Prototype*

The figure illustrates the Input–Process–Output (IPO) model of the Smart Traffic Management System. The input consists of data from the camera, laptop interface, and database, which provide real-time traffic information and system commands. The process is handled by the Raspberry Pi 5 and Arduino Uno, where image processing, decision-

making, and traffic control logic are executed. The output includes the activation of relays and bulbs corresponding to the North, East, West, and South directions, representing the operation of the traffic lights based on analyzed traffic conditions.

Software

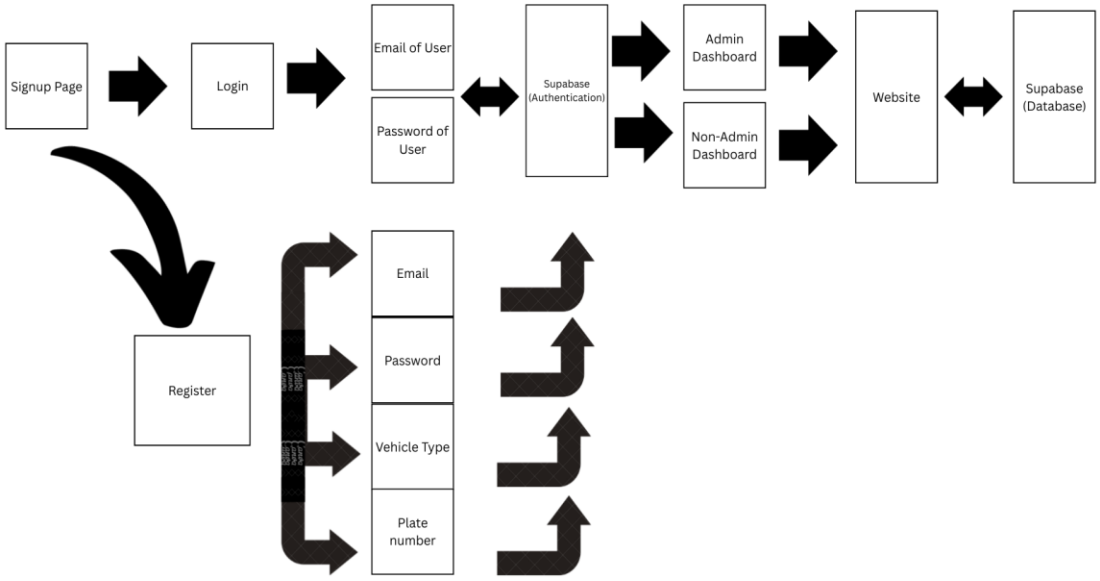


Figure 5. *Block Diagram of the Website*

The block diagram shows a vehicle management system with two main user flows.

New users start at the Signup Page and can either login directly or complete Registration by providing their Email, Password, Vehicle Type, and Plate Number. All user data and authentication are managed through Supabase, which serves as the central database and authentication service.

During Login, users enter their credentials, and Supabase verifies their identity. Based on their role, users are directed to either the Admin Dashboard (for administrators) or Non-Admin Dashboard (for regular users). Both dashboards connect to the main website, which communicates bidirectionally with the Supabase Database to retrieve and update user information, vehicle records, and system data in real-time.

The arrows show how data flows from signup through authentication to role-based dashboard access, with continuous interaction between the website interface and backend database for seamless system operation.

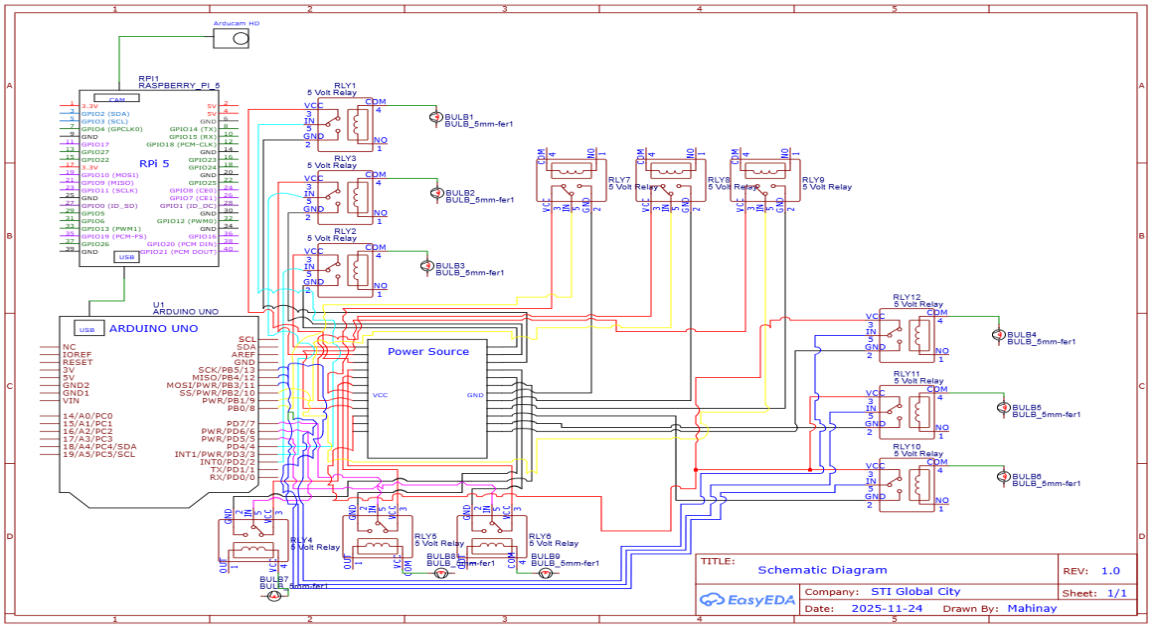


Figure 6: *Schematic Diagram of the Prototype*

This Schematic Diagram represents the overall wiring of the Smart Traffic Flow Optimization System. These wires show the connections of each component of our prototype.

Hardware and Software Requirements

Hardware

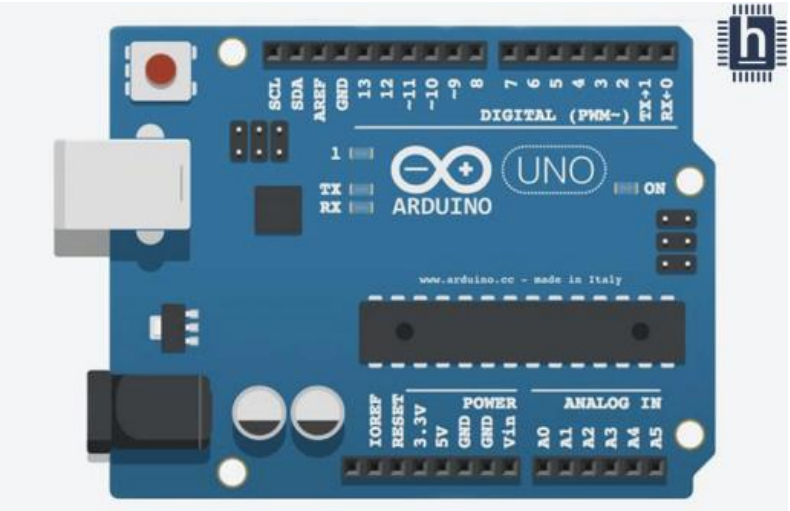


Figure 7: *Arduino Uno – source image* www.hackerearth.com/blog/developers/a-tour-of-the-arduino-uno-board

The Arduino served as the primary microcontroller for handling low-level controls in the prototype. The Developers used it to read sensor inputs, manage timing logic, and trigger output devices such as relays and indicator lights. The Developers selected the Arduino because of its reliability, ease of programming, and compatibility with modules commonly used in prototyping. During development, it allowed us to quickly upload and test logic for traffic light intervals, ambualances overrides, and communication with other components. The Arduino worked alongside the Raspberry Pi, with the Pi acting as the main processor and Artificial Intelligence for sensing Traffic, Ambulances, and Violations while the Arduino handled device-level actions such as Traffic Light Control.



Figure 8. *Portable Station 200w/220v – source image* https://shopee.ph/Power-Station-200W-220V-Portable-78800mAh-Big-Capacity-Powerbank-Multi-Function-Solar-Generator-i.1480154765.28180633733?sp_atk=da4f2d6d-6e1b-45bd-8798-deaf3e46514f&xptdk=da4f2d6d-6e1b-45bd-8798-deaf3e46514f

The generator provided a stable power supply for the system during testing, especially when simulating real-world roadside conditions. It ensured that the prototype could continue running even during power interruptions, which allowed us to verify system behavior under fluctuating voltage conditions. The Developers used the generator mainly to test the reliability and continuity of our hardware setup, ensuring that critical functions such as lighting and camera modules stayed operational.



Figure 9. Relay – *source image* https://forum.arduino.cc/t/difference-between-using-relay-alone-or-relay-module/539550

The relay module was used to control the traffic light signals in the prototype. It acted as the switching device that allowed the Arduino to turn each traffic light represented by bulbs on or off based on the system's programmed timing. The relay became especially important during the ambulance prioritization feature. When the system detected an approaching ambulance, the Arduino activated the relay channels to override the normal traffic sequence and switch the lights to green on the ambulance's lane while forcing the other directions to red. The Developers used relays because they safely isolate the low-voltage control signals of the Arduino from the higher-voltage load of the traffic light indicators. During development, the relay allowed the Developers to physically simulate real-world traffic light changes and test how fast the system could shift to ambulance priority mode when an ambulance was detected.



Figure 10. *Arducam Raspberry Pi – source image* https://shop.scorpion.vision/products/arducam-12-3mp-hq-camera-with-cs-lens-for-raspberry-pi

The Arducam was used as the main imaging/sensor device for capturing real-time images, traffic flow, vehicle violations, and ambulance detection. The Developers chose this camera because it integrates smoothly with the Raspberry Pi, provides high-quality image output, and supports low-light performance important for simulating round-the-clock monitoring. During development, the Arducam was used to test our image processing pipeline, plate detection attempts, and the logging system. Image frames captured by the Arducam were sent to the Raspberry Pi for processing and storing on the database.



Figure 10: *Wire – source image* https://promiseelectric.com/electrical-wire-colors/

Wires were used for all electrical connections between the modules, including power distribution, sensor-to-Arduino connections, relay outputs, and communication lines. The

Developers used both jumper wires and insulated power cables depending on current requirements. During development, wiring played a critical role in ensuring stable signal transmission, minimizing noise, and avoiding short circuits. Proper wire management also allowed us to troubleshoot faster and ensure the reliability of the overall hardware setup.



Figure 11: *Adaptor 12V 2A – source image* https://shopee.ph/5V-9V-12V-1A-2A-3A-5A-AC-DC-AdaptorCharger-Power-Supply-for-CCTV-Security-adapter-i.180027163.8521061874

The adaptor served as the regulated power source for the Raspberry Pi and some low-voltage modules. It converted AC power from the wall outlet into a safe DC voltage required by the devices. The Developers chose an Adaptor 12 Volts 2 Amplifier adaptor to ensure stable current delivery, as unstable power can cause the Pi to reboot or the camera to malfunction. During the testing phase, different adaptors were used to determine which delivered consistent voltage under load, ensuring uninterrupted operation of the monitoring system.



Figure 12: *Light Bulb – source image* https://kmlighting.my/products/philips-eridani-led-downlight-dl190b

The light bulbs served as the physical representation of traffic signals or status indicators within the prototype. Through the relay module, the Arduino controlled when each bulb would turn on or off depending on the traffic management logic. The Developers used light bulbs instead of LEDs in some parts to simulate real-life visibility and load behavior. During development, the bulbs helped evaluate the timing accuracy, ambulance response switching, and system synchronization between Arduino and Raspberry Pi.



Figure 13: *Raspberry Pi – source image*

https://www.raspberrypi.com/products/raspberry-pi-5/

The Raspberry Pi functioned as the main processing unit and AI inference device for the system. It handled higher-level tasks such as running Python scripts, processing image data from the Arducam, performing AI-based detection, managing database communication, and linking the system to the web dashboard. The Developers chose the Raspberry Pi because it provides enough computing power to run lightweight machine learning models locally, offers flexible GPIO support, and integrates smoothly with both the Arducam and Supabase backend.

During development, Raspberry Pi served as the central hub for intelligence and decision-making. It processed camera frames in real time, used the integrated AI model to detect vehicles and identify potential violations or ambulances, and then generated system responses based on the model's output. The Pi sent all recognized data such as timestamps, violation types, and captured images to the Supabase database for storage and visualization on the React dashboard. It also communicated with the Arduino when low-level hardware actions were required, such as activating specific relay channels during ambulance prioritization or traffic light changes.



Figure 14. *Digital Sound Detector Sensor Module*- https://www.makerlab-electronics.com/products/digital-sound-sensor-detector-module?srsltid=AfmBOorbzEVM-qy1mLq1PfRfkrfTcxfhcVJuTWZl5GnQ-GlPlbg_VHnd

The Sound Sensor module served as a secondary detection device designed to identify the presence of ambulances through audio cues, specifically sirens. The Developers used it to capture decibel sound waves, allowing the system to detect an approaching ambulance even if it was not yet fully visible to the camera. The Developers selected this sensor because of its adjustable sensitivity and digital output capabilities, which allowed for easy integration with the microcontroller to distinguish loud sirens from ambient traffic noise. During development, the sensor was calibrated to trigger a signal only when sound levels exceeded a specific threshold, helping to verify the visual detection data and ensure faster response times during ambulance prioritization. It communicated directly with the Arduino to send an interrupt signal, ensuring the traffic lights could switch to the ambulance sequence immediately upon detecting a siren.

Software



Figure 15. *Arduino IDE – source image* https://iconduck.com/icons/151740/arduino-ide

The Arduino IDE was used to write, compile, and upload the control logic for the Arduino microcontroller. It allowed the Developers to develop and test the traffic light timing code, ambulance override behavior, and relay activation sequences. The Developers used this IDE because of its simplicity and compatibility with a wide range of sensor libraries. During development, the Arduino IDE also enabled rapid iteration the Developers could easily tweak delays and relay switching logic to refine the ambulance prioritization feature.

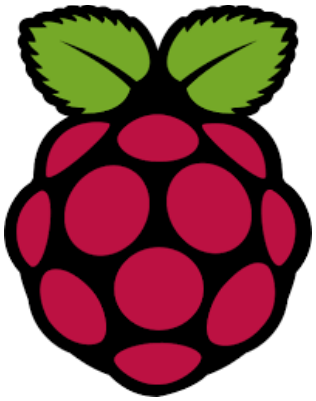


Figure 16. *Raspberry Pi IDE – source image*
https://iconduck.com/icons/14184/raspberry-pi

The Raspberry Pi IDE was used as the main environment for writing and executing Python scripts on the Raspberry Pi. This included code for capturing camera frames, running inference using the AI model, logging data, and sending results to the Supabase database. the Developers used this IDE because it runs natively on the Pi, supports real-time debugging, and provides a straightforward interface for managing Python packages required for AI processing. It helped streamline the development of Pi's core automation tasks.



Figure 17. *Supabase – source image* https://logowik.com/supabase-icon-logo-vector-71414.html

Supabase serves as our backend database, authentication system, and real-time data storage

solution. It stored vehicle violation logs, timestamps, Users Information such as Email, Plate number, Vehicle type, detected plate numbers, and image URLs captured by the Raspberry Pi. the Developers used Supabase because it provides a fast, scalable, and easy-to-integrate database without requiring a dedicated server. During development, Supabase enabled real-time synchronization with the React dashboard, allowing the Developers to instantly view new detection logs, system updates, and ambulance events.



Figure 18. *TensorFlow – source Image* https://www.hiclipart.com/free-transparent-background-png-clipartdpfou/download

TensorFlow was used to train, optimize, and export the machine learning model responsible for object detection and ambulance vehicle recognition. It allowed the Developers to process training datasets, convert the model into a format compatible with the Raspberry Pi, and fine-tune hyperparameters for better accuracy. the Developers used TensorFlow because it supports edge-optimized models and integrates well with Python, making it suitable for deployment on the Pi. During development, TensorFlow played a major role in generating the AI model that powers the system's detection capabilities.



Figure 19. *VS Code – source Image* https://upload.wikimedia.org/wikipedia/commons/thumb/9/9a/Visual_Studio_Code_1.35_icon.svg/

VS Code was used as the primary code editor for developing the React web dashboard, API integration scripts, and documentation. the Developers chose VS Code because of its extensions, debugging tools, and built-in Git support. It streamlined front-end development by providing a clean environment for testing UI layouts, handling Supabase client queries, and integrating real-time traffic analytics. It also allowed collaborative development through version control.

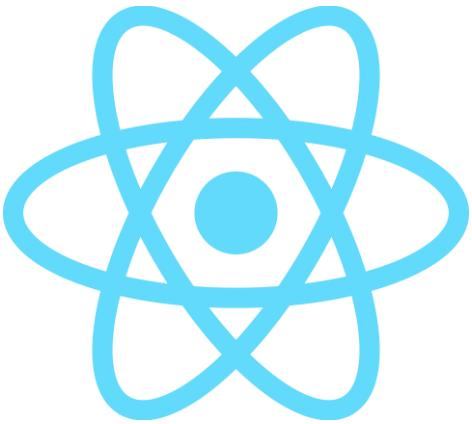


Figure 20. *React JS – source Image* https://medium.com/@pawaryogita309/why-is-react-js-so-popular9b102bd269d9

React.js was used to build the web dashboard that displays Authentication page, real-time traffic data, violation logs, camera snapshots, and analytics. the Developers selected React because it enables fast rendering, reusable components, and smooth integration with our Supabase backend. During development, React allowed the Developers to create dynamic

interfaces such as charts, real-time logs, and monitoring panels, which helped visualize the performance of the traffic system.



Figure 21. *You Only Look Once (YOLO) – source Image* https://www.linkedin.com/pulse/yolo-you-only-lookonce-rahi-saoji-3cglf/

YOLO served as the main object detection architecture for identifying vehicles, detecting violations, and recognizing ambulances in real time. The Developers used YOLO because it offers a high accuracy-to-speed ratio, making it ideal for deployment on devices like the Raspberry Pi. During development, YOLO's model weights were trained and optimized through Google Colab and later deployed to the Pi for inference. YOLO enabled fast, frame-by-frame detection essential for responsive traffic monitoring



Figure 22. *Python – source Image* https://junilearning.com/blog/guide/what-is-python-101-for-students/

Python was the core programming language used for writing the Raspberry Pi scripts, AI inference logic, database communication, and data preprocessing. the Developers used Python because of its extensive library supports such as OpenCV, TensorFlow, Supabase client, readability, and compatibility with edge devices. During development, Python handled frame capture, detection results, automatic logging, and communication between hardware and the backend as well as sending Database that it captured from the Arducam to the Supabase via the “INSERT” command.



Figure 23. *Google Colab – source Image* https://colab.research.google.com

Google Colab was used as a cloud-based environment for training the AI model. It provided GPU acceleration for faster training of YOLO/TensorFlow models and allowed the Developers to preprocess datasets, perform augmentations, and export trained weights. the Developers used Colab because it is free, accessible, and integrates easily with tools like Roboflow. During development, Colab played a major role in training, testing, and refining the AI model prior to deployment on the Raspberry Pi.



Figure 24. *RoboFlow – source image* https://blog.roboflow.com/tag/roboflow-universe/

Roboflow was used for dataset preparation, image annotation, data augmentation, and exporting datasets into YOLO or TensorFlow formats. It streamlined the process of labeling vehicles, ambulances, and violation-related images. the Developers chose Roboflow because it simplifies dataset management and ensures compatibility with common AI architectures. During development, Roboflow helped maintain a clean, structured dataset and allowed us to generate multiple versions of the dataset optimized for training in Colab. Used for labeling each set of Violation that was used and vehicles, for artificial training and algorithm training.

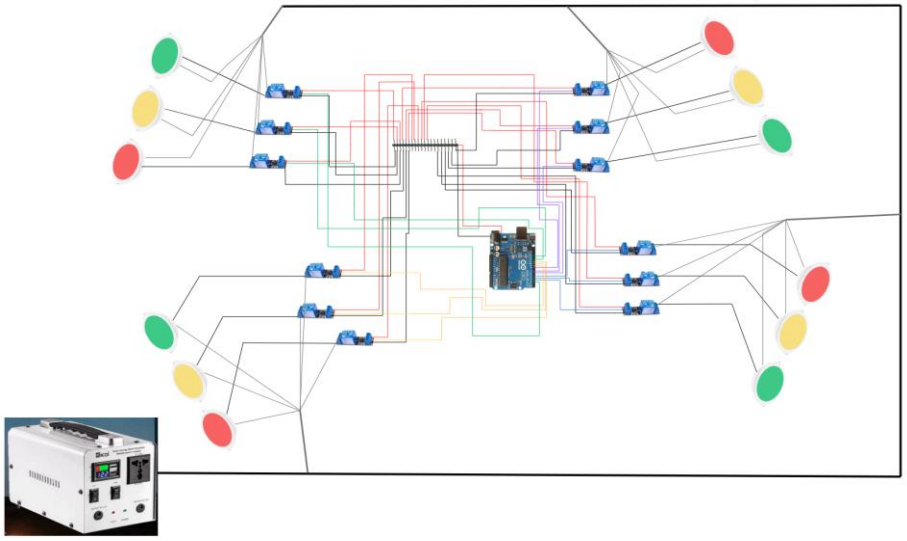


Figure 25. *Wiring Diagram (Stoplight)*



Figure 26. *Wiring Diagram (Central)*



Figure 27. *Wiring Diagram (Connection)*

The wiring diagram illustrates the Smart Traffic Flow Optimization System, detailing the connections between traffic light bulbs, relay modules, an Arduino Uno, a Raspberry Pi 5, and an Arducam camera. The Arduino Uno controls the relay modules, which switch the power to each red, yellow, and green light bulb at the intersection. The Raspberry Pi 5, powered by a power bank, handles real-time image processing and vehicle detection through the Arducam camera. A generator supplies power to the traffic lights and relays, ensuring consistent operation. This integrated setup allows the system to optimize traffic flow, prioritize emergency vehicles, and manage signal timing dynamically.

Architectural Design of Prototype Device

Hardware Design

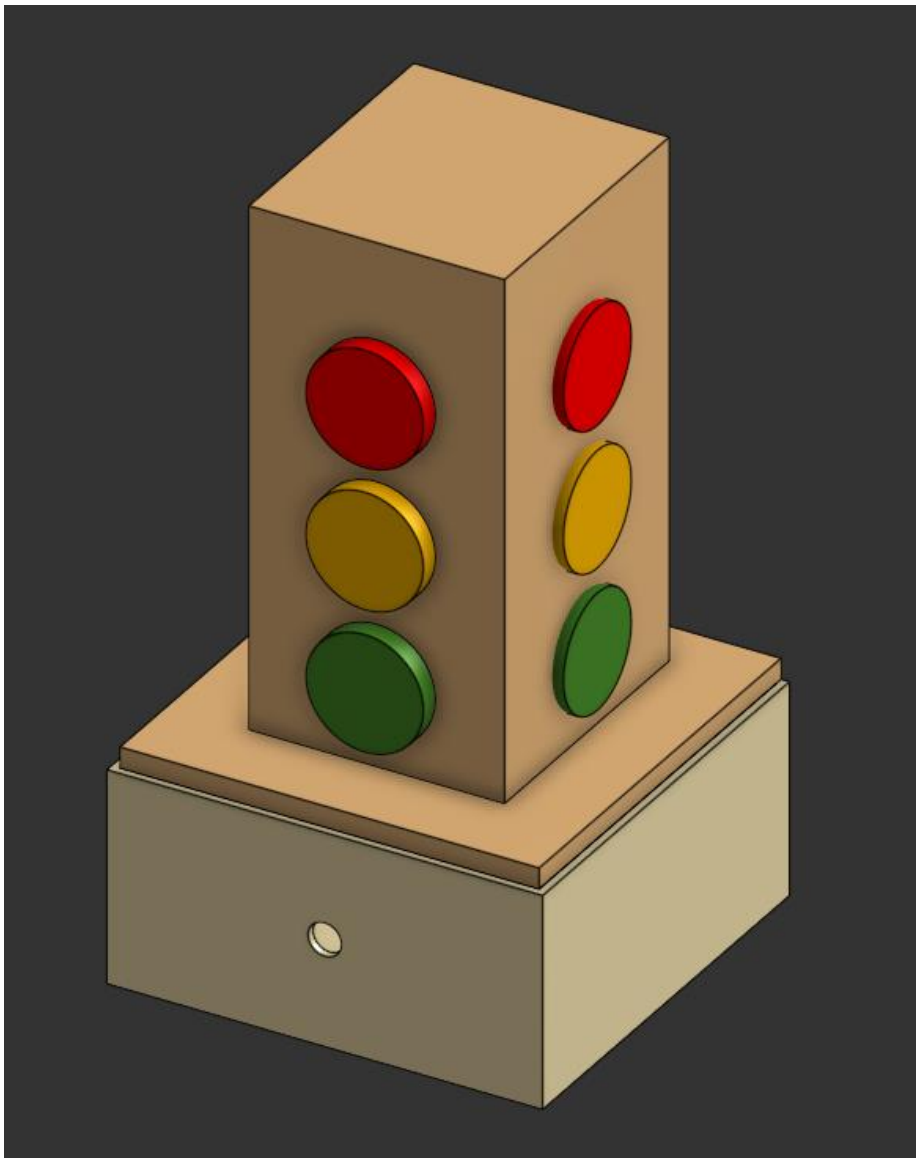


Figure 28. 3D model of Assemble Stoplight (Isometric View)

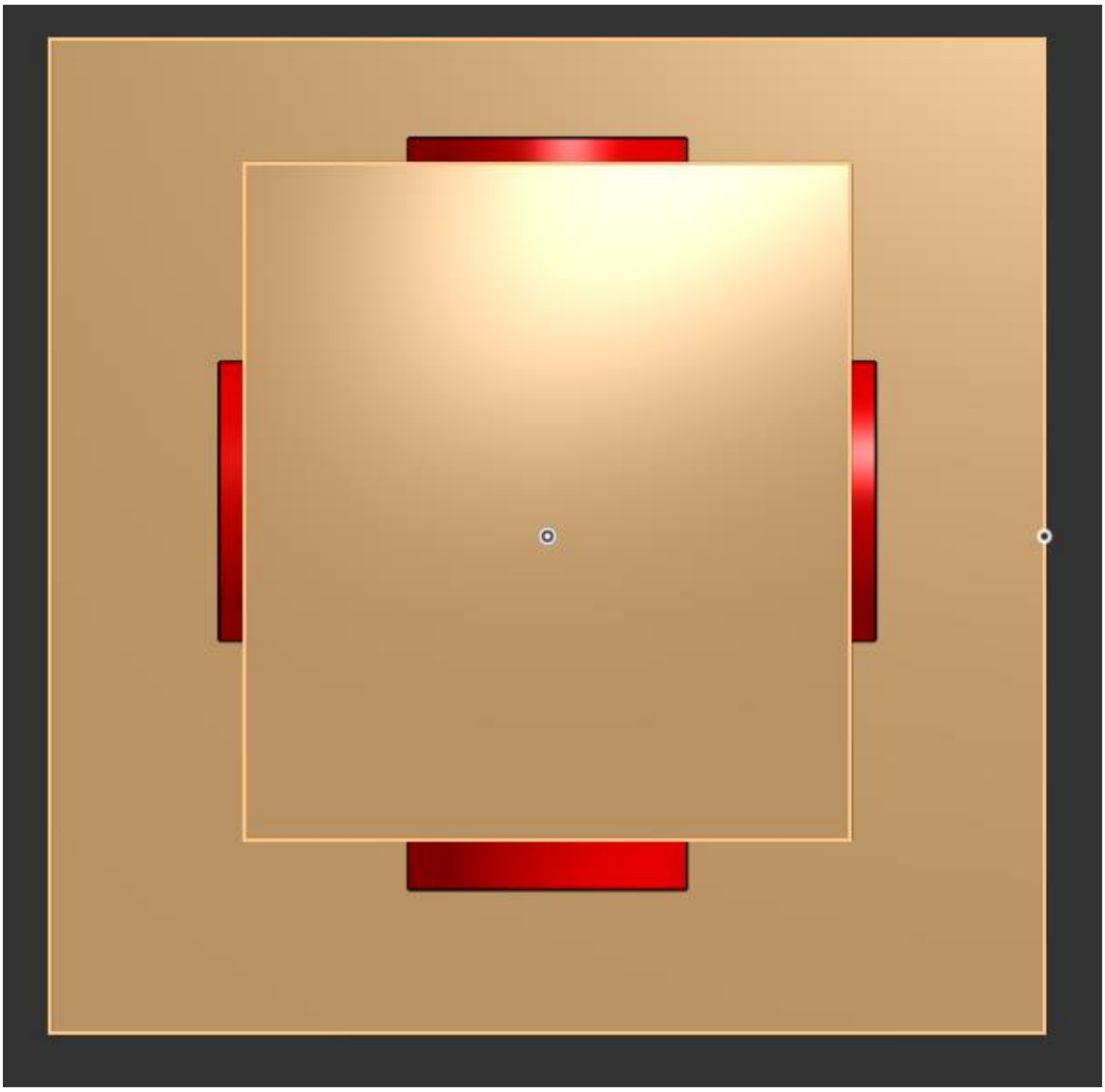


Figure 29. *3D model of Assemble Stoplight (Top View)*

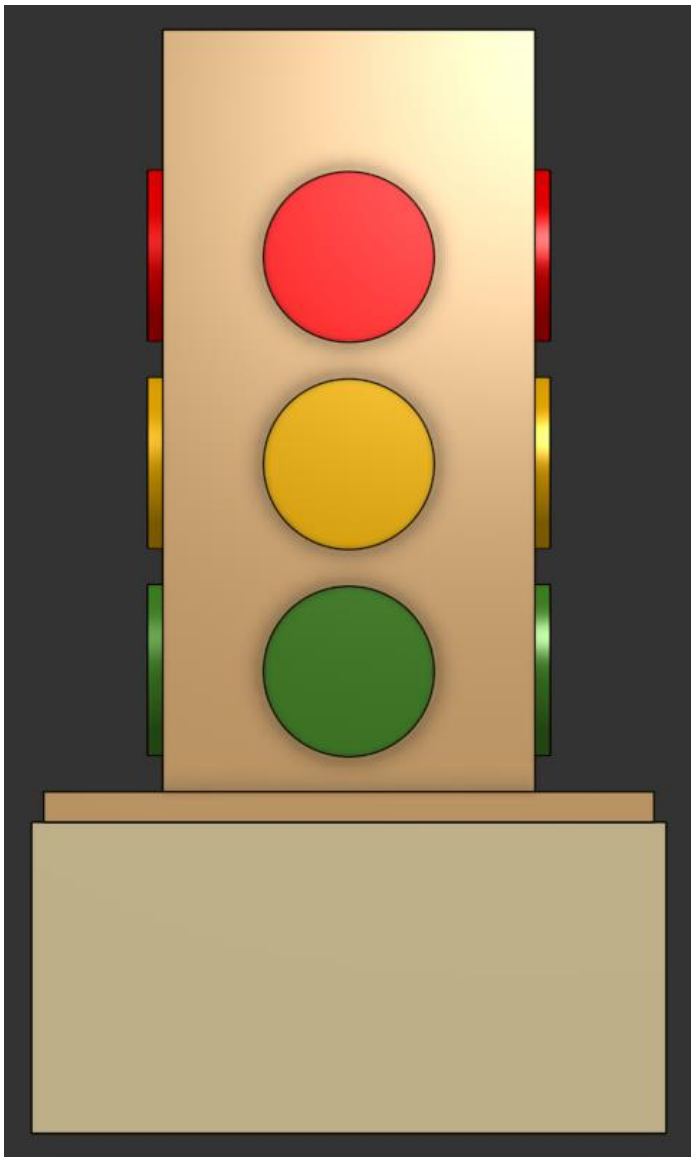


Figure 30. *3D model of Assembled Stoplight (Back and Side View)*

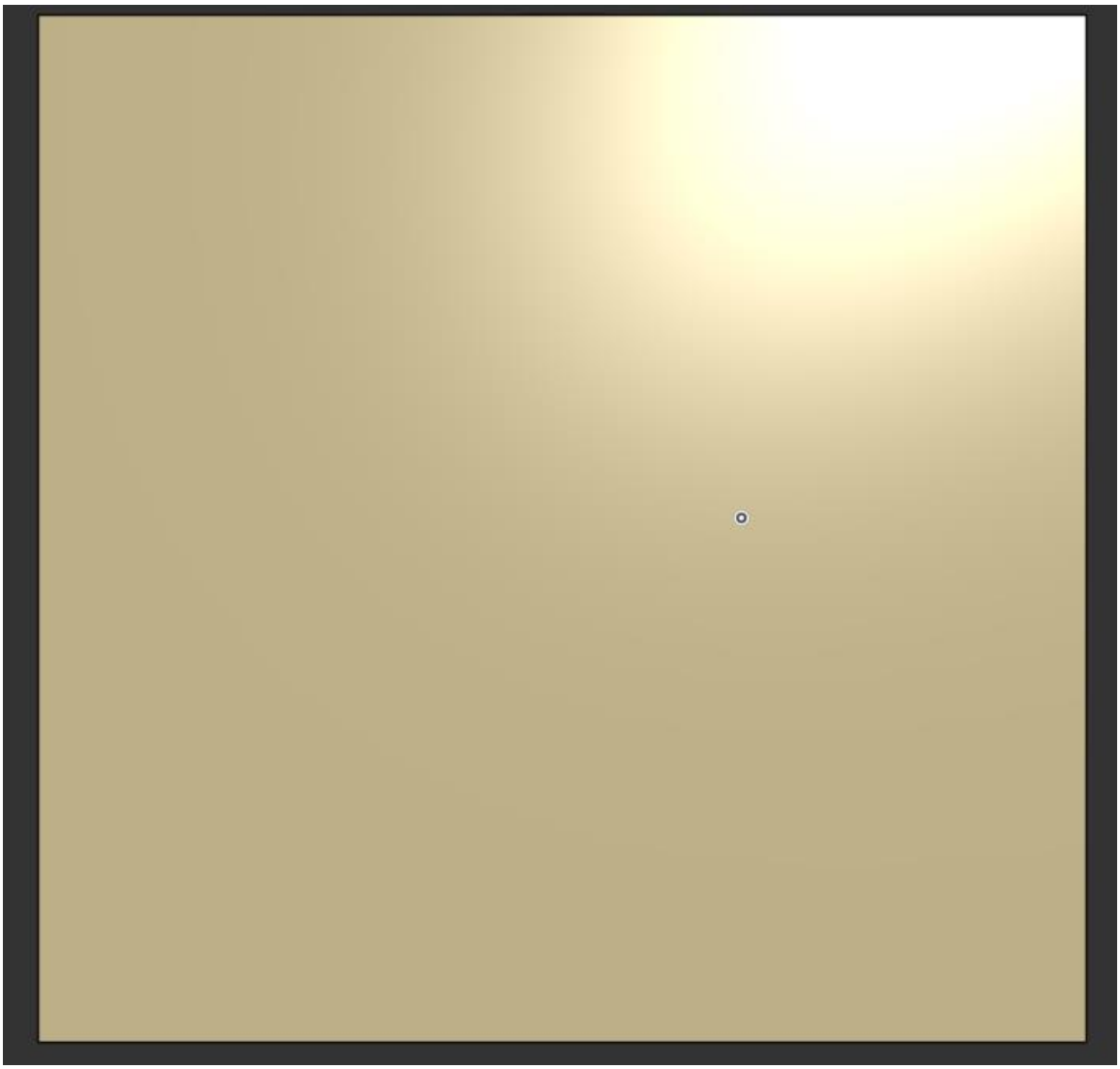


Figure 31. *3D model of the Stoplight (Bottom View)*

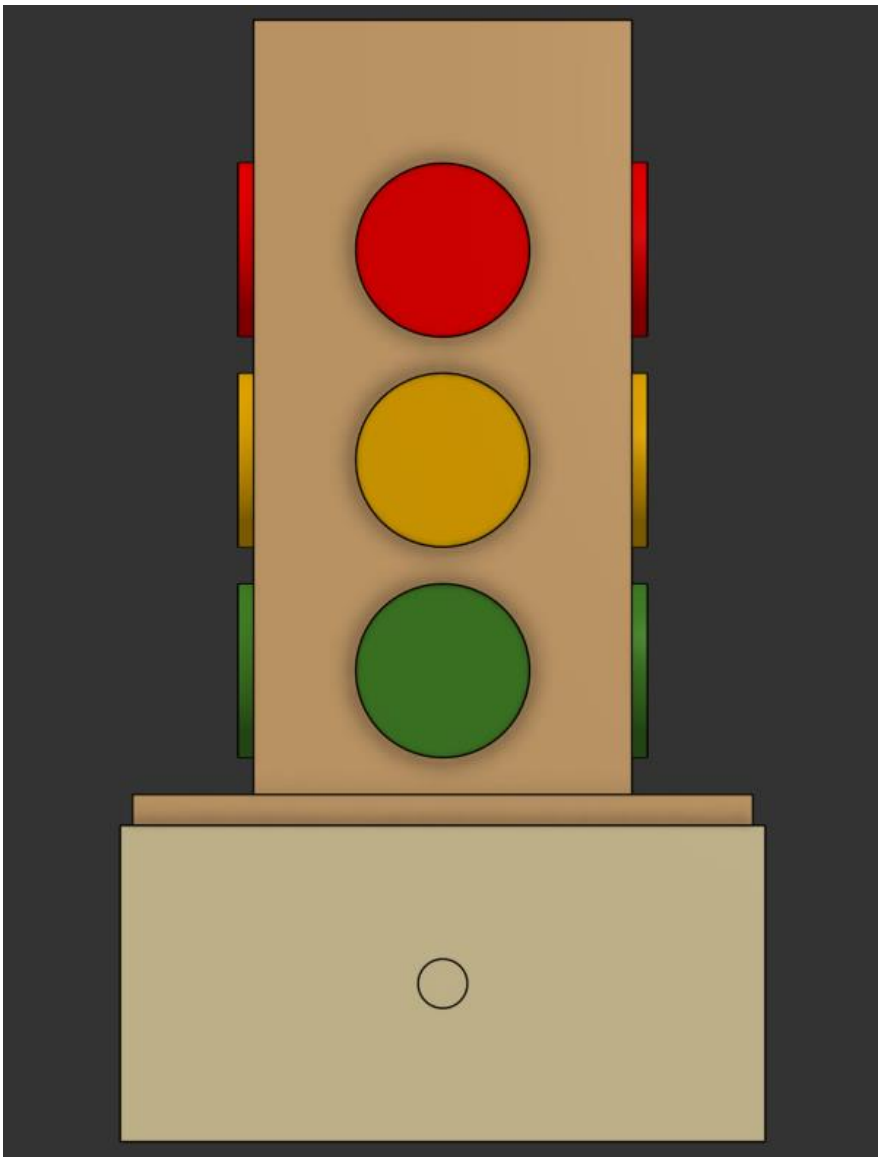


Figure 32. *3D model of Assembled Stoplight (Front View)*

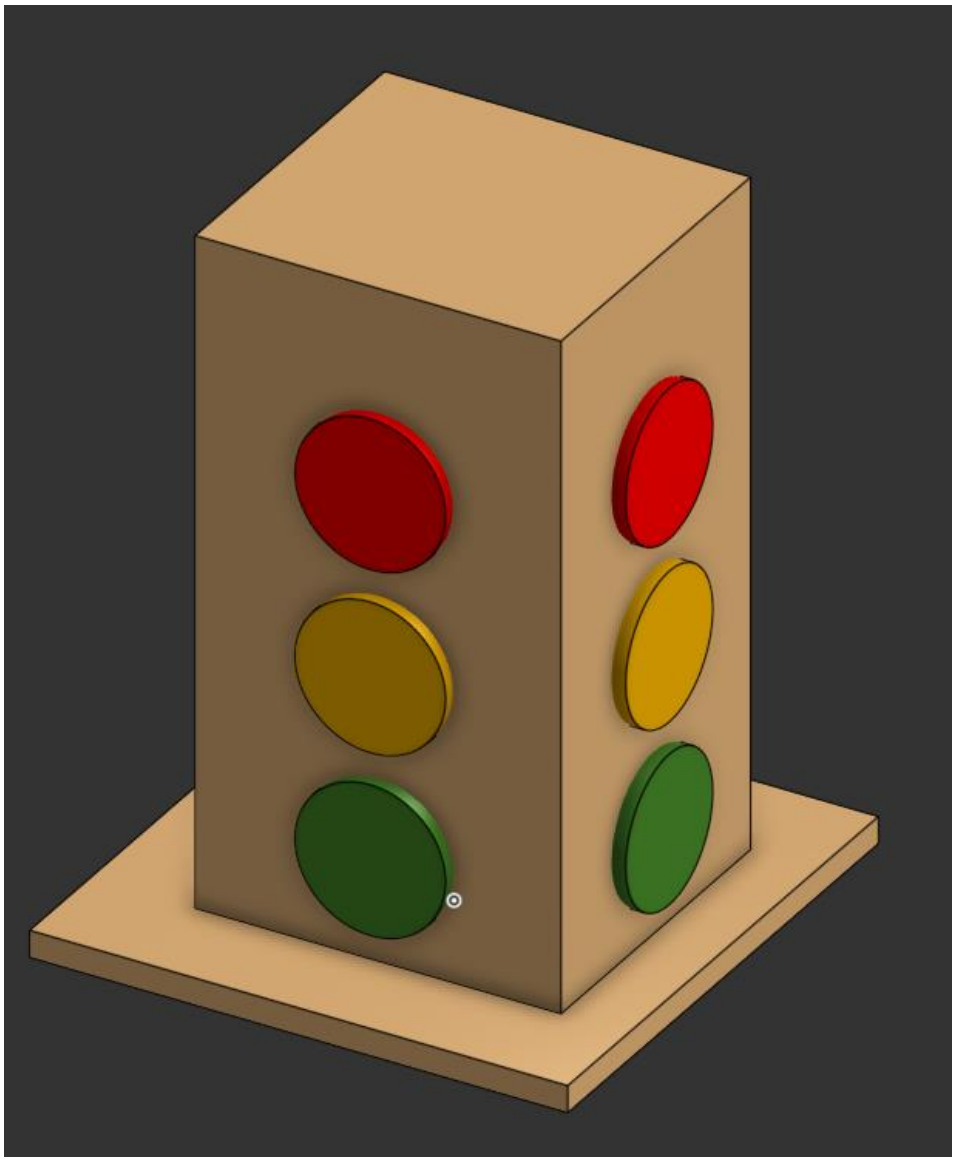


Figure 33. *3D model of the Stoplight (Isometric View)*

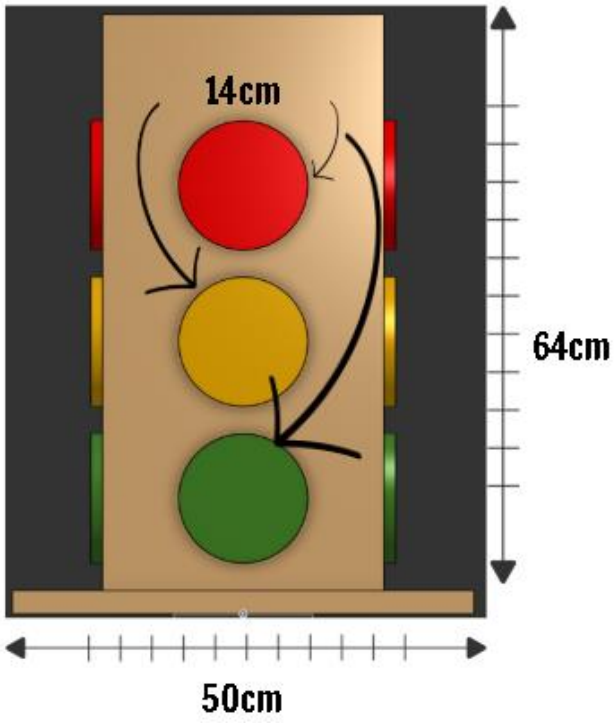


Figure 34. *3D model measurement of the Stoplight (Front, Side, Back View)*

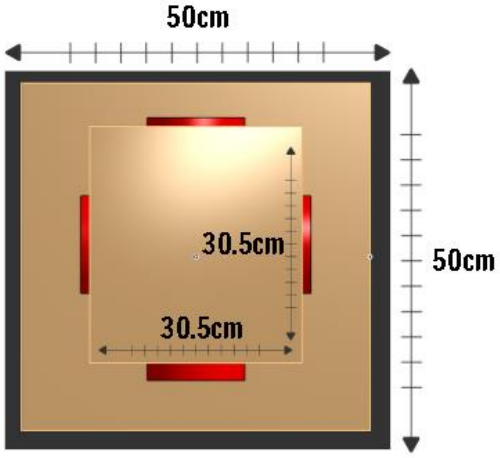


Figure 35. *3D model measurement of the Stoplight (Top View)*

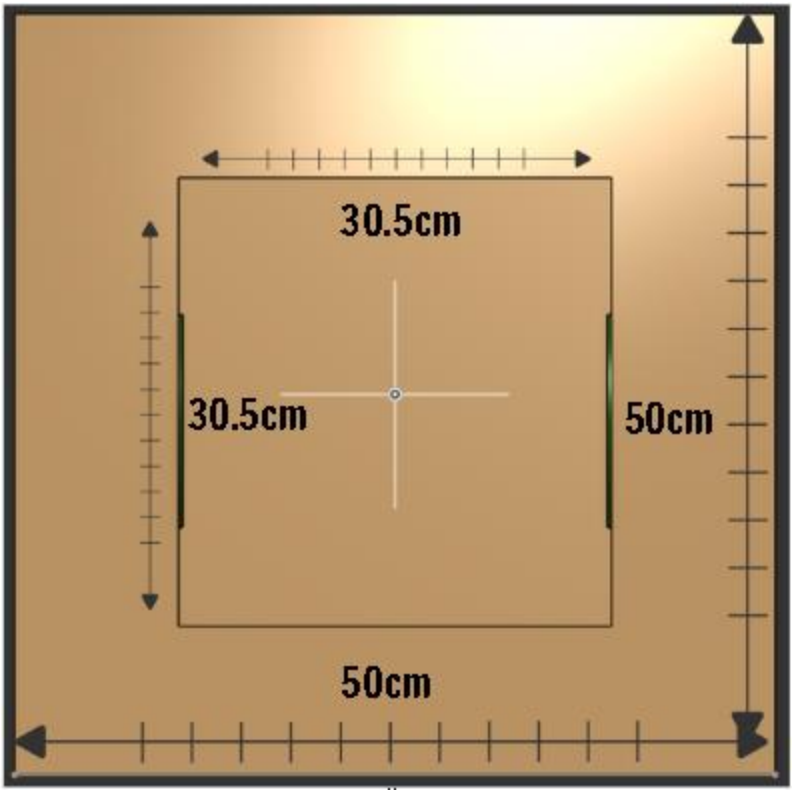


Figure 36. *3D model measurement of the Stoplight (Bottom View)*

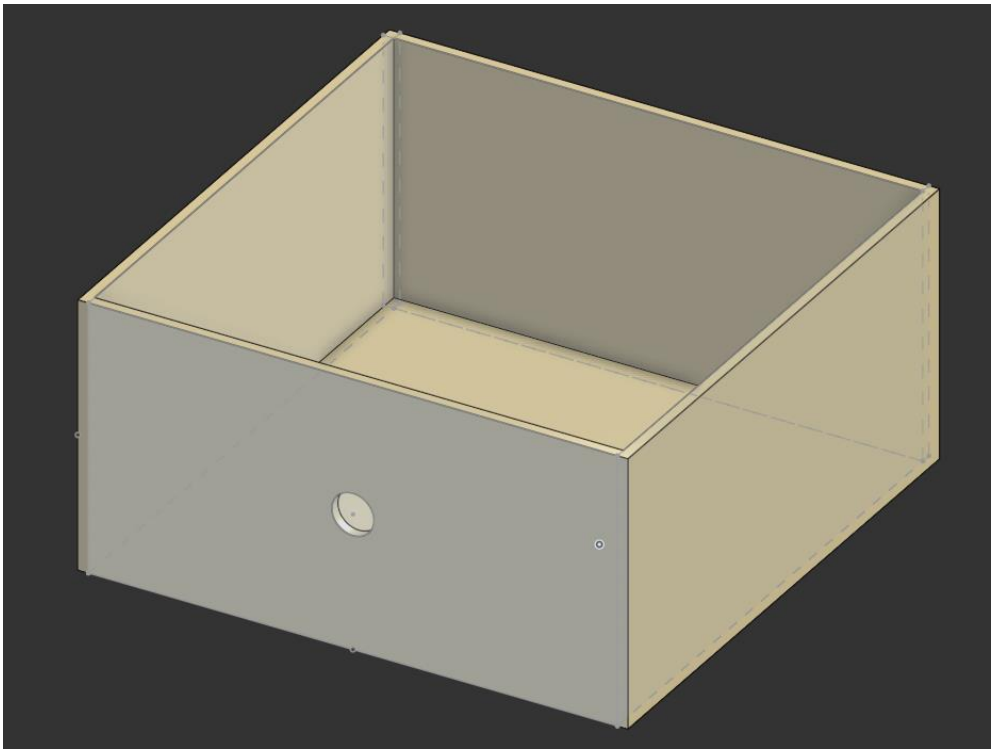


Figure 37. *3D model measurement of the Component Box (Isometric View)*

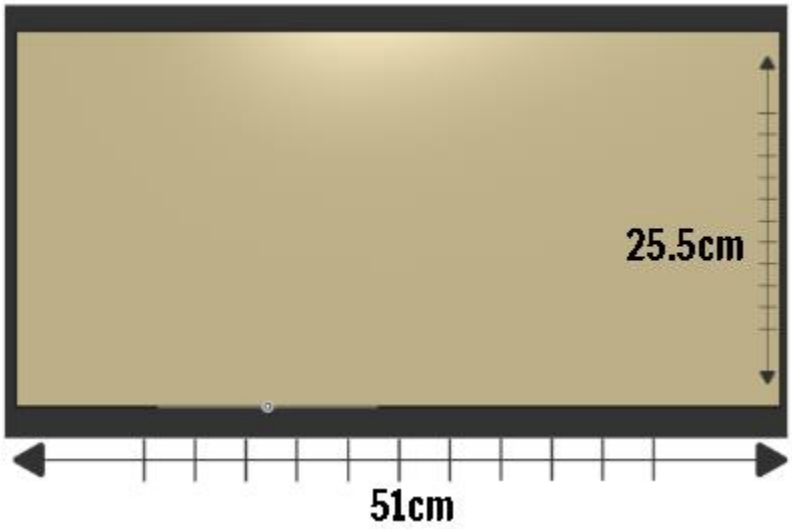


Figure 38. *3D model measurement of the Component Box (Side and Back View)*

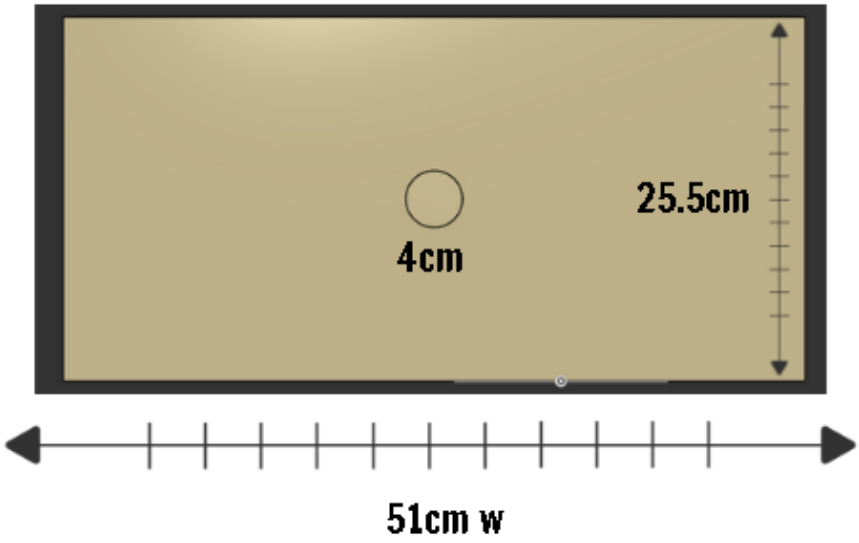


Figure 39. *3D model measurement of the Component Box (Front View)*

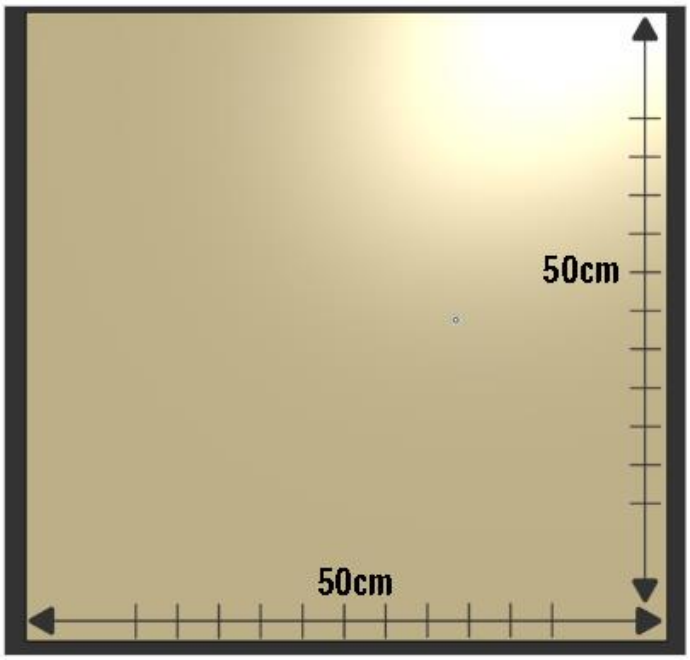


Figure 40. *3D model measurement of the Component Box (Bottom View)*

Summary

The system requirement has been developed in a systematic manner whereby the hardware and software requirement to assemble the Smart Traffic Flow Optimization System was identified first. The hardware choice consisted of purchasing trusted components including an Arduino Uno to control the traffic lights, the Raspberry Pi 5 to process artificial intelligence, the Arducam camera to observe the traffic in real time, relay modules to switch the signals, LED light bulbs to imitate the traffic lights, power supplies, wiring, and a digital sound sensor to detect ambulance sirens. These components have been chosen according to compatibility, functionality, and according to their capability to work together in the architecture of the system. In line with hardware planning, software applications to implement system logic and AI detection were also found, such as Arduino IDE to program the microcontroller, Python on the Raspberry Pi to process camera feeds, TensorFlow and YOLO to develop AI models, Supabase to store data in a database, and ReactJS to display analytics in the dashboard interface. The stage of development combines assembly of hardware and software configuration, wiring all the modules, training the AI models, connecting the Raspberry Pi to the database, and programming the Arduino to behave as a

proper stoplight. After assembling the system, there was a lot of testing done to test the accuracy of detection, response time, traffic lights synchronization, database records and dashboard display. Changes were made depending on the performance observed to make sure the entire system operated effectively, reliably, and as per the planned smart traffic management design.

RESULTS AND DISCUSSIONS

Introduction

Traffic congestion and road safety remain pressing challenges in many urban areas, especially as vehicle volume continues to rise. Traditional stoplight systems often rely on fixed timers that fail to respond to real-time traffic conditions, leading to longer delays and inefficient road flow. This capstone project aims to develop a smarter, more adaptive stoplight system that uses modern technology to optimize signal timing, improve traffic movement, and support safer intersections. By enhancing how stoplights operate, the project seeks to contribute to a more efficient and responsive traffic management solution for today's growing cities.

Experimental

The development and testing of the Smart Traffic Flow Optimization System went through many changes and trials before it worked properly. At first, the space between the stoplights in our design was too wide, so they decided to buy a new plywood board and create a new layout. The new design was smaller and more stable. The developers also adjusted the stoplight poles because the first metal frames did not fit well with the traffic light box.



Figure 41. *Physical Layout of Stoplight*

During the testing of the system, the developers conducted several training sessions to make the detection of ambulances and vehicles more accurate. Since the Raspberry Pi was

new to us, they experienced many trial and error steps to connect it properly with the Arduino. It was important that both devices had the same timing—when the Raspberry Pi turned light green, the Arduino had to do the same. After several tests and code fixes, the lights finally matched correctly.



Figure 42. *Stoplight Light Functionality*

Some hardware problems also happened. The Raspberry Pi camera first showed a blue-colored view, which made the video look unnatural. After editing the code several times, we restored the normal camera color. The power supply was another issue—our first one provided only 4 volts instead of 5 volts. They solved this by using a power bank to supply enough power. To prevent the Raspberry Pi from overheating, we added small air vents inside the box.

The Raspberry Pi's frame rate was also slow at first, which made the video lag. To fix this, we changed the code and replaced the detection boxes with names instead, which made the system faster. For the violation detection, we placed a red line on the road area. If a vehicle crossed this line while the light was red, it was detected as a "beating the red light" violation.



Figure 43. *Detection Line and Boxes Logic*

For the website, we changed the design several times to make it easier to use. They faced some issues with the login system and password recovery using Supabase. The magic link feature did not work well, so we used a simpler local login system instead. We also improved the login and logout functions so that the system worked smoothly for admin users.

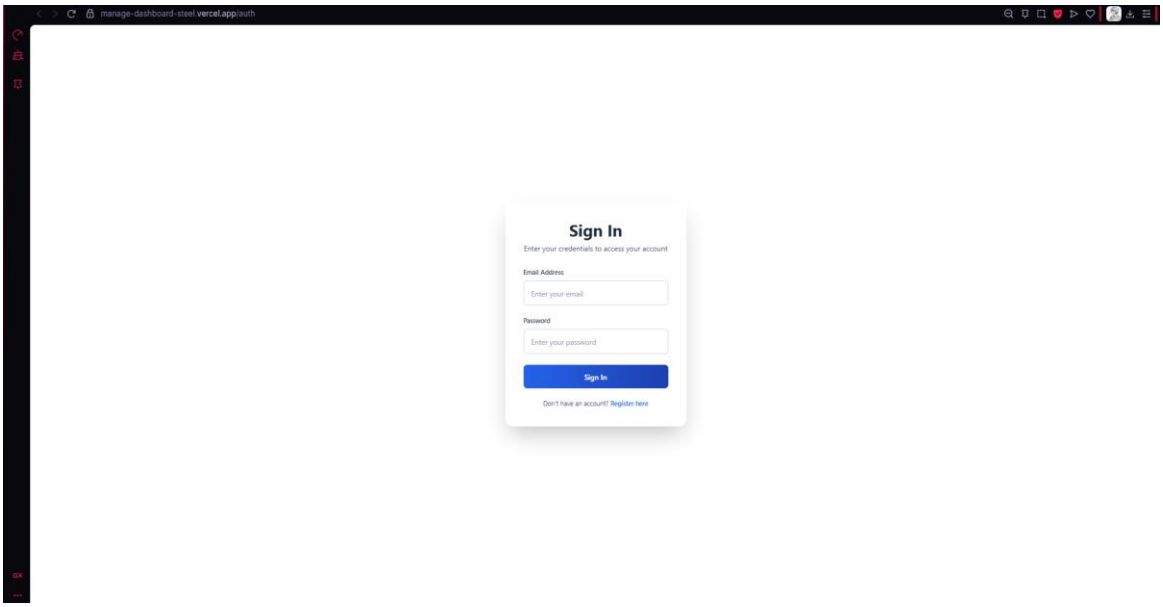


Figure 44. *Signup Page Website*

For the AI model training, Google Colab's 4-hour limit made it hard for us to train longer. To make the most of the time, they managed the training manually and adjusted the epochs. Even with the short training time, the model still worked well in detecting ambulances and violations. We also tried the Cloud, but it was laggy and not suitable for our purpose that's why we change it to localhost since this is near at real-time.

In the end, our system worked successfully. The stoplight hardware, AI detection, and website were all connected and functioning together. The many tests and adjustments showed our effort to make the system reliable and effective for real traffic management use.

Results and Analysis:

Stoplight

Test #	Detection (Meter)	Camera Height	Total Response Time (Seconds)
Test 1	30 meters	6 feet	7 seconds
Test 2	10 meters	7 feet	2 seconds
Test 3	20 meters	6 feet	3 seconds

Table 2. *Ambulance Detection Response Time*

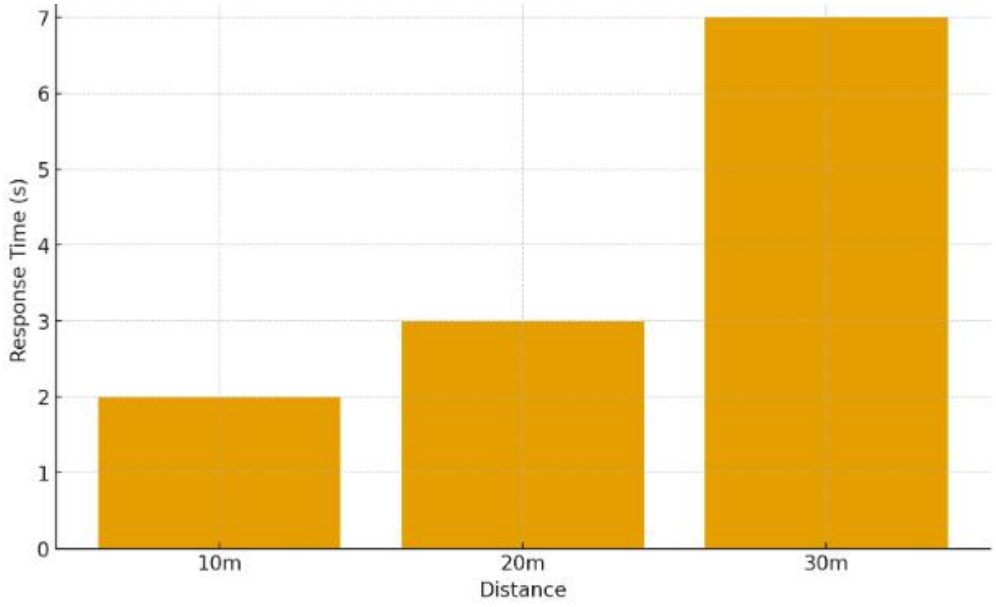


Figure 45. *Bar Graph of Ambulance Detection Response Time*

We have Tested the ambulance Detection response time in accordance with the camera height detecting the Color scheme of the Ambulance Emergency Lights. This was measured by distance.

According to the Graph the farther the Ambulance the longer the response time of the System for detecting the Ambulance while the closer the faster. The System responded from 5 seconds to 3 seconds indicating a near real-time reaction for a suitable ambulance prioritization.

Class	Total Observed	Correct Detection	Accuracy (%)
Car	21	21	100%
Motorcycle	14	14	100%
Ambulances	6	6	100%
Violations	53	53	100%

Table 3. *Detection Accuracy AI Performance*

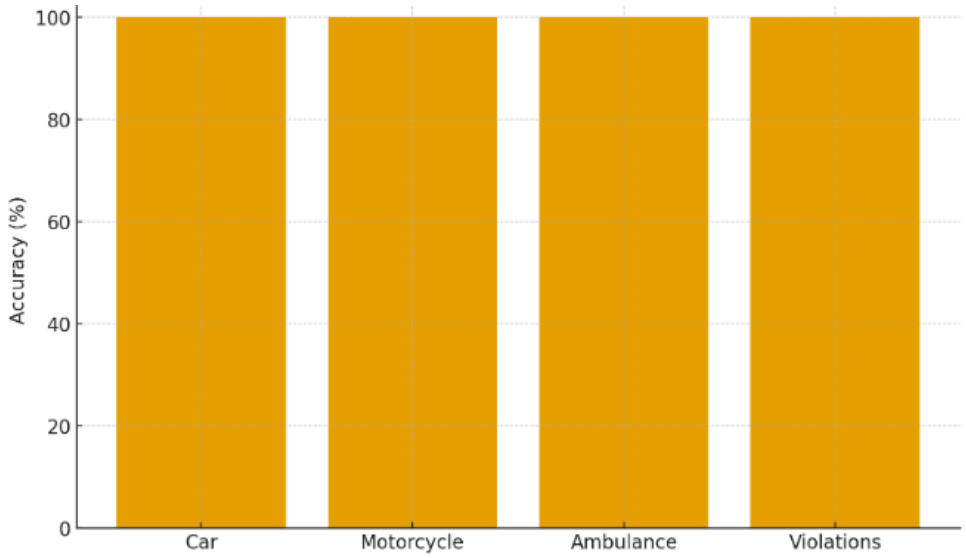


Figure 46. *Bar Graph of Detection Accuracy*

Table 2 summarizes the performance testing results of the AI detection model. The system achieved a 100% accuracy rate across all tested categories, correctly identifying all 21 cars, 14 motorcycles, and 6 ambulances observed during the trials. Additionally, the system successfully detected all 53 traffic violations without error. These results confirm that the AI model has been successfully trained and is capable of reliable, real-time object and violation detection.

Violation Type	Times it was Detected
Speeding	25
Beating Red Light	18
No Helmet	6
Improper Attire (Wearing Shorts, Wearing Slippers, and No Visor	4

Table 4. *Traffic Violation Counts*

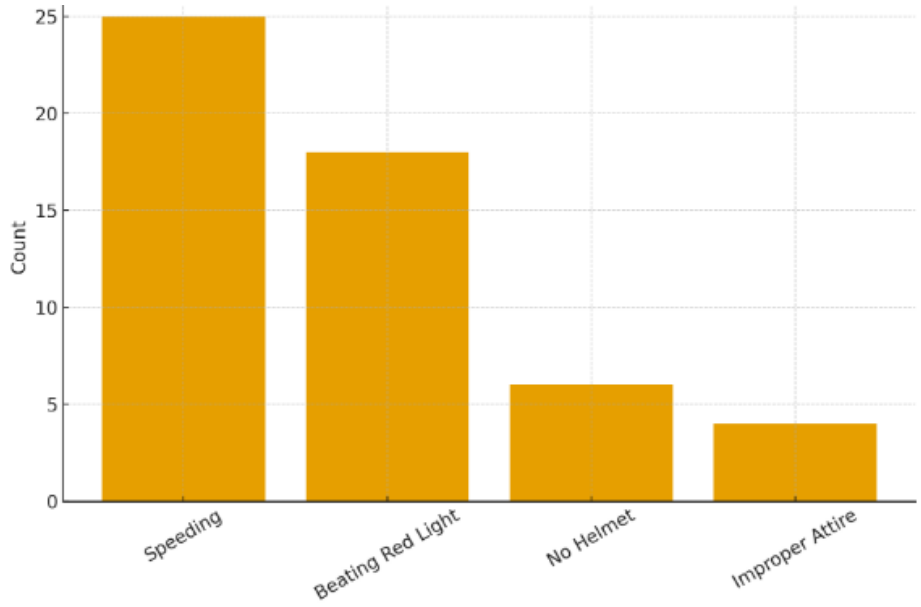


Figure 47. *Bar Graph of Traffic Violation Counts*

Table 3 presents the breakdown of traffic violations detected during the testing phase in Pinagsama, Taguig City. Out of 53 total recorded violations, 'Speeding' was the most violated with 25 instances, indicating a more significant for vehicles to exceed speed limits in the area. 'Beating the Red Light' was the second most common issue with 18 instances. On the other hand, 'Improper Attire' was the least frequent violation (4 instances), suggesting a generally higher level of compliance among residents regarding proper attire for motorcycles laws.

Modes	Red Duration	Yellow Duration	Green Duration
Normal	20 seconds	4 seconds	10 seconds
Ambulance Detected	20 seconds	4 seconds	20 seconds

Table 5. *Ambulance and Normal Light Switching*

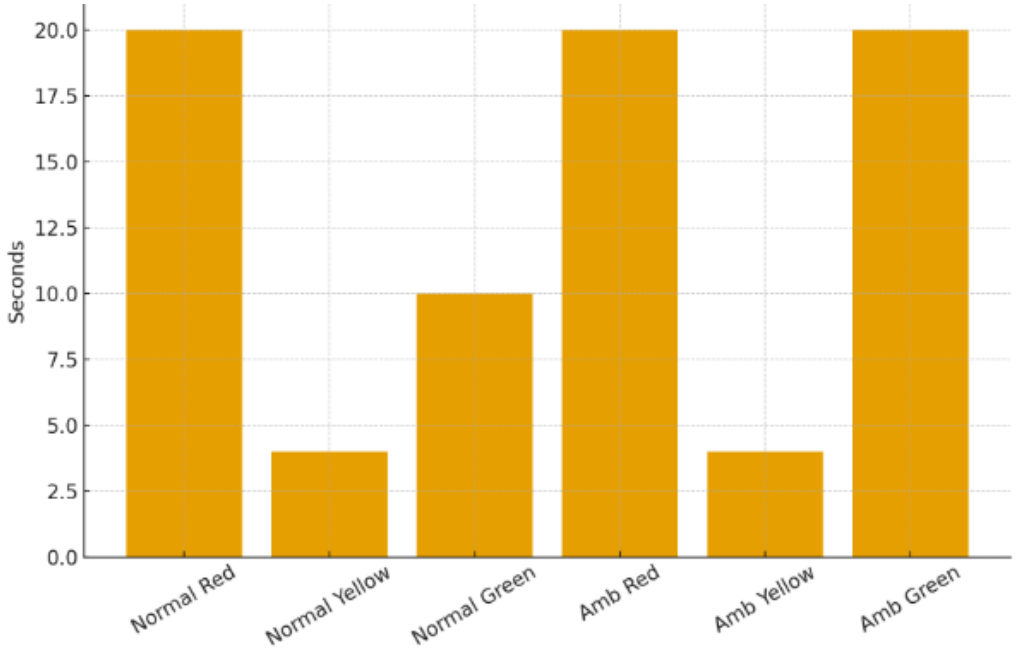


Figure 48. *Bar Graph of Light Switching Duration*

Table 4 compares the traffic light timing sequences between Normal Mode and Ambulance Detected Mode. Under normal conditions, the green light duration is set to 10 seconds. However, when an ambulance is detected, the system automatically prioritizes the ambulance vehicle by extending the green light duration to 20 seconds to ensure it has ample time to clear the intersection. The Yellow light duration remains constant at 4 seconds in both modes to maintain a consistent warning interval for motorists.

Logics	Pros	Cons	Outcome
Keep me logged in button logic	• User control and flexibility • Balance between security and convenience • Users on shared devices can opt out • Follows security best practices	• Users may not understand implications • Can be forgotten to check on public computers • Requires proper token management	

Always Keep the User Logged in but with inactivity Session time out	• Maximum convenience • Better user experience • Automatic logout on inactivity protects abandoned sessions • Reduces risk on shared devices • Good for internal/trusted environments	• Still no explicit user control • Users might not expect auto-logout • Timeout value is critical • Activity tracking required	Best Condition
Put the user in Login Page if refreshed	• Highest security level • Minimal attack window • Good for high-sensitivity applications	• Very poor user experience • Impractical for most use cases • May frustrate users • Could lead to weak password choices	

Table 6. *Testing Security of Website Signup Page*

Three authentication methods were evaluated, with the "inactivity timeout" approach recommended as the best balance between user convenience and security, while the constant re-login method was rejected for poor usability.

Test Number	Delay	Display	Total
Test 1	2 seconds	2 seconds	4 seconds
Test 2	1 second	3 seconds	4 seconds
Test 3	2 seconds	1 second	3 seconds

Table 7. *Testing Data Fetching (Website)*

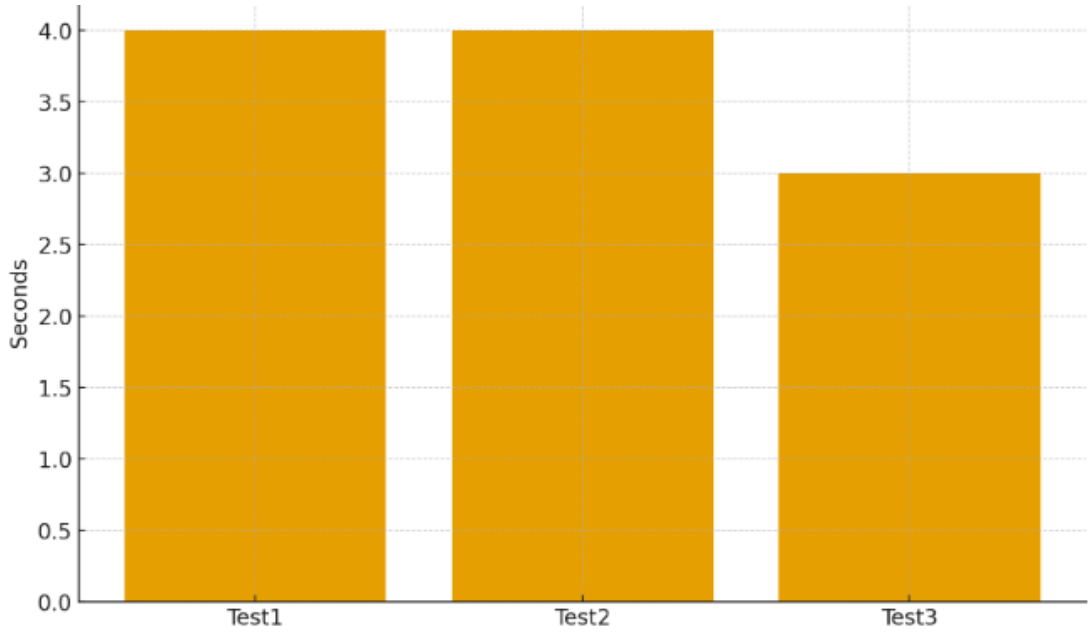


Figure 49. *Bar Graph of Data Fetching*

The system achieved consistent loading times of 3-4 seconds across tests, with the Deployed Website implementation successfully resolving previous cloud-based lag issues and providing acceptable performance for real-time traffic monitoring.

Number of Test	Range (meters)	Result of Detection
Test 1	1-3 meters	Consistent Detection
Test 2	4-6 meters	Needs a strong sound
Test 3	7-20+ meters	No Response need stronger sound sensor

Table 8. Sound sensor module response (seconds)

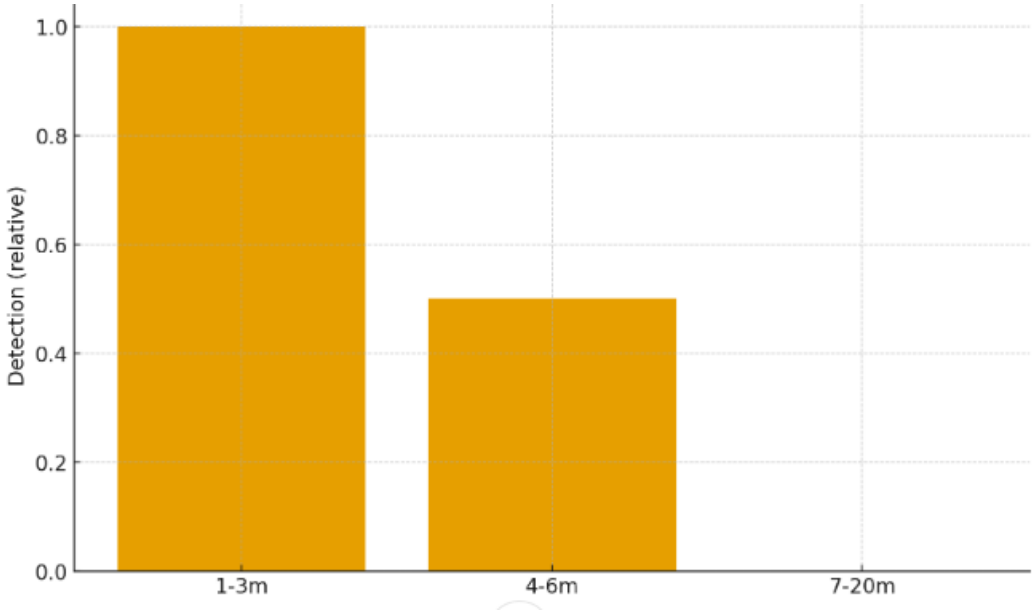


Figure 50. *Bar Graph of Sound Sensor Detection Strength*

Table 7 evaluates the detection capability of the sound sensor module based on distance. In this test, a recorded ambulance siren was played using a mobile phone speaker to simulate an emergency vehicle. The results showed that the module could consistently detect the sound within a close range of 1 to 3 meters. At 4 to 6 meters, detection was inconsistent and required a louder siren to trigger the sensor. However, 7 meters and beyond, no detection occurred.

This outcome is primarily due to the low decibel output of the mobile phone, which cannot replicate the volume of an actual ambulance siren. In addition, the LM386-based sound sensor has limited sensitivity, making it difficult to detect faint or distant sound waves. In real-world conditions—where emergency sirens are significantly louder—the system is expected to detect sound at greater distances. However, under the simulated test conditions, detection was limited to close-range performance.

Summary

The Smart Traffic Flow Optimization System was successfully developed through extensive testing and multiple design iterations to create an AI-powered traffic management solution with ambulance prioritization capabilities. The project overcame

numerous technical challenges, including hardware integration issues between the Raspberry Pi and Arduino, camera calibration problems, power supply inadequacies, and performance optimization needs. The team made significant adjustments such as redesigning the physical layout on a more stable plywood board, fixing the camera's blue tint through code modifications, upgrading to appropriate power sources, adding ventilation to prevent overheating, and optimizing detection algorithms to reduce frame rate lag.

The system demonstrated impressive performance results across all testing parameters. AI detection achieved 100% accuracy, correctly identifying all 21 cars, 14 motorcycles, 6 ambulances, and 53 traffic violations during trials, confirming successful model training despite Google Colab's 4-hour session limitations. Ambulance detection showed distance-dependent response times ranging from 2 seconds at 10 meters to 7 seconds at 30 meters, demonstrating near real-time reaction capabilities suitable for Ambulance vehicle prioritization. The traffic light system automatically adjusts timing from a normal 10-second green light to an extended 20-second duration when ambulances are detected, while maintaining consistent 4-second yellow intervals in both modes.

Field testing in Pinagsama, Taguig City revealed valuable data on traffic violation patterns, with speeding being the most common offense at 25 instances, followed by red light violations at 18 instances, no helmet violations at 6 instances, and improper attire at 4 instances. The web dashboard integration proved effective with consistent 3-4 second loading times after switching from cloud-based to localhost data fetching, resolving previous lag issues. Security implementation utilized an inactivity timeout authentication method after evaluating multiple approaches. However, the Digital Sound Detector showed limited range during testing, with consistent detection only at 1-3 meters when using a mobile phone speaker to simulate ambulance sirens, though real-world performance with actual high-decibel Ambulance vehicle sirens is expected to be significantly better. Overall, the integrated system successfully connected stoplight hardware, AI detection capabilities, and web dashboard functionality, demonstrating reliability and effectiveness for practical traffic management applications.

CONCLUSION AND RECOMMENDATIONS

Conclusion

This study developed and implemented a Smart Traffic Flow Optimization System designed to reduce congestion, prioritize emergency vehicles, and detect road violations in real time. The system integrated Artificial Intelligence (AI), Internet of Things (IoT), and computer vision technologies using Arduino Uno, Raspberry Pi 5, Arducam, sensors, relays, and a web-based analytics platform. After several tests and improvements, the system demonstrated its capability to dynamically adjust traffic light timings, recognize ambulances for lane prioritization, and identify common violations such as red-light beating and speeding. Despite challenges encountered during calibration, coding synchronization, and power optimization, the prototype successfully achieved its objectives in enhancing road safety, improving emergency response, and providing an efficient, automated traffic management solution.

The results of this study concluded that the Smart Traffic Flow Optimization System effectively addressed three major problems:

First, it improved traffic congestion management by using real-time data from cameras and AI algorithms to optimize signal timings according to vehicle density, resulting in smoother traffic flow.

Second, it enhanced emergency vehicle prioritization, as the system was able to detect ambulances and automatically give them the right of way, reducing their response time.

Third, it strengthened violation detection efficiency through automated image analysis and database logging, enabling authorities to monitor and record offenses more accurately and consistently.

Based on the findings, the study concludes that implementing AI- and IoT-based systems in urban traffic management offers a more reliable and adaptive alternative to conventional fixed-time traffic control. The project also supports the concept of smart cities by demonstrating how automation and real-time data processing can create safer, faster, and

more sustainable transportation systems.

For future improvements, the developers recommend expanding the system's application to larger and more complex intersections, integrating additional sensors and cameras for wider coverage, and enhancing AI model training to include other emergency vehicles such as police cars and fire trucks. Further optimization of network latency and hardware performance is also encouraged to ensure smoother operation in real-world environments. Additionally, collaboration with local government traffic agencies could help in testing the system under actual road conditions for validation and deployment. Finally, continuous research and upgrades in AI algorithms, database management, and analytics visualization are suggested to improve accuracy, reliability, and scalability toward a fully functional smart city traffic ecosystem.

Recommendation

The Smart Traffic Flow Optimization System successfully validated the application of Artificial Intelligence (AI) and the Internet of Things (IoT) in urban traffic management, demonstrating remarkable technical proficiency with 100% detection accuracy for vehicles, violations, and ambulances during laboratory testing. The system's quick, near real-time ambulance prioritization response (ranging from 2 to 7 seconds) effectively validates its core concept: solving the dual problems of traffic congestion and delayed emergency response. However, the project's success as a prototype highlights several critical limitations related to its small scope, constrained hardware capabilities, and reliance on local infrastructure. To realize its potential as a scalable, reliable, and comprehensive solution for smart city deployment, the following recommendations are essential steps for future research and development.

The primary constraint on the system's real-world viability is its foundational architecture, which is confined to a small-scale, two-phase intersection and operates solely on a local host. To achieve meaningful impact across the complex road networks typical of Metro Manila, the system must undergo significant architectural scaling. We recommend developing a distributed, hybrid computing framework. This model would leverage cloud services (e.g., Supabase or a dedicated cloud backend) for centralized, large-scale data

aggregation, historical analysis, and administrative oversight. Simultaneously, it must retain local edge computing (like the existing Raspberry Pi) to handle immediate, low-latency tasks such as real-time detection, traffic light decision-making, and primary control signals. This hybrid approach ensures both responsiveness and scalability. Furthermore, the system logic must be generalized to manage the signal complexity of multi-phase intersections (e.g., four-way junctions with dedicated turning lanes), moving beyond the simplified two-phase design to prove its efficacy in real-world urban environments.

The integrity of the system relies on robust, reliable hardware, yet the prototype showed vulnerability in two critical areas: power supply and sensor range. The reliance on a portable power station drastically limits continuous operation to just 2–3 hours, making it entirely impractical for 24/7 public service. We strongly recommend implementing a perpetual power solution, such as a solar power array augmented by a high-capacity battery backup system, or securing direct, protected integration with municipal power grids. This step ensures 24/7, uninterrupted functionality regardless of power fluctuations. Concurrently, the effectiveness of the emergency prioritization feature must be improved. This requires two actions: expanding the AI training to identify and prioritize all critical responders, including police cars and fire trucks, and replacing the existing LM386-based sound sensor with a high-sensitivity, high-decibel acoustic sensor. The current sensor's limited detection range (1–3 meters) is a major risk; successful deployment demands reliable siren detection from over 50 meters, providing sufficient warning for traffic clearance. The system's utility as an automated enforcement tool can be significantly amplified. While current success lies in detecting speeding and red-light violations, the scope of enforcement needs to be broadened to encompass more nuanced traffic infractions. This includes expanding the AI model's capacity to identify and log violations such as illegal turns, pedestrian crossing violations, and attempts at license plate tampering. Maximizing the utility of the deployed cameras in this manner will provide enhanced value to traffic management authorities and contribute to a stronger culture of road discipline. Lastly, while the shift to a local host environment solved lag issues in the prototype phase, the final deployment model should focus on ensuring minimal latency across the recommended hybrid network architecture to maintain the current system's swift response times in a fully integrated, public environment.

REFERENCES

Agorku, G., Agbobli, D., Chowdhury, V., Amankwah-Nkyi, K., Ogungbire, A., Ankamah Lartey, P., & Aboah, A. (2023). Real-time helmet violation detection using YOLOv5 and ensemble learning. *arXiv*. https://doi.org/10.48550/arXiv.2304.09246

Ben Laoula, E., Elfahim, O., El Midaoui, M., Youssfi, M., & Bouattane, O. (2023). Traffic violations analysis: Identifying risky areas and common traffic violations. *Heliyon*. https://www.sciencedirect.com/science/article/pii/S2405844023062667

Bhakre, R., Madane, N., & Narkhede, M. (2025). AI-based traffic violation detection. https://www.researchgate.net/publication/396973847_AI_Based_Traffic_Violation_Detection

Buniel, G. G., & Tantoy, C. P. (2024). SURATSA: Implementation of Surigao real-time adaptive traffic signal algorithm (RATSA) for traffic management in Barangay Luna, Surigao City, Philippines. *International Journal of Research and Scientific Innovation, 11*(8), 342–357. https://www.researchgate.net/publication/383666744_SURATSA_Implementation_of_Surigao_RealTime_Adaptive_Traffic_Signal_Algorithm_RATSA_for_Traffic_Management_in_Barangay_Luna_Surigao_City_Philippines

De Ocampo, B. M., Clark, E., & De Pedro, M. (2021). An Industry 4.0 framework towards an integrated smart traffic management solution. In *9th National Business and Management Conference*. https://www.researchgate.net/publication/367431579_An_Industry_40_framework_towards_an_Integrated_Smart_Traffic_Management_Solution

Ghaleb, B., Qiu, T., Ghafoor, K. Z., & Shaikh, F. K. (2023). A low-cost IoT-based smart traffic management system for smart cities. *Sensors, 23*(11), 5324. https://doi.org/10.3390/s23115324

Hasrouny, H., Bassil, C., Samhat, A. E., & Laouiti, A**.** (2017). Security risk analysis of a trust model for secure group leader-based communication in VANET. In *Vehicular Ad-*

Hoc Networks for Smart Cities: Second International Workshop, 2016 (pp. 71–83). Springer Singapore. https://www.researchgate.net/publication/315512135_Security_Risk_Analysis_of_a_Trust_Model_for_Secure_Group_Leader-Based_Communication_in_VANET

Karmakar, G., Chowdhury, A., Kamruzzaman, J., & Gondal, I. (2020). A smart priority-based traffic control system for emergency vehicles. *IEEE Sensors Journal, 21*(14), 15849–15858. https://ieeexplore.ieee.org/abstract/document/9193898

Nguyen, N. V., Le, H. T., Phan, M. N., & Lai, L. N. T**.** (2022). Red-light running violation detection of vehicles in video using deep learning methods. In *EAI International Conference on Artificial Intelligence for Communications and Networks* (pp. 175–185). https://eudl.eu/doi/10.1007/978-3-031-08878-0_15

Pang, J. L. (2023). Enhancing urban traffic management: Advanced strategies in image recognition-based intelligent traffic monitoring. *Traitement du Signal, 40*(6), 2587–2597. https://doi.org/10.18280/ts.400621

APPENDICES

APPENDIX A. PROJECT ACCEPTANCE

ADVISER'S ACCEPTANCE

APPROVED **Smart Traffic Flow Optimization System**

RESEARCH TITLE:

NAME OF DEVELOPERS: Juliana Paula P. Baloy

Duane Benedict A. Mahinay

Harvey G. Malicse

Alvin Ken G. Modina

CONFOMER:

Sherwin M. Ostria

Thesis Adviser

APPROVED:

Sherwin M. Ostria
Thesis Coordinator

November 2025
Date

NOTED BY:

Mark Frederick V. Salonga
Program Head

CERTIFICATE OF PROOFREADING

This document certifies that the thesis listed below was reviewed and edited for proper English language, grammar, punctuation, spelling, and overall style. Neither the research content nor the authors' intentions were altered in any way during the editing process.

THESIS TITLE

"SMART TRAFFIC FLOW OPTIMIZATION SYSTEM"

THESIS AUTHOR(S)

Juliana Paula P. Baloy
Duane Benedict A. Mahinay
Harvey G. Malicse
Alvin Ken G. Modina

YEAR AND PROGRAM

4th Year Bachelor of Science in Computer Engineering

SCHOOL

STI College Global City

DATE ISSUED

November 2025

Mark Frederick V. Salonga
Grammarian

APPENDIX B. TRANSCRIPT OF INTERVIEW

Title of Study: Smart Traffic Flow Optimization System

Date of Interview: November 17, 2025

Location: Bonifacio Global City

Type of Interview: Question and Answer

Interviewee: Enforcer, Julius Empalmado

Interviewer: Mahinay, Duane Benedict A. and Malicse, Harvey G.

Duration: 17 minutes

Transcript:

Mr. Mahinay: Good afternoon, sir. May we have your consent to record this interview for our capstone project titled Smart Traffic Flow Optimization System. This study aims to support the current traffic system and improve the prioritization of ambulances.

Enforcer Empalmado: Yes, I give my consent.

Mr. Mahinay: How do you determine which lane should have a longer green light duration than the others?

Enforcer Empalmado: Each intersection has a programmed sequence. For example, this lane (referring to Every Nation to Market Market) is given a 30-second interval so pedestrians can cross safely. The remaining time is based on what we call the “Sekuyar Rotation.” This accounts for pedestrians who are running or crossing late, allowing them additional time to pass.

Mr. Malicse: Do you have specific times when the interval changes?

Enforcer Empalmado: Yes. There are set times when the stoplight sequence changes. In the morning around 4 AM and in the afternoon around 4 PM, the cycle rotates to adjust for traffic flow.

Mr. Mahinay: So, does the sequence change depending on the time of day, especially when there is heavy traffic in the afternoon?

Enforcer Empalmado: Yes. The stoplight prioritizes lanes based on where traffic is heavier.

Mr. Mahinay: So, is there a different time sequence for morning, afternoon, and nighttime?

Enforcer Empalmado: Correct. Between 7 AM and 10 AM, for example, traffic management or marshals often assist due to the volume of vehicles.

Mr. Malicse: How do you time when each lane should go or stop?

Enforcer Empalmado: Personally, I count in my head. Each forward motion of my hand signal is equivalent to one second. After counting to 60 hand movements, that becomes roughly 60 seconds. The counting may not be exact, but it helps control the traffic.

Mr. Mahinay: You don't use any type of clicker or device for timing?

Enforcer Empalmado: No, because it would be too complicated for me.

Mr. Malicse: So, is the timing fixed?

Enforcer Empalmado: The timing is programmed and monitored by our panel box located on 3rd or 1st Street. It operates 24/7.

Mr. Mahinay: Does the headquarters handle the changes in time intervals?

Enforcer Empalmado: Yes. We also radio the headquarters to inform them if we are assisting with traffic. We receive directions from them, especially if there is a malfunction in the stoplight—like when vehicles are moving but pedestrians still need to cross.

Mr. Mahinay: So, are there two of you managing the traffic?

Enforcer Empalmado: Yes. Sometimes they instruct us to take over a lane so we can manually direct traffic, even if it means disregarding the stoplight temporarily.

Mr. Mahinay: So the default hand-signal timing is 60 seconds?

Enforcer Empalmado: Yes. One forward motion of the hand is roughly one second.

Mr. Malicse: Does that timing ever change?

Enforcer Empalmado: Yes. It depends on how I count in my head.

Mr. Malicse: Does your hand-signal counting correspond to the stoplight's programmed timing?

Enforcer Empalmado: Yes. Since I've been doing this every day, I'm familiar with the traffic flow and the signal timing.

Mr. Mahinay: Last question. What is the default or normal time interval of the stoplight?

Enforcer Empalmado: It depends on the direction where vehicles are coming from. For example, this lane (pointing to the EDSA side) is prioritized because most vehicles come from EDSA. That lane gets a green light for about 3 minutes. The other lane (pointing to Market Market) gets around 1 minute and 25 seconds.

Mr. Mahinay: So, it really depends on the direction of traffic?

Enforcer Empalmado: Yes. This is a main road, and vehicles come from all directions.

Mr. Malicse: So, the timing is not purely programmed but also depends on the location and direction of incoming vehicles?

Enforcer Empalmado: Yes. And sometimes, when traffic gets too heavy—like for vehicles coming from the eastbound heading to C-5, we must let them go first to prevent congestion. Sometimes we let them move even if it means disregarding the stoplight, if it prevents gridlock.

Mr. Mahinay: So, is the time interval really based on the traffic route?

Enforcer Empalmado: Yes. If a lane is more congested, it needs to be prioritized.

Mr. Mahinay: That ends our Q&A. Thank you so much for your time.
Mr. Malicse: Thank you very much.

Enforcer Empalmado: Thank you also. I hope you do well in your capstone.

.

APPENDIX C. INVOLVEMENT FORM

THESIS PROJECT INVOLVEMENT FORM

THESIS TITLE: **SMART TRAFFIC FLOW OPTIMIZATION SYSTEM**	
DATE OF DEFENSE: **November 15, 2025**	SCHOOL CAMPUS: **STI College Global City**
NAME OF ADVISER: **Sherwin M. Ostria**	SIGNATURE:

Each member of the group indicates the chapter(s) he/ she had written in the documentation, module(s) he/ she had developed in the software and the topic(s)/ sub-topics(s) that he/ she will be discussing during thesis defense. The items given by a group member are to be used by the panelists as a guide and reference for the evaluation of the individual participation and contribution of the student in their thesis project.

NAME: Baloy, Juliana Paula P.	Signature:
Chapter(s)/ sub-topic(s) written in the documentation: • Introduction • Objectives of the Study • Overview of the Current State of Technology • Statement of the Problem • Literature Review • General Problem • Specific Problem • Overview of the Project • System Design Specification • Features of Developed System/Project • Schedule of Activities • Budgetary Expenses	
Module(s) developed in the software: • Assistant in developing the web application	
Module(s) developed in the hardware: • Prototype Design • Wirings • Measured Material • Assistance in Designing the stoplight	

Topic(s) to be discussed during defense: • Introduction • Demonstration of Prototype
Other contributions: • Printing • Presentation • Buying materials • Checking documents

NAME: Mahinay, Duane Benedict	Signature:
Chapter(s)/ sub-topic(s) written in the documentation: • Background of the problem • Literature Review • Hardware Flowchart • Schematic Diagram • Block Diagram • Bench Marking • Acknowledgement • Objective of the Study • Limitation	
Module(s) developed in the software: • Developed Raspberry Pi Codes	
Module(s) developed in the hardware: • Prototype Design • Wirings • Soldering • Troubleshooting of Emergency lights • Coding • System Optimization • System Calibration	
Topic(s) to be discussed during defense: • Specific Objectives	
Other contributions: • Printing • Presentation	

• Buying materials • Checking documents

NAME: Malicse, Harvey G.	Signature:
Chapter(s)/ sub-topic(s) written in the documentation: • Background of the Study • Overview of the current technology • Scope and Limitation • Limitation • Literature Review • Hardware and Software Requirements • Survey Tally and Results • HIPO • ERD User's Manual • Screen Design • Survey Questionnaires • Database Schema • Sample System Reports • Transcript of Interview • Summary • Result and Analysis • Experimental • Block Diagram	
Module(s) developed in the software: • Developed Web application • Design UI	
Module(s) developed in the hardware: • Wiring • Assistance in Relays • Measurement of Materials • Assist in Troubleshooting	
Topic(s) to be discussed during defense: • Specific Objectives	
Other contributions: • Presentation	

• Buying materials

NAME: Modina, Alvin Ken G.	Signature:
Chapter(s)/ sub-topic(s) written in the documentation: • Literature Review • General / Specific Problem • General / Specific Study	
Module(s) developed in the software: • Developed Raspberry Pi Codes	
Module(s) developed in the hardware: • Prototype Design • Wirings • Soldering • Troubleshooting of Emergency lights • System Calibration • Enclosure Calibration • Hardware Testing • Circuit Design • Sensor Integration • Component Assembly • Traffic Light Assembly • Camera Setup • System Optimization	
Topic(s) to be discussed during defense: • Specific Objectives	
Other contributions: • Printing • Presentation • Buying materials • Checking documents	

APPENDIX D. SCHEDULE OF ACTIVITIES

Gantt Chart

The Gantt chart shows the plan of the project and its activities that are being done to develop the Smart Traffic Flow Optimization System. It demonstrates the planned activities, their time, and the order of their execution starting with the brainstorming and preparation of the proposal till with the defense and final revisions. A certain activity is represented by each bar in the chart, which enabled the researchers to follow the progress, spend their time wisely, and remember whether the milestones were reached within the time provided. The Gantt chart was a good project management tool as it provided a clear visualization of the overlap of tasks and the deadlines that would keep track of the general flow of the thesis development.

PLAN **FINISHED**

Month	Week	Brainstorming for Title Proposal	Consultation for Title Proposal	Revision for Proposed Title Proposal	Consultation of Revised Proposed Title	Thesis Title Proposal Defense
August	W1					
	W2					
	W3					
	W4					
September	W1					
	W2					
	W3					
	W4					
October	W1					
	W2					
	W3					
	W4					
November	W1					

	W2					
	W3					
	W4					
December	**W1**					
	W2					
	W3					
	W4					
January	**W1**					
	W2					
	W3					
	W4					
February	**W1**					
	W2					
	W3					
	W4					
March	**W1**					
	W2					
	W3					
	W4					
April	**W1**					
	W2					
	W3					
	W4					

Table 9. *List of Activities through the Semester*

APPENDIX E. SCREEN DESIGN

Website Screen

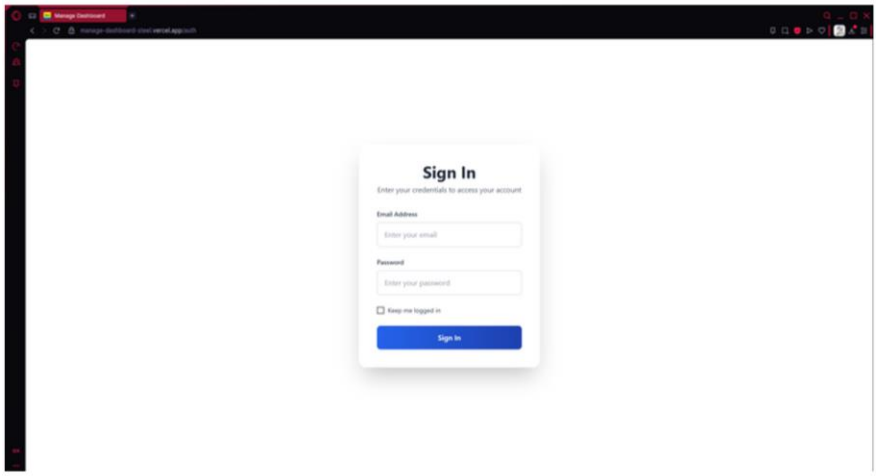


Figure 51. *Sign in page*

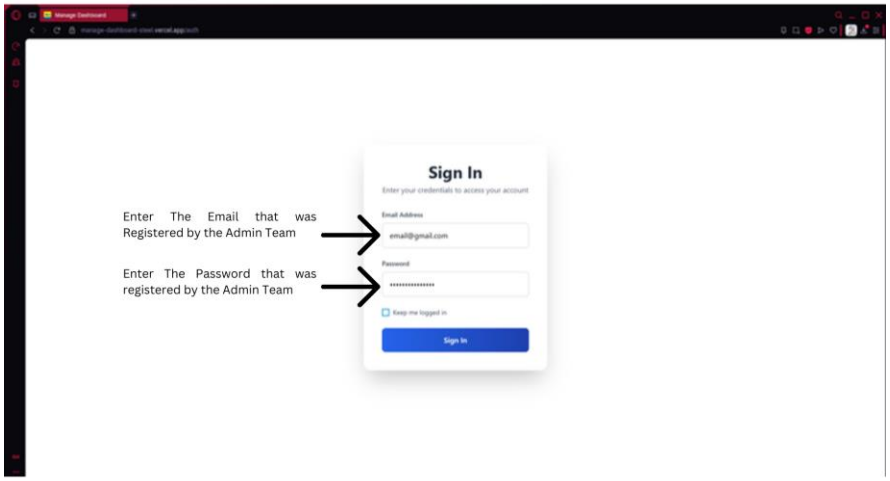


Figure 52. *Sign in your account*

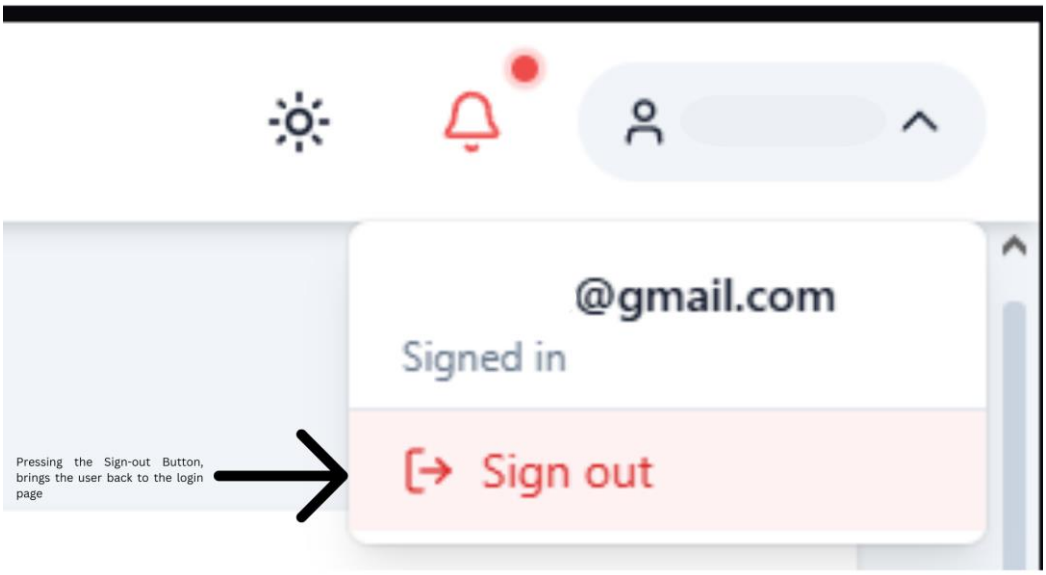


Figure 53. *Sign out*

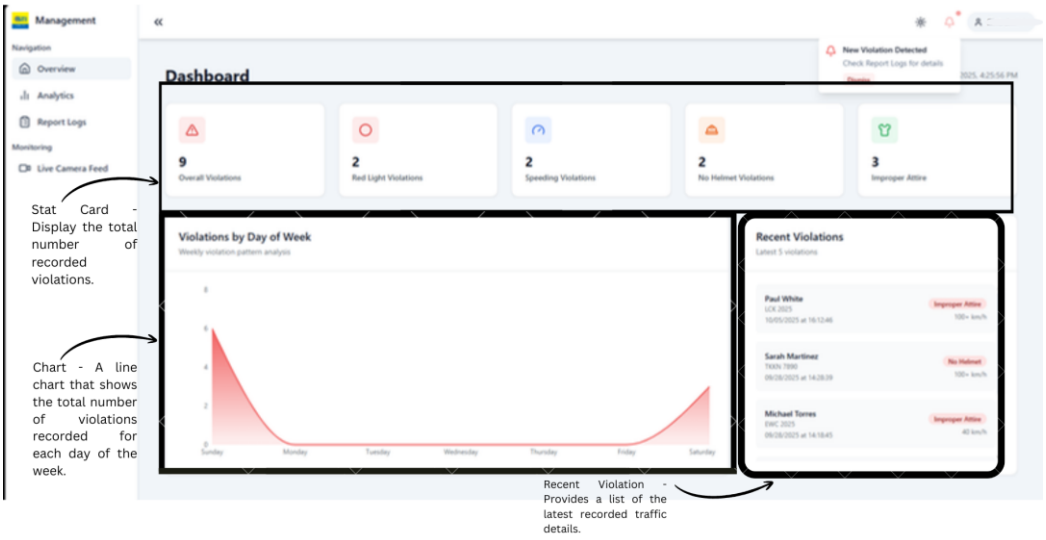


Figure 54. *Dashboard*

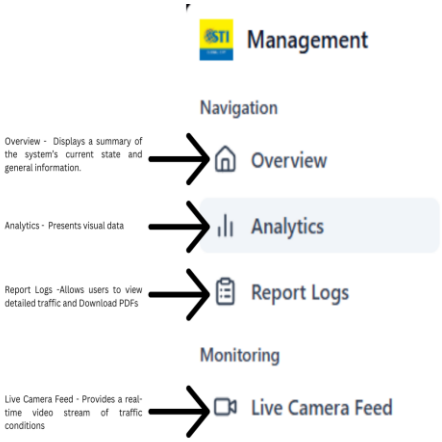


Figure 55. *Navigation Buttons*

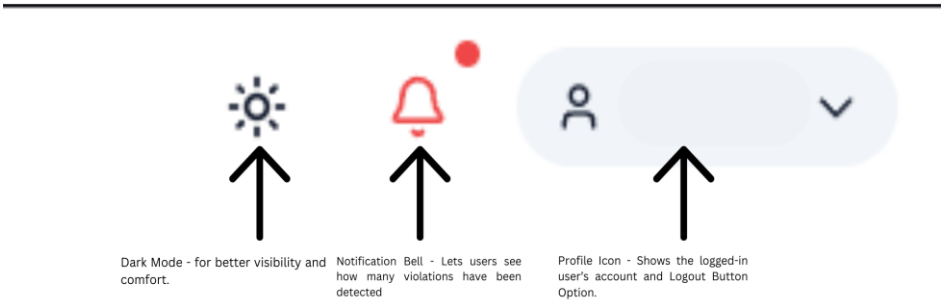


Figure 56. *Other Buttons*



Figure 57. *Website Analytics*

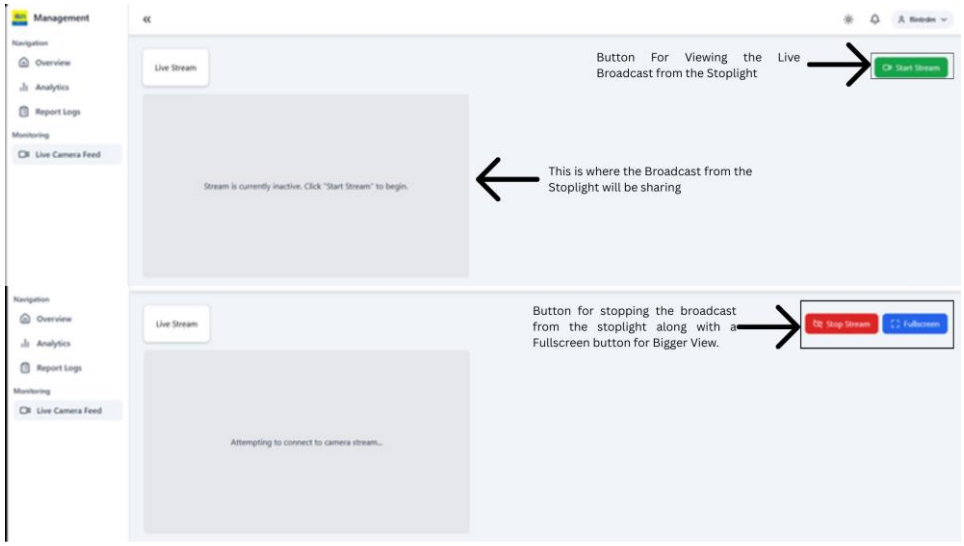


Figure 58. *Camera Monitoring*

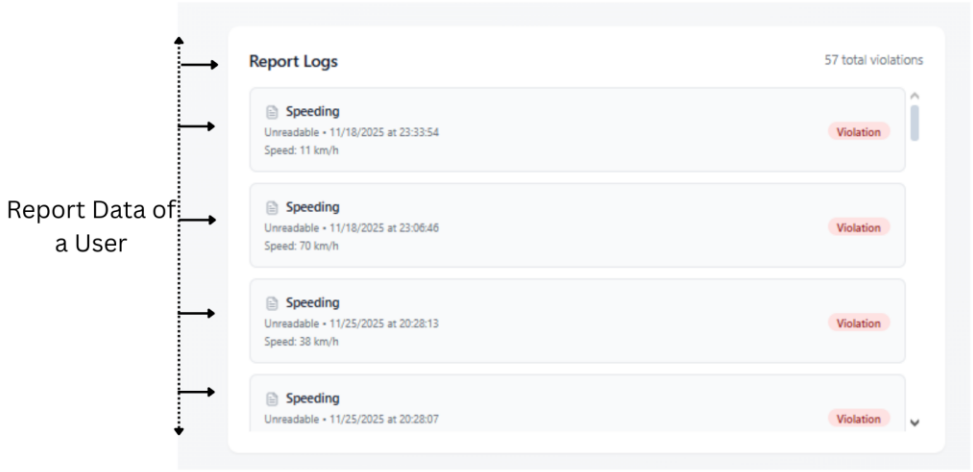


Figure 59. *Report Data of User*

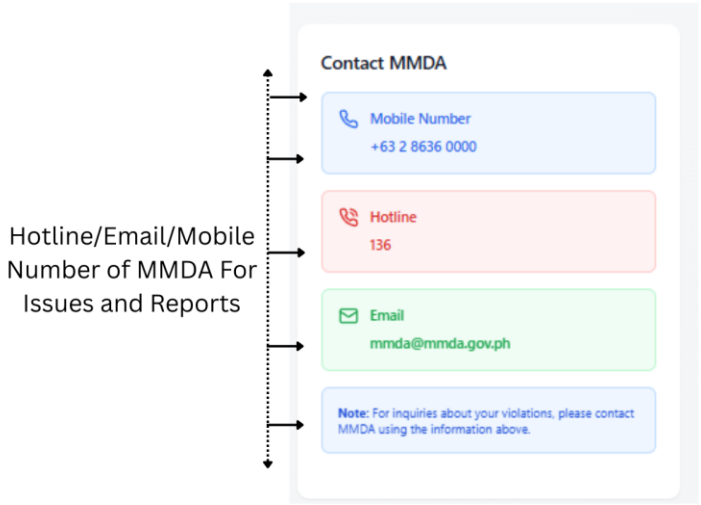
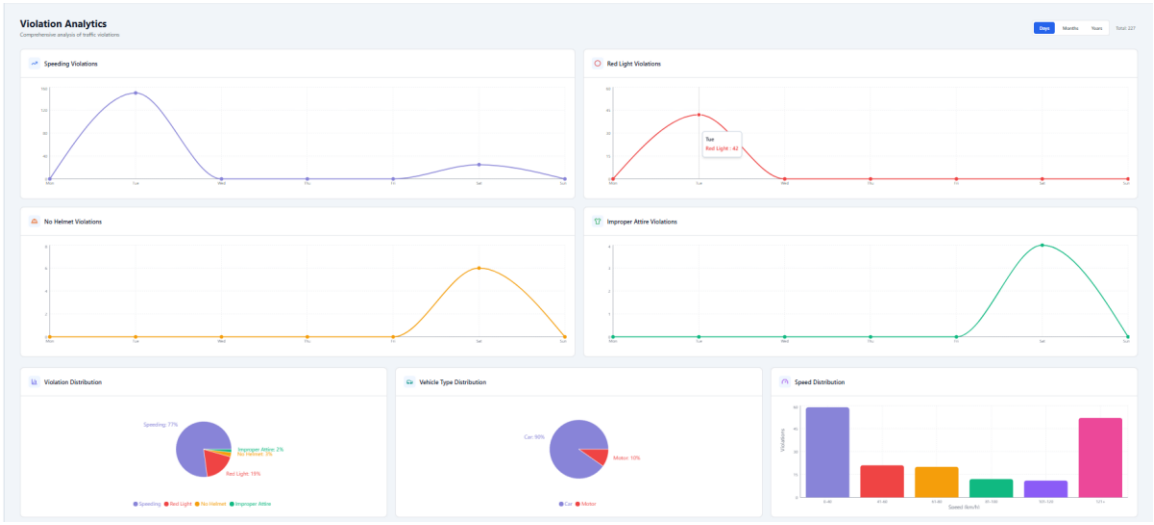
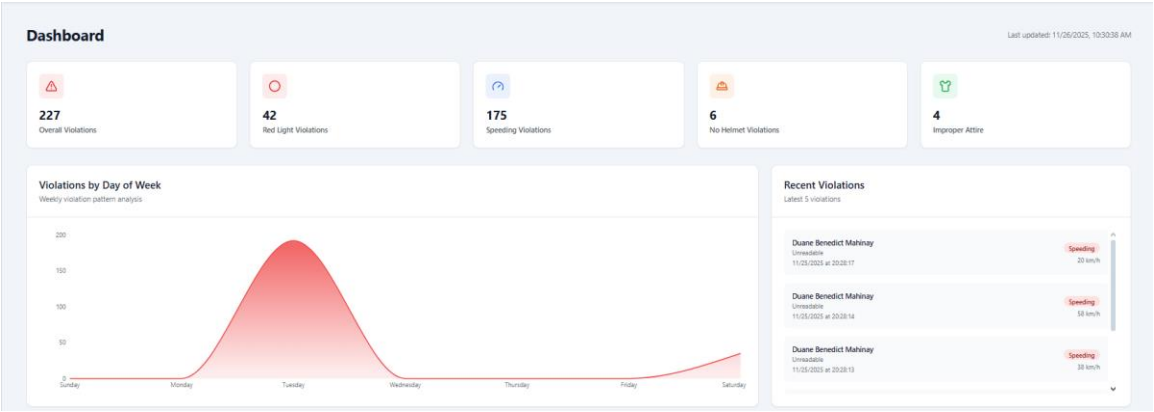


Figure 60. *Contact for MMDA*

APPENDIX F. SAMPLE SYSTEM REPORT





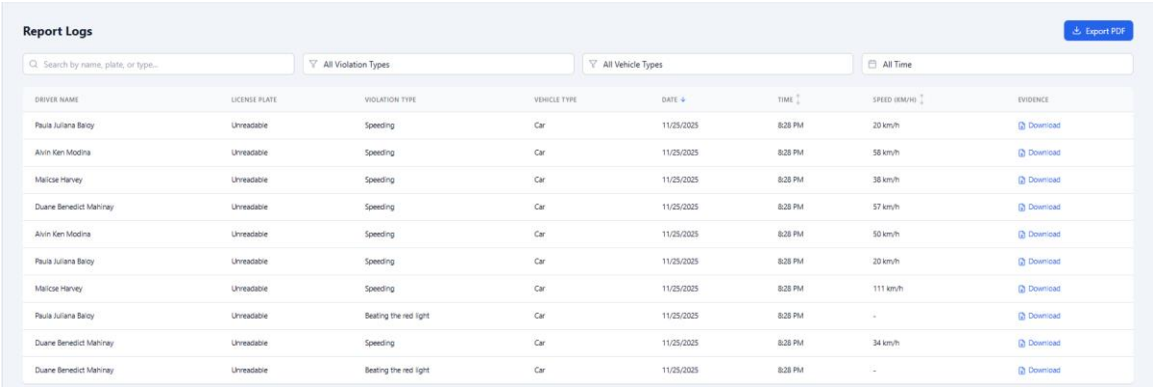


Figure 61. Sample Report System on the Website

APPENDIX G. USER'S MANUAL

TABLE OF CONTENTS

	Page
Abstract	126
Preface	127
Introduction	128
System Access Overview	139
Main Interface Overview	131
Navigation and Features	132
Modules Dashboard Manual	134
User Stoplight	137
Safety Reminders	141
Glossary	145
	145

Abstract

The Manage Traffic Website is a web-based platform that serves as the visual and monitoring interface for the Smart Traffic Flow Optimization System—an automated traffic control solution that uses AI-controlled stoplights to improve road efficiency through real-time monitoring, ambulance detection, and violation management. The system's primary hardware component is an automated stoplight controlled by Arduino Uno, which receives intelligent commands from a Raspberry Pi that processes live video feeds using artificial intelligence.

Designed to address increasing urban traffic congestion and road violations, the platform provides a centralized dashboard for presenting real-time and historical traffic data. Upon powering, the system automatically detects vehicles, ambulances, and traffic violations through camera feeds. Based on these detections, the Raspberry Pi sends control signals to the Arduino Uno, which operates the stoplight accordingly adjusting light timing for optimal traffic flow and prioritizing emergency vehicles. All detected events and stoplight operations are logged and stored in Supabase for comprehensive data storage and review.

The website enables authorized users including traffic management authorities, city administrators, data analysts, system developers, and researchers to monitor automated stoplight operations, observe traffic flow, review recorded violations, and access system-generated reports through intuitive visual representations such as charts, logs, and monitoring views.

Key features include a secure login portal, a user-friendly dashboard displaying real-time stoplight status and traffic visuals, data analytics tools, and downloadable reports. By transforming complex traffic data and stoplight operation logs into clear, organized visual formats, the system helps users evaluate the effectiveness of automated stoplight control, identify problem areas, and assess traffic patterns.

The integrated Smart Traffic Flow Optimization System centered on its intelligent stoplight control and its accompanying Manage Traffic Website contribute to enhanced traffic management and support the development of smarter, data-driven strategies for urban

traffic control.

Preface

This document presents the Manage Traffic Website, a web-based monitoring platform developed as part of the Smart Traffic Flow Optimization System. This project represents our effort to address the growing challenges of urban traffic congestion and road safety through the integration of artificial intelligence, hardware automation, and data visualization.

The development of this system stems from the recognition that traditional traffic management methods often struggle to keep pace with increasing vehicle density and the need for real-time responsiveness. By combining AI-powered detection with automated stoplight control and comprehensive monitoring capabilities, we aim to contribute to the evolution of intelligent transportation systems.

Throughout the development process, we focused on creating a solution that balances automation with accessibility. The Smart Traffic Flow Optimization System operates autonomously through its Arduino-controlled stoplights and Raspberry Pi processing unit, while the accompanying website provides authorized users with clear visibility into system operations and traffic conditions.

This project would not have been possible without the guidance of our advisors, the support of our institution, and the collaborative effort of our development team. We are grateful for the opportunity to work on a system that has practical applications in improving urban traffic management and public safety.

We hope that this system serves as a foundation for further research and development in smart traffic solutions, and that it demonstrates the potential of integrating hardware automation with intelligent monitoring platforms to create more efficient and safer road environments.

This documentation is intended to provide a comprehensive overview of the system's purpose, functionality, and technical implementation for future users, developers, and

researchers who may build upon this work.

Introduction

1.1 Purpose

The Manage Traffic Website is a web-based platform integrated with the Smart Traffic Flow Optimization System, designed to visually present real-time traffic data collected from sensors and monitoring devices. It serves as a centralized dashboard where administrators can observe traffic flow, view recorded violations, and access system-generated reports.

The primary purpose of this website is to provide clear and organized visual representations of traffic conditions. It allows users to easily interpret data such as traffic violations, vehicle movements, and reported violations through charts, logs, and monitoring views. While the website itself does not directly control or optimize traffic flow, it plays an important role in data visualization and monitoring, which supports further analysis and system evaluation.

This system was developed in response to the growing issues of traffic congestion and road violations in urban areas. By offering a platform that presents real-time and historical data, the Manage Traffic Website helps authorities and researchers understand traffic behavior, identify problem areas, and evaluate traffic patterns more effectively.

The Manage Traffic Website acts as the visual and monitoring interface of the Smart Traffic Flow Optimization System. It enables users to observe, analyze, and document traffic-related information, ultimately contributing to better awareness, improved study of traffic flow, and the development of smarter traffic management strategies in the future.

1.2 Target Users

Since the website contains confidential traffic data and system-generated reports, access is limited to authorized personnel only. The following users are granted permission to use the platform:

Traffic Management Authorities – to monitor traffic conditions, review reports, and oversee system data.

City Administrators – to assess traffic performance and use the data for planning or policy development.

Data Analysts – to interpret visualized data, generate insights, and evaluate traffic trends.

System Developers and Testers – to maintain, improve, and ensure the proper functionality of the system.

1.3 System Overview

The system features a secure login portal and a user-friendly dashboard that displays real-time traffic visuals and data analytics. It includes monitoring tools and downloadable reports that help users observe and analyze traffic conditions. Through these features, the system supports better understanding and evaluation of traffic flow and violations.

System Access

2.1 Requirements

The requirements for the website don't heavily need high powered processor or graphics memory, most of it relies on internet connectivity and decent computer machine to run the website.

The Following are the recommended requirements:

Stable Internet Connectivity.

Modern Website Browser (Google Chrome, Microsoft Edge, or Firefox).

Authorized Accounts credentialed by the System Administrator.

2.2 How to Log In

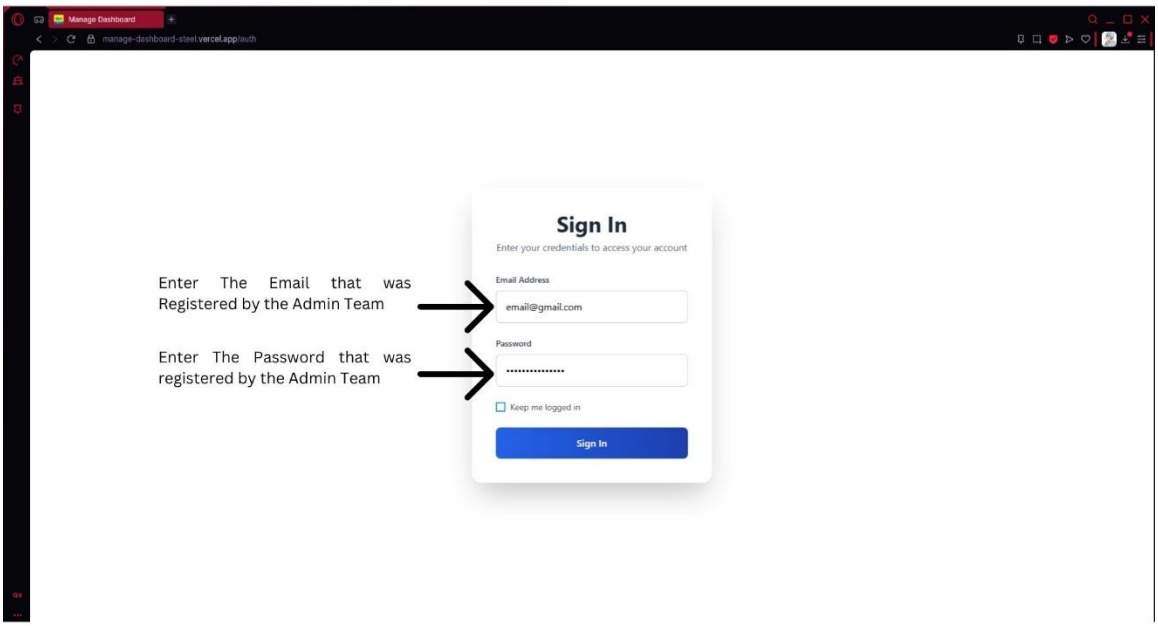


Figure 62. *Sign up page*

Provided by the System Administration Team, they will hand a user a private email containing the Private link to the User. (https://manage-dashboard-steel.vercel.app/)

In The Login page prompt, you will be asked to enter your credentialed email and password that was given by the Administration, through the form that was asked about a small information about the user. You will be also asking if you want to keep on logged in or not

After entering the right credentialed email and password you will then be redirected to the Manage traffic Website.

2.3 Logging Out

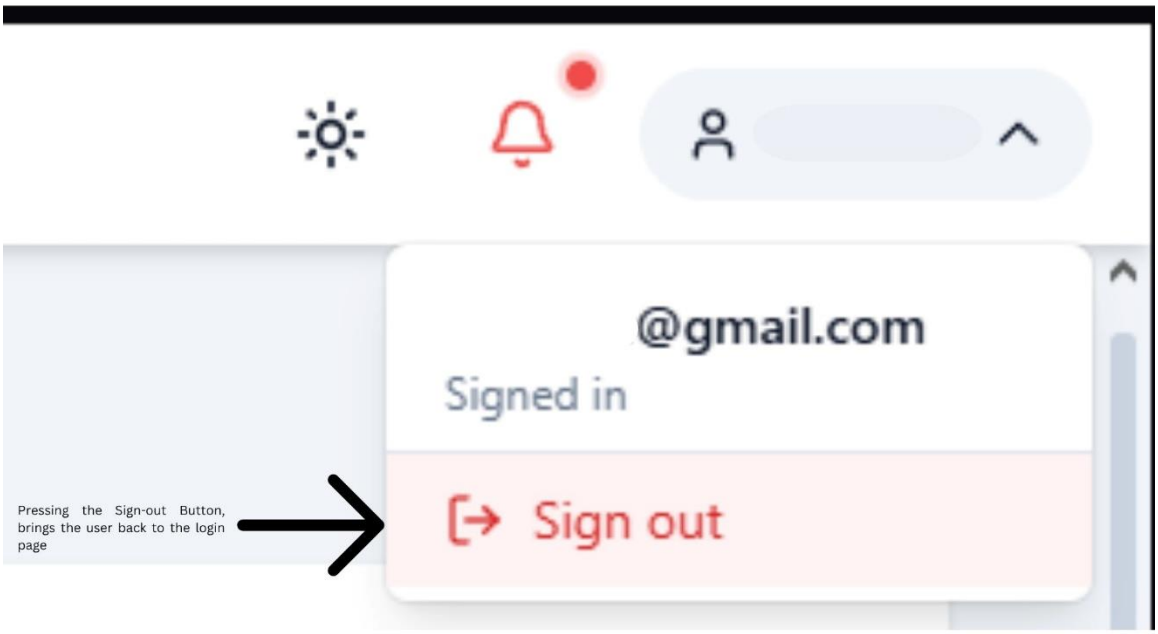


Figure 63. *Log out button*

Looking at the Top-right of the website, you'll be seeing your email name. Pressing it will show the user an option to logout to the website.

If the "Keep me logged in" option is not selected, the user will be automatically logged out after the session and will need to log in again when accessing the website.

3. Main Interface Overview

3.1 Dashboard Overview

In the Dashboard Overview, users can easily view summarized traffic data and recent system activity. It serves as the main page that provides a quick visual representation of recorded traffic violations.

Body of the Page:

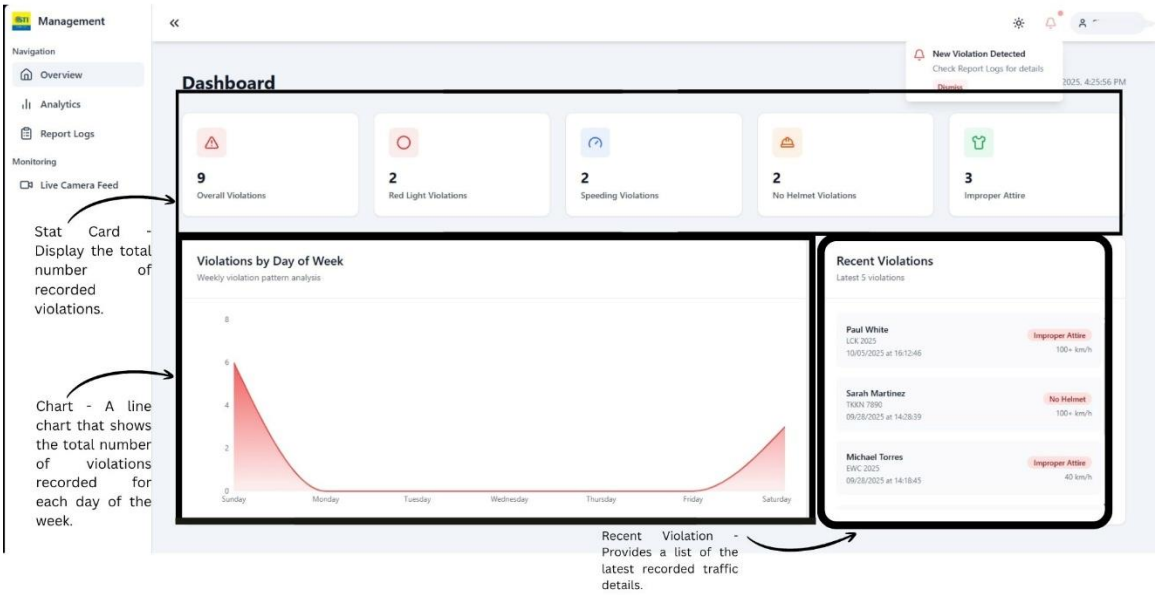


Figure 64. *Dashboard Overview*

Stat Cards – Display the total number of recorded violations, categorized by type such as Beating the Red Light, Speeding, No Helmet, and Incomplete Attire/Gear. These cards give a quick summary of how many incidents have occurred.

Violations by Day of Week (Chart) – A line chart that shows the total number of violations recorded for each day of the week. This helps users visualize when most violations occur and identify traffic trends over time.

Recent Violations (Table) – Provides a list of the latest recorded traffic violations, including details such as the Driver Name, License Plate, Type of Violation, Date, Time, and Speed (KM/H). This allows users to quickly review recent incidents for monitoring and reporting purposes.

4. Navigations Overview

4.1 Side bar Navigation

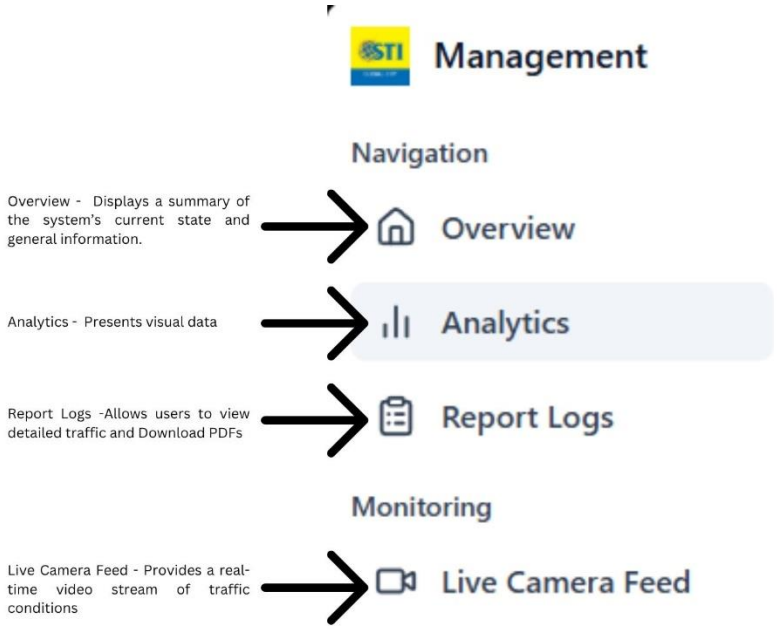


Figure 65. *Side Bar Navigation*

The sidebar provides quick access to the main sections of the system:

Overview - Displays a summary of the system's current state and general information.

Analytics - Presents visual data, such as graphs and charts, to help users analyze traffic patterns and trends.

Report Logs - Allows users to view detailed traffic or system records and download PDF reports for documentation or analysis.

4.2 Head Bar Navigation



Figure 66. *Head bar Icons*

The header, located at the top of the page, contains the following tools:

Dark Mode - Lets users switch between light and dark themes for better visibility and comfort.

Notification Bell - Lets users see how many violations have been detected while the user was away.

Profile icon - Shows the logged-in user's account and provides option such as logging out.

4.3 Monitoring Overview

Live Camera Feed - Provides a real-time video stream of traffic conditions, allowing users to visually monitor road activity and detect possible incidents.

5. Modules and Features

5.1 Dashboard

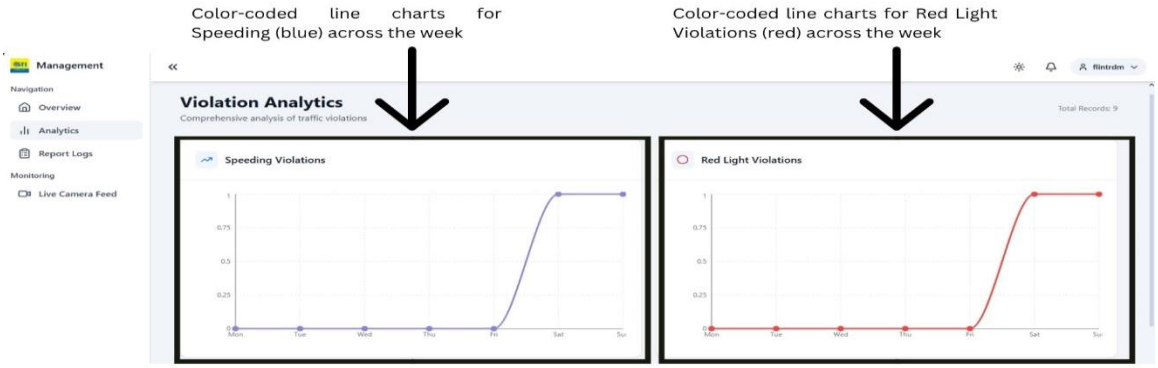
The main landing page displaying traffic violation overview with:

Violation Summary Cards - Total counts for Overall Violations, Beating Red Light, Speeding, No Helmet, and Incomplete Attire

Weekly Trend Chart - Interactive area chart showing violations by day with hover tooltips

Recent Violations Table - Latest incidents with driver name, license plate, violation type, date, time, and speed

5.2 Analytics



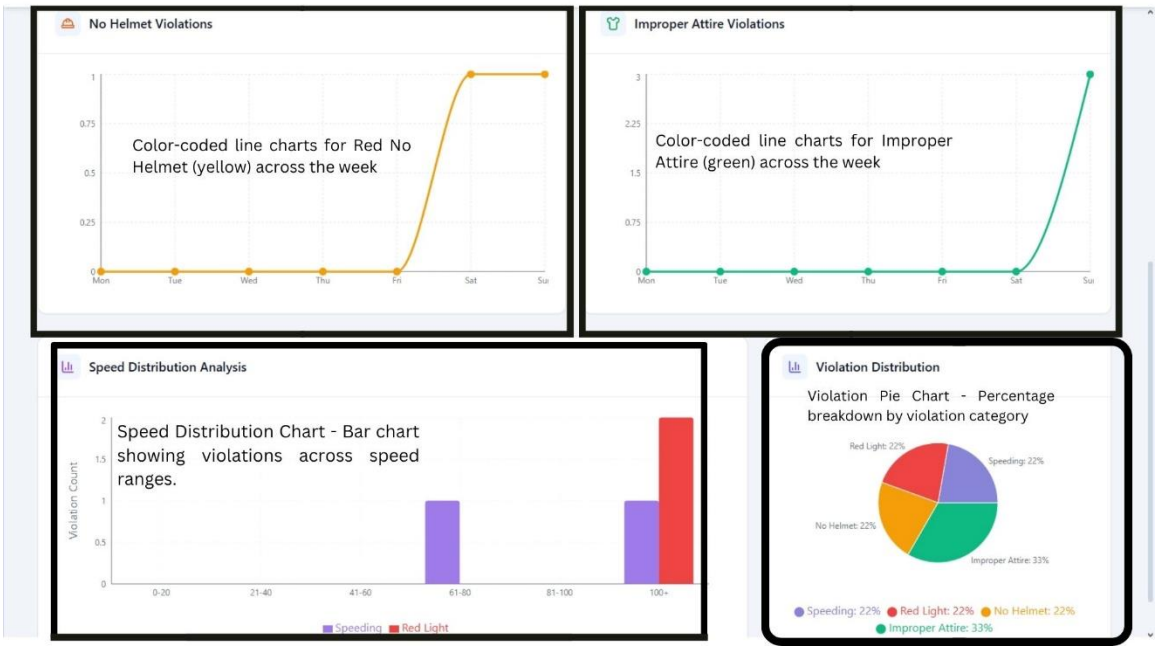


Figure 67. *Line Charts Representation of Violation*

Visual data analysis through multiple chart types:

Violation Trend Charts - Color-coded line charts for Speeding (blue), Red Light (red), No Helmet (yellow/orange), and Incomplete Attire (green) violations across the week

Speed Distribution Chart - Bar chart showing violations across speed ranges (0-20, 21-40, 41-60, 61-80, 81-100, 100+ km/h)

Violation Distribution Pie Chart - Percentage breakdown by violation category with interactive legend

5.3 Report Logs

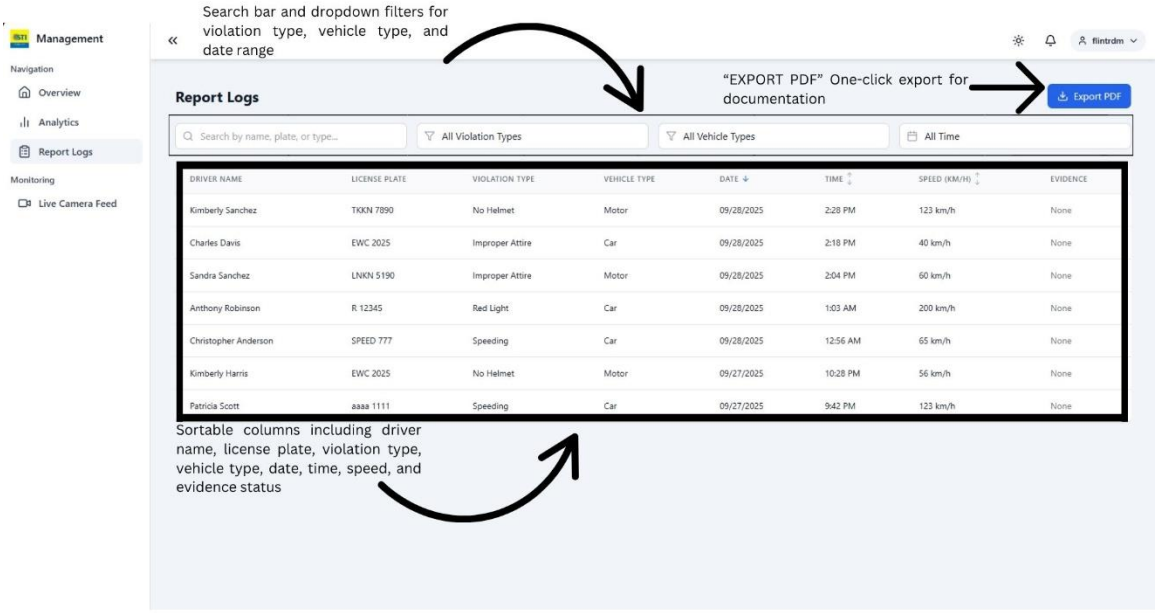


Figure 68. *Report Logs of Violators*

Comprehensive violation database with:

Search & Filters - Search bar and dropdown filters for violation type, vehicle type, and date range

Violations Table - Sortable columns including driver name, license plate, violation type,

vehicle type, date, time, speed, and evidence status

Export PDF - One-click export for documentation, legal proceedings, and reporting

5.4 Monitoring System

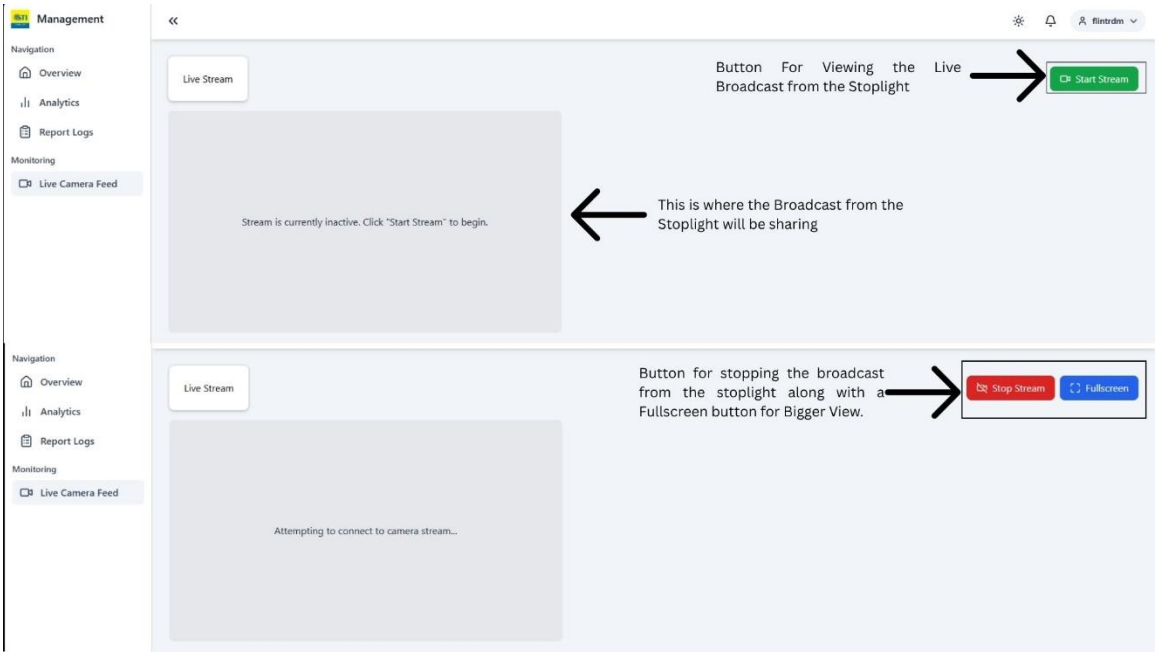


Figure 69. *Monitoring page*

Real-time surveillance capabilities featuring:

Live Stream Interface - Video viewing panel with start/stop controls

Active Monitoring - Real-time observation of traffic flow and violation verification

System Integration - Automatic logging of detected violations to Dashboard and Report Logs

6. User-Dashboard Manual

This system allows you to view and manage your traffic violation records online. You can check your violation history, view detailed summaries, and contact the Metropolitan Manila Development Authority (MMDA) for assistance.

6.1 Getting Started

Accessing Your Account

1. Log in using your registered email address.
2. Your dashboard will display upon successful login.

6.2 Dashboard Overview

When you log in, you'll see your personal dashboard with the following information:

Personal Information Section:

Full Name: Your registered name in the system

Vehicle Type: Type of vehicle associated with your account

Plate Number: Your vehicle's registration number

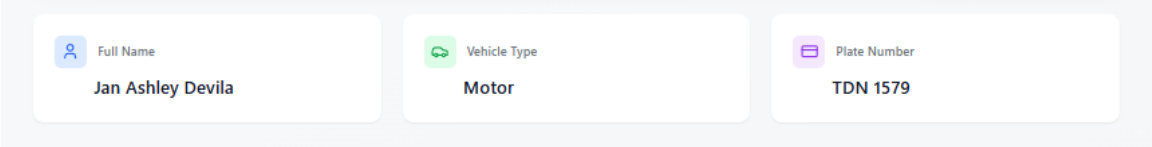


Figure 70. *User Brief Information*

Welcome Message: The system greets you and displays your current violation count (e.g., "You have 1 violation on record")

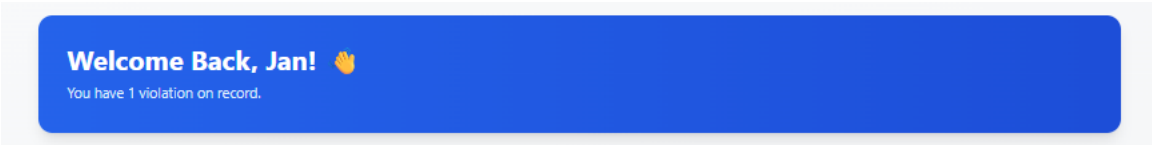


Figure 71. *Welcome and number of violation detection*

6.3 Understanding Your Violations

Violation Categories

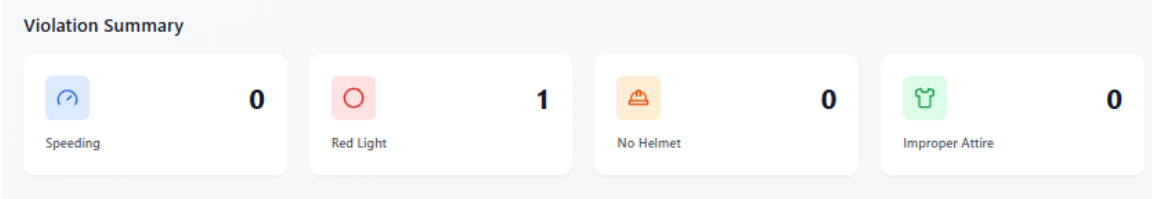


Figure 72. *Violation Summary*

The system tracks four main types of traffic violations:

1. Speeding – Exceeding posted speed limits and Count Displayed in real-time
2. Red Light – Running red traffic lights
3. No Helmet - Riding motorcycles without proper headgear and Applies to both riders and passengers
4. Improper Attire – Dress code violation while driving and Specific to local traffic regulation

Violation Summary

Icon representation, Violation types, and Total counts per violations (numerical)

6.4 Reports Log Section

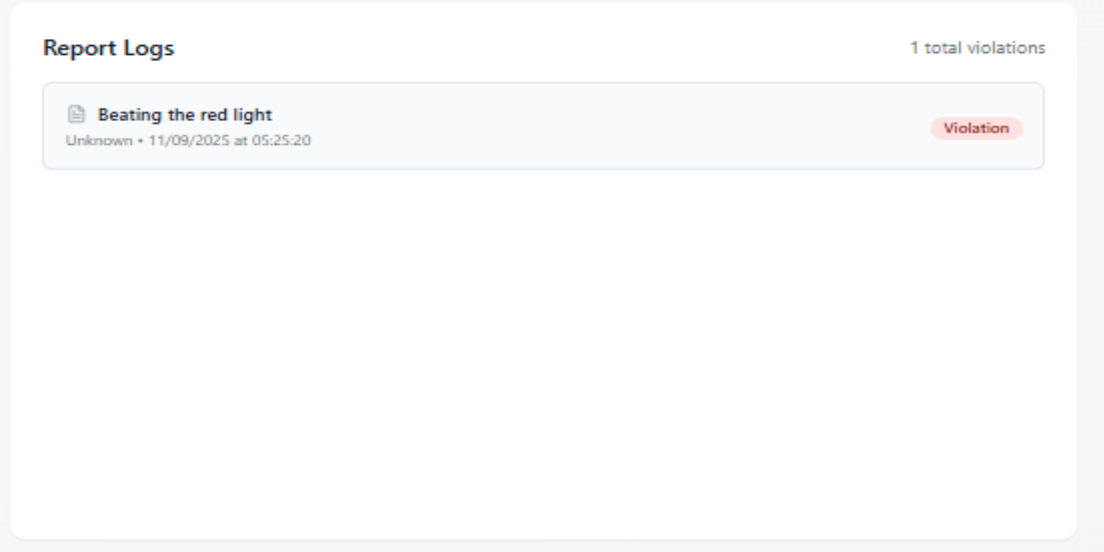


Figure 73. *Report Logs of the User*

Viewing records

In here you'll be viewing different violation logs that as a user have committed, if a user has not committed any violation the user will be prompt with "no violation records" while if there is a violation committed a user will see a Report log containing the user's violation name, date, and time.

6.5 Contact MMDA

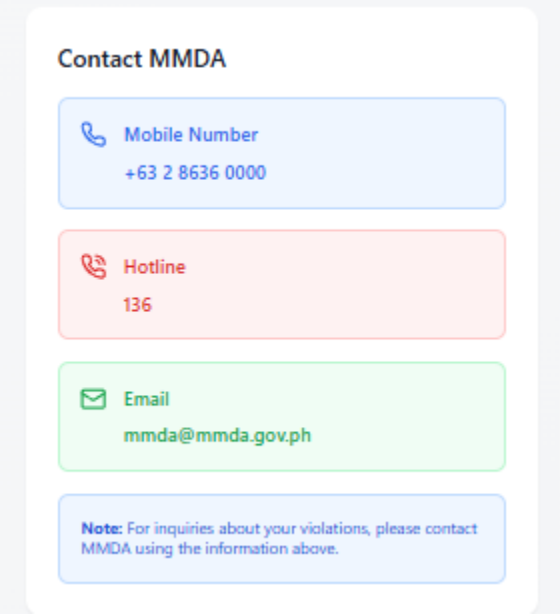


Figure 74. *Contact MMDA Side*

In this side of the webpage there is a "Contact MMDA" where a user may contact if there's any disputes with the records or wants to pay the fines that he has committed.

In the Contact side there are the followings:

1. Mobile Number Available during Business hours and General Inquiries Aswell as Updates.

2. Hotline for 24/7 emergency hotline and quick response when there is an accident or for any emergency queries.

3. Email for detailed inquiries, Submission of documents, and Formal complaints or

disputes.

7. Stoplight

7.1 System Description

The Smart Traffic Flow Optimization System is an automated traffic control solution designed to improve road efficiency through real-time monitoring, ambulance detection, and violation management. It uses artificial intelligence to analyze live video from the camera and control the traffic lights automatically via Arduino and Raspberry Pi

.



Figure 75. *Prototype Stoplight On*



Figure 76. *Prototype Stoplight off*

7.2 Target Users

Since the system manages live traffic data and real-time detections, access is limited to authorized personnel only. The following users are allowed to operate or maintain the system:

Traffic Management Authorities – to monitor intersections and oversee automated stoplight operations.

City Administrators – to assess traffic performance and plan for infrastructure improvements.

System Developers and Testers – to maintain, troubleshoot, and update the system.

Researchers or Demonstrators – for educational and testing purposes under supervision.

7.3 System Overview

The system automatically starts upon powering the Raspberry Pi and continuously operates without manual input. It detects vehicles, ambulances, and traffic violations in real time using a connected camera. Based on detections, the Raspberry Pi sends signals to the Arduino Uno, which controls the stoplights. All events are logged and uploaded to Supabase for data storage and review.

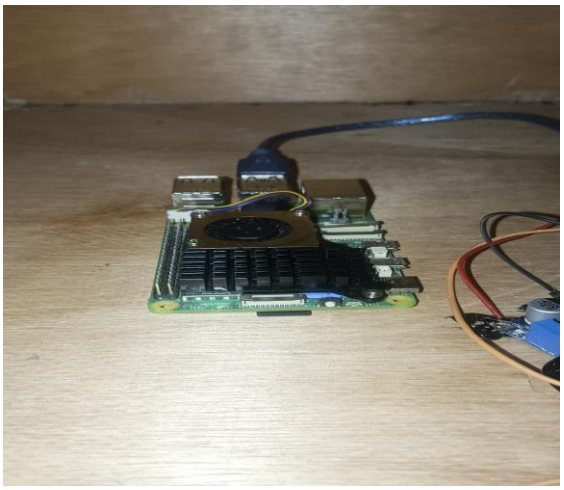


Figure 77**.** *Raspberry Pi 5*

7.4 System Features

Automated Stoplight Control – manages traffic flow without manual switching.

Ambulance Prioritization – automatically turns green for detected emergency vehicles.

Violation Detection – captures red light and other violations, saving images to the database.

Real time monitoring – display detection names and status of the stoplight.

Data storage – uploads captured events to supabase for record-keeping and analysis.



Figure 78. *Camera detected Ambulance*

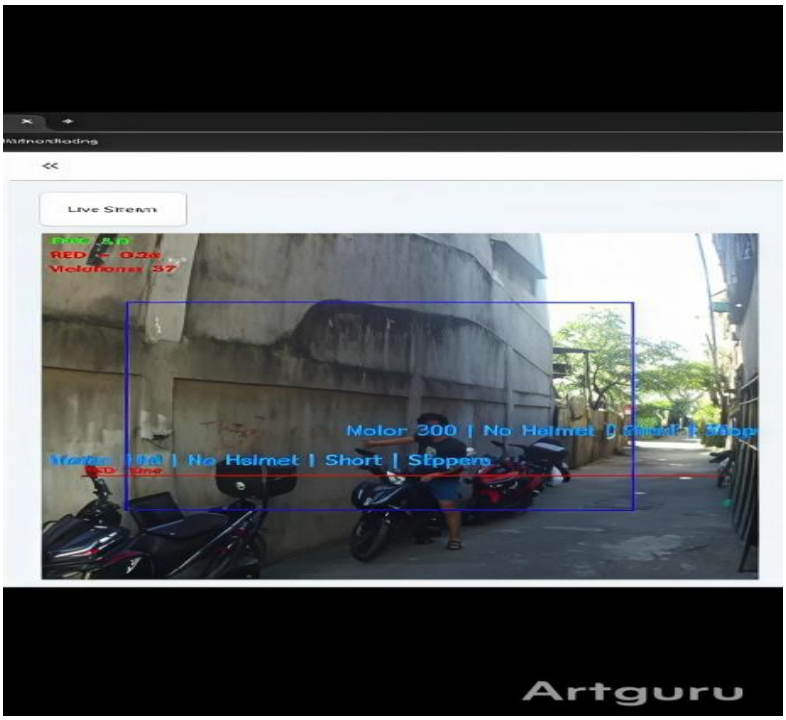


Figure 79. *Camera detected Violation*

7.5 System Operation

1. Turn on the portable power station.

2. The raspberry pi automatically boots and runs the main program.

3. The camera activates and begins detecting vehicles and ambulances.

4. The stoplights respond automatically based on the detected traffic conditions.

5. Detected violations are stored in the Supabase database**.**

7. Troubleshooting Guide

8. Finding Issues, Cause and Solution

Problem	Possible Cause	Solution
Stoplights not lighting up	Loose wire or Arduino not connected	Check connections and restart
Camera not working	Loose or wrong port	Reconnect and reboot Raspberry Pi
No detections showing	YOLO model file missing	Make sure your .pt is in the correct folder
Data not uploading	**I**nternet connection issue	Reconnect Wi-Fi or check Supabase credentials

Table 10. *Fixing Errors Guide*

Safety Reminders

1. Double-check wiring before turning on power**.**

2. Do not unplug the Raspberry pi and Arduino while powered.

3. Always shut down the system properly to avoid corruption.

Glossary

Ambulance Detection - An automated feature that identifies emergency vehicles in real time and adjusts the stoplight timing to prioritize their passage through the intersection.

Analytics - A section of the website that displays visual data analysis through charts and

graphs to help users understand traffic patterns, violation trends, and speed distributions.

Arduino Uno - A microcontroller board that receives signals from the Raspberry Pi and controls the physical stoplight operations in the Smart Traffic Flow Optimization System.

Automated Stoplight Control - A system that manages traffic light operations without manual intervention, adjusting timing based on real-time traffic detection and conditions.

Beating the Red Light - A traffic violation that occurs when a vehicle enters an intersection after the traffic signal has already turned red.

Dashboard - The main interface of the Manage Traffic Website that displays summarized traffic data, violation counts, trends, and recent system activity.

Dark Mode - A display setting that switches the website interface to a darker color scheme to improve visibility and reduce eye strain.

Data Storage - The process of saving detected events, violations, and system logs to the Supabase database for record-keeping, analysis, and future reference.

Head Bar Navigation - The navigation menu located at the top of the website page, containing tools such as the Dark Mode toggle, Notification Bell, and Profile Icon.

Incomplete Attire/Gear - A traffic violation referring to improper dress code or missing safety equipment required by law while operating a vehicle, such as lack of visor or proper footwear.

License Plate - The vehicle registration number captured and recorded during violation detection for identification purposes.

Live Camera Feed - A real-time video stream of traffic conditions that allows users to monitor road activity and verify detected incidents.

Live Stream Interface - The system’s monitoring panel that includes controls to start and stop live surveillance of traffic intersections.

Manage Traffic Website - A web-based platform that serves as the monitoring and visualization interface for the Smart Traffic Flow Optimization System.

MMDA (Metropolitan Manila Development Authority) - The government agency responsible for traffic management and enforcement in Metro Manila, which users may contact regarding disputes or fine payments.

Monitoring System - A feature that provides real-time surveillance through live video feeds and logs violations automatically.

No Helmet - A traffic violation that occurs when motorcycle riders or passengers operate a vehicle without the required protective headgear.

Notification Bell - An icon in the header that alerts users to newly detected violations while they were not actively viewing the system.

Portable Power Station - An external power supply unit that provides electricity to the Raspberry Pi and other hardware components during operation and testing.

Profile Icon - A user interface element that displays the logged-in user's account information and provides access to account settings or logout options.

Raspberry Pi - A compact computer that processes live video feeds using artificial intelligence and sends control signals to the Arduino for stoplight management.

Real-Time Monitoring - Continuous observation and analysis of traffic conditions as they happen, displaying detection results and stoplight status immediately.

Recent Violations Table - A dashboard component listing the latest recorded traffic incidents, including driver name, license plate, violation type, date, time, and speed.

Report Logs - A system module that allows users to view detailed traffic records, apply filters, search violations, and export data as PDF documents.

Search & Filters - Tools that enable users to narrow down violation records using criteria such as violation type, vehicle type, or date range.

Side Bar Navigation - The vertical menu on the website that provides quick access to main sections such as Overview, Analytics, and Report Logs.

Smart Traffic Flow Optimization System - An automated traffic control solution that integrates AI, camera detection, and hardware components to improve road efficiency and manage violations.

Speeding - A traffic violation that occurs when a vehicle exceeds the posted speed limit for a road or zone.

Speed Distribution Chart - A bar chart in the Analytics section showing how frequently violations occur across different speed ranges (0–20, 21–40, 41–60, 61–80, 81–100, 100+ km/h).

Stat Cards - Dashboard elements that display summary statistics of recorded violations categorized by type, providing quick numerical overviews.

Supabase - A cloud-based database platform used to store detected events, violations, images, and system logs for the traffic management system.

System Administrator - Personnel responsible for managing user accounts, credentials, and overall system access.

Traffic Management Authorities - Government officials responsible for monitoring traffic conditions, reviewing reports, and overseeing enforcement operations.

Violation Detection - The automated identification of traffic violations using AI analysis of camera feeds, including capturing evidence and logging incidents.

Violation Distribution Pie Chart - A circular graph that displays the percentage breakdown of violations by category with interactive viewing options.

Violation Summary Cards - Dashboard components that show total counts for various violation types, such as Overall Violations, Beating the Red Light, Speeding, No Helmet, and Incomplete Attire.

Violation Trend Charts - Color-coded line graphs that display the frequency of specific violation types across days of the week for pattern analysis.

Weekly Trend Chart - An interactive area chart on the dashboard that visualizes the total number of violations recorded for each day of the week.

YOLO Model - A machine learning model file (.pt format) used for real-time object detection, enabling the system to identify vehicles, ambulances, and violations from camera footage.

APPENDIX H. DATABASE SCHEMA AND TABLES

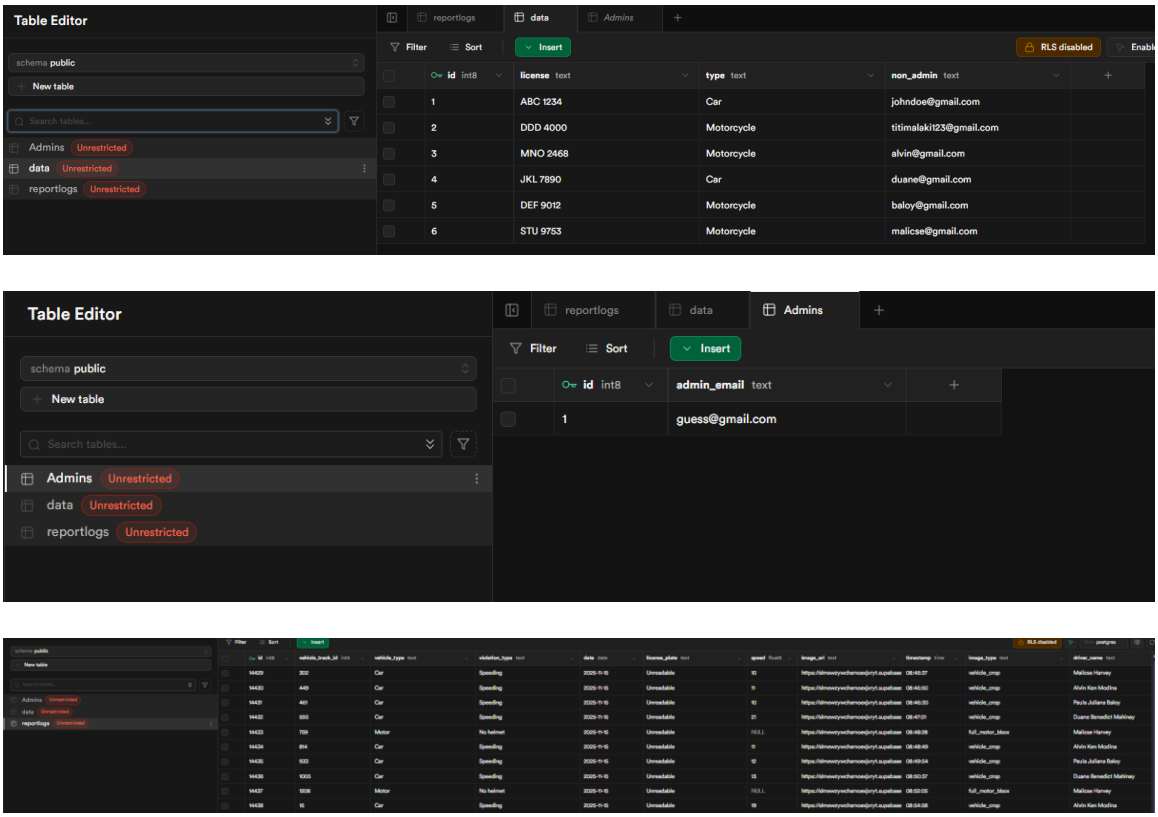


Figure 79. *Database Schema and Tables*

The figures present the Supabase Table Editor with the structure and content of three database tables: report logs, Admins and data. The violations are recorded in the table called *report logs* that include vehicle tracking ID, vehicle type, types of violations, timestamps, links to pictures, and the names of the drivers. The table of admins contains the record of email addresses of the administrators who were stored. There will be a table of data (called data) holding the information of registered vehicles like license number and type of vehicles and even the email of the user. Collectively, these screenshots give a graphical representation of the way the system manages to store and organize the vehicle data, the account of the administrators, and the records of violations.

APPENDIX I. SURVEY QUESTIONNAIRE

SMART TRAFFIC FLOW OPTIMIZATION SYSTEM

NAME: _____ Age: ____ Date: _____

Please answer the questions honestly based on your experience using the Smart Traffic Flow Optimization system.

A. Functionality	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The traffic lights change correctly according to the system's detection.				
2. The system automatically gives a green light when an ambulance is detected				
3. The system can detect traffic violations such as beating the red light.				
4. The traffic light behavior in the software follows the				

correct sequence (red–yellow–green).				
5. The system can detect vehicles and ambulances through its camera.				

B. Performance	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The system performs well even when several vehicles are detected.				
2. The system is designed to respond properly when there are changes in traffic flow				
3. The traffic lights change correctly based on the system's detection during testing.				

4. The system performs well and responds properly based on its current setup and design.				
5. The system runs smoothly during the demonstration without major interruptions				

C. Usability	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The traffic light colors are easy to understand.				
2. The system is easy to use and operate.				
3. The information displayed on the monitor is clear and simple.				

4. The system design is organized and easy to follow.				
5. Even non-technical people can understand how the system works.				

D. Reliability	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The system works properly every time it is tested.				
2. The system is dependable and does not often malfunction.				
3. The system works properly during testing and short periods of continuous use.				
4. The results from the system are consistent in every test.				

5. The system looks stable and reliable based on its performance.				

E. Portability	Strongly Agree	Agree	Disagree	Strongly Disagree
1. I believe this system can also work in other areas that have the same traffic light setup.				
2. The system looks easy to install in other intersections with a similar two-phase design.				
3. The system can be useful in different road situations that follow the same traffic pattern.				
4. The design seems suitable for small intersections like those found on most city roads.				
5. The system can still be improved and expanded in				

the future to support more types of intersections.				

APPENDIX. J SURVEY RESULTS AND TALLY

A. Functionality	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The traffic lights change correctly according to the system's detection.	38%	61%	1%	0%
2. The system automatically gives a green light when an ambulance is detected	47%	47%	6%	0%
3. The system can detect traffic violations such as beating the red light.	39%	54%	7%	0%
4. The traffic light behavior in the software follows the correct sequence (red–yellow–green).	43%	54%	3%	0%
5. The system can detect vehicles and ambulances through its camera.	44%	53%	3%	0%

Table 11. *Functionality Results*

The functionality of the traffic light system received strong positive feedback from respondents. The majority agreed that the system correctly detects and responds to traffic conditions, with 99% agreeing (Strongly agree + Agree) that traffic lights change according to the system's detection. The automatic green light feature for ambulance detection was well-received, with 94% in agreement. Vehicle and ambulance detection through the camera system showed a 97% agreement, demonstrating reliable object recognition capabilities. The system's adherence to proper traffic light sequences (red-yellow-green) achieved a 97% agreement, indicating correct implementation of standard traffic protocols. Violation detection for red light running received 93% positive responses, though this had the highest disagreement rate at 7%, suggesting minor room for improvement in this feature.

B. Performance	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The system performs well even when several vehicles are detected.	29.17%	59.72%	9.72%	1.39%
2. The system is designed to respond properly when there are changes in traffic flow	29.17%	59.72%	11.11%	0%
3. The traffic lights change correctly based on the	31.94%	62.50%	5.56%	0%

system's detection during testing.				
4. The system performs well and responds properly based on its current setup and design.	40.28%	55.56%	4.17%	0%
5. The system runs smoothly during the demonstration without major interruptions	25%	62.50%	11.11%	0%

Table 12. *Performance Result*

The system's performance during testing and demonstration garnered predominantly positive responses, though with slightly more variation than functionality ratings. The system's ability to handle multiple vehicles simultaneously received an 88.89% agreement, with 11.11% expressing concerns about performance under heavy load. Smooth operation during demonstrations achieved an 87.5% agreement, though 11.11% noted interruptions. The system's response to changing traffic flow patterns and detection accuracy during testing both received approximately 89-94% positive feedback. Notably, 95.84% of respondents agreed that the system performs well based on its current set-up and design, indicating strong overall satisfaction despite minor performance concerns under certain conditions.

C. Usability	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The traffic light colors are easy to understand.	38.89%	59.72%	9.72%	1.39%
2. The system is easy to use and operate.	29.17%	59.72%	11.11%	0%
3. The information displayed on the monitor is clear and simple.	50%	45.83%	4.17%	0%
4. The system design is organized and easy to follow.	33.33%	56.94%	9.72%	0%
5. Even non-technical people can understand how the system works.	30.56%	62.50%	6.94%	0%

Table 13. *Usability Results*

Usability ratings revealed strong agreement that the system is user-friendly and accessible. The clarity of information displayed achieved the highest rating in this category, with 95.83% agreement. Traffic light color comprehension received 98.61% agreement,

demonstrating intuitive design. The system's ease of use 88.89% agreement, while 93.06% of respondents found non-technical accessibility satisfactory, suggesting the system is approachable for users without technical backgrounds. System organization and design received 90.27% positive feedback. The slight disagreement percentages (ranging from 4.17% to 11.11%) indicate minor usability challenges that could be addressed through interface refinements or additional user guidance.

D. Reliability	Strongly Agree	Agree	Disagree	Strongly Disagree
1. The system works properly every time it is tested.	30.56%	62.50%	6.94%	0%
2. The system is dependable and does not often malfunction.	30.56%	54.17%	15.28%	0%
3. The system works properly during testing and for short periods of continuous use.	33.33%	63.89%	2.78%	0%
4. The results from the system are consistent in every test.	26.39%	58.33%	15.28%	0%

5. The system looks stable and reliable based on its performance.	33.33%	63.89%	2.78%	0%

Table 14. *Reliability Results*

Reliability assessments showed generally positive results with some areas requiring attention. Proper functionality during testing received a 93.06% agreement, while consistency in test results achieved 84.72% agreement, with 15.28% expressing concerns about result variability. System dependability also received an 84.72% agreement, with the same percentage noting occasional malfunctions. Performance during continuous use periods showed a 97.22% agreement, indicating strong short-term reliability. The system's perceived stability and reliability based on overall performance achieved a 97.22% agreement. While the majority expressed confidence in the system's reliability, the 15.28% disagreement rate regarding consistency and dependability suggests opportunities for stability improvements and more rigorous testing protocols.

E. Portability	Strongly Agree	Agree	Disagree	Strongly Disagree
1. I believe this system can also work in other areas that have the same traffic light setup.	41.67%	52.78%	4.17%	1.39%

2. The system looks easy to install in other intersections with a similar two-phase design.	33.33%	54.17%	11.11%	1.39%
3. The system can be useful in different road situations that follow the same traffic pattern.	36.11%	61.11%	2.78%	0%
4. The design seems suitable for small intersections like those found on most city roads.	44.44%	51.39%	4.17%	0%
5. The system can still be improved and expanded in the future to support more types of intersections.	63.89%	34.72%	1.39%	0%

Table 15. *Portability Results*

Portability received the most enthusiastic responses across all categories. The system's potential for expansion and future improvements achieved the highest rating in the entire

survey at 98.61% agreement, with 63.89% strongly agreeing. Respondents showed strong confidence in the system's adaptability, with 94.45% agreeing it can work in other areas with similar traffic light setups and 97.22% believing it can handle different road situations following the same traffic pattern. The system's suitability for small city intersections received a 95.83% agreement. Installation feasibility at other intersections garnered an 87.5% agreement. The overwhelmingly positive portability ratings demonstrate strong respondent confidence in the system's scalability and potential for widespread deployment across various similar traffic environments.

Summary

The traffic light system received predominantly positive feedback across all evaluation criteria, with agreement rates ranging from 84.72% to 98.61%. Functionality and portability emerged as the strongest aspects, while reliability showed the most room for improvement, particularly regarding consistency and malfunction prevention.

APPENDIX K. SOURCE CODE

Raspberry Pi code

```
import time

import numpy as np

import threading

import torch

from ultralytics import YOLO

from collections import

defaultdict, deque

from supabase import

create_client

import pytesseract

from datetime import datetime

import os

import uuid

import asyncio

import serial

from serial import

SerialException

import cv2

from enum import IntEnum

from flask import Flask,

Response,

render_template_string

 try:
 FLASK_AVAILABLE =
Trueprint("✓ Flask imported
successfully")
except ImportError:
print("✗ Flask not available.
Web streaming will be
disabled.")
print("  Install it with: pip install
Flask")

   FLASK_AVAILABLE =
False

try:

   from picamera2 import
Picamera2
   from libcamera import
controls
  PICAMERA2_AVAILABLE
= True
  print("✓ Picamera2 imported
successfully")
 except ImportError as e:
 print(f"✗ Picamera2 not
available: {e}")
  print("✓ Using fallback
camera sources (webcam/video
file)")
   PICAMERA2_AVAILABLE
= False

 class Picamera2:
       def __init__(self): pass
       def
create_preview_configuration(se
lf, **kwargs): return None
      def configure(self, config):
pass
       def start(self): pass
      def stop(self): pass
       def capture_array(self):
return None

   class controls:

      Pass

class TrafficMonitoringSystem:
   class LightState(IntEnum):
     GREEN = 0
      YELLOW_GREEN_RED
= 1


 RED = 2

      YELLOW_RED_GREEN
= 3

   def __init__(self):

      self.device = 'cuda' if
torch.cuda.is_available() else
'cpu'

      print(f"Using
{self.device.upper()} device")

      self.model =
YOLO("nov14.pt").to(self.devic
e)

      self.model.fuse()

      self.picam2 = None

      self.cap = None

      self._init_camera()

      self.frame_size = (1280,
720)

      self.display_size = (1280,
720)

       self.state_names =
["GREEN", "YELLOW",
"RED" , "YELLOW_RG"]

      self.light_colors = [(0, 255,
0), (0, 255, 255), (0, 0, 255), (0,
255, 255)]

      self.mode_c = {'green': 20,
'yellow_gr': 4, 'red': 10,
'yellow_rg': 4}

      self.mode_d = {'green': 10,
'yellow_gr': 4, 'red': 20,
'yellow_rg': 4}

      self.mode_e = {'green': 15,
'yellow_gr': 4, 'red': 15,
'yellow_rg': 4}

      self.current_mode =
self.mode_c

      self.current_duration =
self.current_mode['green']

      self.last_change_time =
time.time()

       self.current_signal = 'E'


self.AMBULANCE_CLASS_N
AME = "Ambulance"


self.AMBULANCE_LIGHT_CL
ASS_NAME = "Emergency"

      self.ambulance_detected =
False
```

```
self.emergency_light_detected = False

self.ambulance_audio_detected = False
        self.last_siren_time = 0
        self.ambulance_override = False

self.AMBULANCE_OVERRIDE_DURATION = 20
        self.ambulance_first_seen = None
        self.ambulance_last_seen = None

self.AMBULANCE_ABSENCE_THRESHOLD = 4

self.all_red_cooldown_active = False

self.AMBULANCE_COOLDOWN_DURATION = 3.0
        self.post_cooldown_state = self.LightState.RED
        self.arduino = self._init_arduino()
        self.line_start = (50, 600)
        self.line_end = (1230, 600)
        self.speed_area = np.array([
            [240, 150],
            [900, 150],
            [900, 630],
            [240, 630]
        ], np.int32)
    self.road_length_meters = 20.0
    self.pixel_to_meter = self.road_length_meters / 50.0
        self.speed_threshold = 10

self.CONFIDENCE_THRESHOLD = 0.15
        self.track_history = defaultdict(lambda: deque(maxlen=30))
        self.vehicle_speeds = defaultdict(lambda: {
            'positions': deque(maxlen=30),
            'times': deque(maxlen=30),
            'speed': 0,
            'is_speeding': False,
            'first_seen': None
        })
        self.crossing_records = defaultdict(lambda: {'frames': 0, 'violated': False})
        self.counted_ids = {}

        # Motor Violation Status
        self.motor_violation_status = defaultdict(lambda: {
            'no_helmet': False,
            'wearing_shorts': False,
            'wearing_slippers': False,
            'no_visor': False,
            'last_violation_time': 0
        })
        self.violation_count = 0
        self.violations = {}
        # Threading and performance
        self.running = True
        self.latest_frame = None
        self.frame_lock = threading.Lock()
        self.frame_ready = threading.Event()
        # Supabase configuration
        self.SUPABASE_URL = "https://slmewzywchsmoexjxryt.supabase.co"
        self.SUPABASE_KEY = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InNsbWV3enl3Y2hzbW9leGp4cnl0Iiwicm9sZSI6InNlcnZpY2Vfcm9sZSIsImlhdCI6MTc0MzQzMjI3NCwiZXhwIjoyMDU5MDA4Mjc0fQ.wIKAIxX_3rFfzNyhnktH50m2PAXS-xRcHC-2cy7hW5o"
        self.SUPABASE_BUCKET = "test"
        self.supabase = create_client(self.SUPABASE_URL, self.SUPABASE_KEY)
        self.image_storage_dir = "temp_violations"

os.makedirs(self.image_storage_dir, exist_ok=True)

os.makedirs("plates_images", exist_ok=True)
        if os.name == 'posix':

pytesseract.pytesseract.tesseract_cmd = r"/usr/bin/tesseract"
        else:  # Windows

pytesseract.pytesseract.tesseract_cmd = r"C:\Users\Butchoy\AppData\Local\Programs\Tesseract-OCR\tesseract.exe"
```

```
        self.fps_avg = 0.0
        self.alpha = 0.9
        self.colors = {
            "Ambulance": (180, 255, 0),
            "Cars": (200, 0, 200),
            "Helmet": (255, 0, 0),
            "License Plate": (0, 255, 255),
            "Motor": (255, 140, 0),
            "No Helmet": (0, 0, 255),
            "Short": (255, 255, 0),
            "Slippers": (255, 105, 180),
            "Visor": (0, 191, 255),
            "Emergency": (255, 0, 0)
        }

self._make_initial_decision()
        self.light_state = self.LightState.GREEN
        self.current_duration = self.current_mode['green']
        self.helmet_detection_times = {}

self.HELMET_VIOLATION_DELAY = 3

self.no_visor_confirmed_time = {}

self.NO_VISOR_DISPLAY_DELAY = 3
        self.streaming_frame = None

        if FLASK_AVAILABLE:
            self.flask_app = Flask(__name__)

self._setup_flask_routes()
            self.flask_thread = threading.Thread(

target=self.flask_app.run,
                kwargs={'host': '0.0.0.0', 'port': 5000, 'debug': False, 'threaded': True, 'use_reloader': False},
                daemon=True
            )
            self.flask_thread.start()
            print("----------------------------------------------------------")
            print("✓ Web streaming server started!")
            print("  View on your laptop at: http://<your_pi_ip_address>:5000")
            print("----------------------------------------------------------")
        else:
            print("✗ Flask is not installed, web streaming will not be available.")
    def _init_camera(self):
        """Initialize camera with Picamera2 as primary, fallback to other sources"""
        try:
            print("Initializing Picamera2...")
            self.picam2 = Picamera2()
            config = self.picam2.create_preview_configuration(
                main={"size": (640, 480)},
                controls={"FrameRate": 30}
            )

self.picam2.configure(config)
            self.picam2.start()
            time.sleep(2)
            print("Picamera2 initialized successfully!")
            return
        except Exception as e:
            print(f"Picamera2 initialization failed: {e}")
            print("Falling back to webcam (cv2.VideoCapture(0))...")
        self.cap = cv2.VideoCapture("video1.mp4")

self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

self.cap.set(cv2.CAP_PROP_FPS, 30)
        time.sleep(2)
    def _capture_frame(self):
        """Capture a frame from available camera source"""
        try:
            if self.picam2 is not None and PICAMERA2_AVAILABLE:
                frame = self.picam2.capture_array()
                if frame is not None:
                    return cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
```

```
        elif self.cap is not None and self.cap.isOpened():
            ret, frame = self.cap.read()
            if ret:
                return frame
            else:
                frame_count = self.cap.get(cv2.CAP_PROP_FRAME_COUNT)
                current_pos = self.cap.get(cv2.CAP_PROP_POS_FRAMES)
                if frame_count > 0 and current_pos >= frame_count:

self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
                    ret, frame = self.cap.read()
                    if ret: return frame
                    else: return None
                else: return None
        else: return None
    except Exception as e:
        print(f"Frame capture error: {e}")
        return None
 def _init_arduino(self):
    """Initialize Arduino connection with automatic port detection"""
    ports_to_try = ['/dev/ttyACM0', '/dev/ttyACM1', '/dev/ttyUSB0', '/dev/ttyUSB1', '/dev/ttyAMA0', 'COM3', 'COM4', 'COM5']
    for port in ports_to_try:
        try:
            arduino = serial.Serial(port, 9600, timeout=1)
            time.sleep(2)
            print(f"✓ Connected to Arduino on {port}")
            return arduino
        except SerialException: continue
        except Exception as e: print(f"✗ Error connecting to Arduino on {port}: {e}")
    print("✗ Arduino not found on any standard ports.")
    return None
 def _make_initial_decision(self):
    """Initialize traffic mode based on first frame analysis"""
    frame = self._capture_frame()
    if frame is not None:
        results = self.model(frame, verbose=False)
        vehicle_count1 = len(results[0].boxes) if results[0].boxes else 0
        vehicle_count2 = 1
        command = b'A'
        if vehicle_count1 > vehicle_count2:
            print(f"Initial decision: Lane 1 has more vehicles ({vehicle_count1} > {vehicle_count2}). Starting Mode C.")
            self.current_mode = self.mode_c
        elif vehicle_count1 < vehicle_count2:
            print(f"Initial decision: Lane 2 has more vehicles ({vehicle_count1} < {vehicle_count2}). Starting Mode D.")
            self.current_mode = self.mode_d
        else:
            print(f"Initial decision: Both lanes have equal vehicles ({vehicle_count1} = {vehicle_count2}). Starting Mode E.")
            self.current_mode = self.mode_e

self._send_arduino_command(command)
        self.light_state = self.LightState.GREEN
        self.current_duration = self.current_mode['green']
        self.last_change_time = time.time()
    else:
        print("[ERROR] No initial frame. Defaulting to Mode C, RED light.")
        self.current_mode = self.mode_c
        self.light_state = self.LightState.RED
        self.current_duration = self.current_mode['red']
        self.last_change_time = time.time()

 def _send_arduino_command(self, command):
```

```
        """Thread-safe Arduino communication"""
        if self.arduino and self.arduino.is_open:
            try:
                self.arduino.write(command)
                self.arduino.flush()
            except SerialException as e: print(f"Arduino comm error: {e}")
            except Exception as e: print(f"Unexpected Arduino error: {e}")
    def run_async(self, func, *args, **kwargs):
        """Run asynchronous operations in a new thread"""
        def run_in_thread():
            coro = func(*args, **kwargs)
            loop = asyncio.new_event_loop()
            asyncio.set_event_loop(loop)
            try:
                loop.run_until_complete(coro)
            except Exception as e:
                print(f"[ASYNC THREAD ERROR] Violation logging failed: {e}")
                pass
            finally:
                loop.call_soon_threadsafe(loop.stop)
        threading.Thread(target=run_in_thread, daemon=True).start()
def update_traffic_light(self):
        """Update traffic light state machine with complete cycle control"""
        current_time = time.time()
        elapsed = 0.0
 if time.time() - self.last_siren_time > 1.5:
                self.ambulance_audio_detected = False
    if self.last_change_time is not None: elapsed = current_time - self.last_change_time
        else: return
  if (self.ambulance_detected and
                self.emergency_light_detected and
                self.ambulance_audio_detected and
                not self.ambulance_override):
            self._activate_ambulance_override(); return
        if self.ambulance_override:
                if elapsed >= self.AMBULANCE_OVERRIDE_DURATION:
                    self._deactivate_ambulance_override()
                return
            time_since_last_seen = current_time - (self.ambulance_last_seen or 0)
            if self.ambulance_last_seen is not None and time_since_last_seen > self.AMBULANCE_ABSENCE_THRESHOLD:
                self._deactivate_ambulance_override()
                return
        return
        if self.all_red_cooldown_active:
            if elapsed >= self.AMBULANCE_COOLDOWN_DURATION:
                print("Ambulance cooldown tapos na. Babalik sa normal state.")
                self.all_red_cooldown_active = False
                self.light_state = self.post_cooldown_state
                duration_key = 'red'
                if self.light_state == self.LightState.GREEN: duration_key = 'green'
                elif self.light_state == self.LightState.YELLOW_GREEN_RED: duration_key = 'yellow_gr'
                elif self.light_state == self.LightState.YELLOW_RED_GREEN: duration_key = 'yellow_rg'
                self.current_duration = self.current_mode[duration_key]
                self.last_change_time = current_time
                command_map = {
                    self.LightState.GREEN: b'A',
                    self.LightState.YELLOW_GREEN_RED: b'Y',
```

```
self.LightState.RED: b'B',

self.LightState.YELLOW_RED_GREEN: b'Y'
            }

self._send_arduino_command(command_map.get(self.light_state, b'B'))
            print(f"Nag-transition sa normal state: {self.light_state.name}")
        else:
            return
  if elapsed >= self.current_duration:
self._transition_to_next_state()
   def _activate_ambulance_override(self):
        """Force Lane 1 green for ambulance"""
        print("Activating ambulance override")

self._send_arduino_command(b'O')
        self.ambulance_override = True
        self.last_change_time = time.time()
        self.ambulance_detected = False
        self.light_state = self.LightState.GREEN
        self.current_duration = self.AMBULANCE_OVERRIDE_DURATION
   def _deactivate_ambulance_override(self):
        """Return to normal operation by starting an all-red cooldown"""
        print("Deactivating ambulance override")
        self.ambulance_override = False
        self.ambulance_last_seen = None
        self.ambulance_first_seen = None
        self.ambulance_detected = False

self.emergency_light_detected = False

self.ambulance_audio_detected = False
            print(f"Starting {self.AMBULANCE_COOLDOWN_DURATION}-second ALL RED cooldown...")

self.all_red_cooldown_active = True
        self.last_change_time = time.time()

self._send_arduino_command(b'R')
            frame = self._capture_frame()
        if frame is not None:
            results = self.model(frame, verbose=False)
            vehicle_count1 = len(results[0].boxes) if results[0].boxes else 0
            vehicle_count2 = 1
              if vehicle_count1 > vehicle_count2:

self.post_cooldown_state = self.LightState.GREEN
            elif vehicle_count1 < vehicle_count2:

self.post_cooldown_state = self.LightState.RED
            else:

self.post_cooldown_state = self.LightState.GREEN
        else:
            self.post_cooldown_state = self.LightState.RED
          self.light_state = self.LightState.RED
        self.current_duration = self.AMBULANCE_COOLDOWN_DURATION
        print(f"Will return to {self.post_cooldown_state.name} after cooldown")
    def _transition_to_next_state(self):
        """Handle state transitions with proper timing"""
        current_time = time.time()
        frame = self._capture_frame()
        if frame is not None:
            results = self.model(frame, verbose=False)
            vehicle_count1 = len(results[0].boxes) if results[0].boxes else 0
            vehicle_count2 = 1
              if vehicle_count1 > vehicle_count2:
```

```
            target_mode =
self.mode_c

            mode_name = "C"

        elif vehicle_count1 <
vehicle_count2:

            target_mode =
self.mode_d

            mode_name = "D"

        else:

            target_mode =
self.mode_e

            mode_name = "E"

            if self.current_mode !=
target_mode:

            print(f"Switching to
Mode {mode_name}:
Lane1={vehicle_count1},
Lane2={vehicle_count2}")

            self.current_mode =
target_mode

    next_state_map = {

        self.LightState.GREEN:
(self.LightState.YELLOW_GRE
EN_RED, b'Y', 'yellow_gr'),


self.LightState.YELLOW_GRE
EN_RED: (self.LightState.RED,
b'B', 'red'),

        self.LightState.RED:
(self.LightState.YELLOW_RED
_GREEN, b'Y', 'yellow_rg'),


self.LightState.YELLOW_RED_
GREEN:
(self.LightState.GREEN, b'A',
'green')

    }

    if self.light_state in
next_state_map:

        next_state, command,
duration_key =
next_state_map[self.light_state]
self._send_arduino_command(co
mmand)

            self.light_state =
next_state

            self.current_duration =
self.current_mode[duration_key]

            self.last_change_time =
current_time

    else: print(f"[WARN]
Unknown light state:
{self.light_state}")

    def process_frame(self,
frame):

    """Process frame with
object detection and tracking"""

    start_time = time.time()

    try: results =
self.model.track(frame,
persist=True, verbose=False)

    except Exception as e:
print(f"[ERROR] Model
inference failed: {e}"); return
frame


self.emergency_light_detected =
False

        detections = []

    if results and
results[0].boxes is not None:

            boxes =
results[0].boxes.xyxy.cpu().num
py().astype(int)

            track_ids_tensor =
results[0].boxes.id

            if track_ids_tensor is not
None:

                track_ids =
track_ids_tensor.cpu().numpy().
astype(int)

                confidences =
results[0].boxes.conf.cpu().nump
y()

                class_ids =
results[0].boxes.cls.cpu().numpy
().astype(int)

                current_time_track =
time.time()

                for box, track_id, conf,
cls in zip(boxes, track_ids,
confidences, class_ids):

                    if conf <
self.CONFIDENCE_THRESHO
LD: continue

                    class_name =
self.model.names.get(cls,
f"Unknown_ID_{cls}")

                    center = ((box[0] +
box[2]) // 2, (box[1] + box[3]) //
2)


detections.append({'box': box,
'track_id': track_id, 'class_name':
class_name, 'center': center})


self.track_history[track_id].appe
nd(center)

                    if track_id not in
self.vehicle_speeds:
self.vehicle_speeds[track_id] =
{'positions': deque(maxlen=30),
'times': deque(maxlen=30),
'speed': 0, 'is_speeding': False,
'first_seen': current_time_track}


self.vehicle_speeds[track_id]['po
sitions'].append(center)


self.vehicle_speeds[track_id]['ti
mes'].append(current_time_track
)

                    class_name_lower =
class_name.lower()

                if class_name_lower
==
self.AMBULANCE_CLASS_N
AME.lower():
```

```
self._check_ambulance(class_name_lower, current_time_track)

            elif class_name_lower == self.AMBULANCE_LIGHT_CLASS_NAME.lower():

                if self._is_light_flashing(frame, box):


self._check_ambulance(class_name_lower, current_time_track)

        if detections:

            try:


self._check_violations(frame, detections)

                current_draw_time = time.time()

                for det in detections:

                    box = det['box']; track_id = det['track_id']; raw_class_name = det['class_name']

                    center = det['center']

                    x1, y1, x2, y2 = box


raw_class_name_lower = raw_class_name.lower()

                if raw_class_name_lower in ["no helmet", "short", "slippers"]:

                    continue

                if raw_class_name_lower not in ["motor", "motorcycle", "car", "cars", "ambulance", "emergency", "helmet", "visor", "license plate", "license_plate"]:

                    continue

                    display_name = self._map_class_name(raw_class_name)


motor_id_for_display = track_id if raw_class_name_lower in ["motor", "motorcycle"] else None

                violation_label = ""

                if motor_id_for_display is not None and motor_id_for_display in self.motor_violation_status:

                    status = self.motor_violation_status[motor_id_for_display]

                    violations = []

                    if status.get('no_helmet'): violations.append("No Helmet")

                    if status.get('wearing_shorts'): violations.append("Short")

                    if status.get('wearing_slippers'): violations.append("Slippers")

                    if status.get('no_visor'): violations.append("No Visor")

                    if violations:

                        violation_label = " | " + " | ".join(violations)

                color = self.colors.get(display_name, (255, 255, 255))

                label = f"{display_name} {track_id}{violation_label}"

                if raw_class_name_lower in ["motor", "motorcycle", "car", "cars"]:

                    if self._point_in_polygon(center, self.speed_area):

                        live_speed = self._calculate_speed(track_id)

                        if live_speed > 2:

                            label += f" [{int(live_speed)} km/h]"

                    if raw_class_name_lower == "emergency":

                        label = "Emergency Light"

                        color = self.colors.get("Emergency", (0, 0, 255))

                    elif raw_class_name_lower == "ambulance" and self.ambulance_detected:

                        label = f"AMBULANCE - EMERGENCY MODE"

                        color = (0, 255, 255)

                    label_pos_y = y1 - 10 if y1 > 20 else y1 + 20

                    label_pos_x = x1 + 5

                    cv2.putText(frame, label, (label_pos_x, label_pos_y), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

            except Exception as e: print(f"[ERROR] Violation/Drawing error: {e}")

            frame_time = time.time() - start_time

            fps = 1.0 / frame_time if frame_time > 0 else 0
```

```
        if self.fps_avg == 0.0: self.fps_avg = fps
        else: self.fps_avg = self.alpha * self.fps_avg + (1 - self.alpha) * fps
        return frame

    def _get_iou(self, boxA, boxB):
        """Compute IoU of two bounding boxes (x1, y1, x2, y2)"""
        xA = max(boxA[0], boxB[0]); yA = max(boxA[1], boxB[1])
        xB = min(boxA[2], boxB[2]); yB = min(boxA[3], boxB[3])
        interArea = max(0, float(xB - xA)) * max(0, float(yB - yA))
        boxAArea = float((boxA[2] - boxA[0]) * (boxA[3] - boxA[1]))
        boxBArea = float((boxB[2] - boxB[0]) * (boxB[3] - boxB[1]))
        unionArea = boxAArea + boxBArea - interArea
        return interArea / unionArea if unionArea > 0 else 0.0

    def _check_ambulance(self, class_name_lower, current_time):
        """Helper to update ambulance detection status"""
        if class_name_lower == self.AMBULANCE_CLASS_NAME.lower():
            self.ambulance_last_seen = current_time
            if self.ambulance_first_seen is None:
                self.ambulance_first_seen = current_time
        if class_name_lower == self.AMBULANCE_LIGHT_CLASS_NAME.lower():
            self.emergency_light_detected = True

AMBULANCE_CONFIRM_DURATION = 0.5
            if self.ambulance_first_seen is not None and self.emergency_light_detected:
                if (current_time - self.ambulance_first_seen) >= AMBULANCE_CONFIRM_DURATION:
                    if not self.ambulance_detected:
                        self.ambulance_detected = True
            else:
                self.ambulance_detected = False
        if (self.ambulance_last_seen is not None and
                current_time - self.ambulance_last_seen > self.AMBULANCE_ABSENCE_THRESHOLD):
            self.ambulance_detected = False
            self.ambulance_first_seen = None

    def _map_class_name(self, raw_class_name):
        """Normalize class names from model output for display"""
        raw_class_name_lower = raw_class_name.lower()
        class_map = {
            "ambulance": "Ambulance", "car": "Cars", "cars": "Cars", "helmet": "Helmet",
            "license plate": "License Plate", "license_plate": "License Plate",
            "motor": "Motor", "motorcycle": "Motor", "no helmet": "No Helmet",
            "short": "Short", "slippers": "Slippers", "visor": "Visor",
            "emergency": "Emergency Light"
        }
        return class_map.get(raw_class_name_lower, raw_class_name.capitalize())

    def _check_violations(self, frame, detections):
        """Detect and handle traffic violations based on all detections in frame"""
        motor_detections = [d for d in detections if d['class_name'].lower() in ["motor", "motorcycle"]]
        helmet_detections = [d for d in detections if d['class_name'].lower() == "helmet"]
        visor_detections = [d for d in detections if d['class_name'].lower() == "visor"]
        license_plate_detections = [d for d in detections if d['class_name'].lower() in ["license plate", "license_plate"]]
        if license_plate_detections and
```

```
len(license_plate_detections) > 0:

            if hasattr(self, 'license_plate_print_counter'):

                self.license_plate_print_counter += 1

            else:

                self.license_plate_print_counter = 0

            if self.license_plate_print_counter % 10 == 0:

                print(f"✅ License plates detected: {len(license_plate_detections)}")

        for det in detections:

            box = det['box']; track_id = det['track_id']; raw_class_name = det['class_name']

            center = det['center']; raw_class_name_lower = raw_class_name.lower()

            if raw_class_name_lower in ["ambulance", "emergency"]: continue

            speed = 0

            is_motor_or_car = raw_class_name_lower in ["motor", "motorcycle", "car", "cars"]

            if is_motor_or_car and self._point_in_polygon(center, self.speed_area):

                speed = self._calculate_speed(track_id)


                if speed >= self.speed_threshold:

                    if track_id not in self.counted_ids:

                        self.counted_ids[track_id] = set()

                    if "Speeding" not in self.counted_ids[track_id]:

                        log_vehicle_type = "Motor" if raw_class_name_lower in ["motor", "motorcycle"] else "Cars"

                        license_plate_box = self._find_associated_license_plate(box, license_plate_detections)

                        target_box = license_plate_box if license_plate_box is not None else box

                        extra_motors = motor_detections if log_vehicle_type == "Motor" else []

                        self._handle_violation(frame, target_box, track_id, "Speeding", speed, log_vehicle_type, extra_motors, license_plate_box is not None)

                        self.counted_ids[track_id].add("Speeding")

                if self._is_red_light_active():

                    line_y = self.line_start[1]

                    y1, y2 = box[1], box[3]

                    if (y1 <= line_y <= y2 - 5):

                        if track_id not in self.counted_ids:

                            self.counted_ids[track_id] = set()

                        if "Beating the red light" not in self.counted_ids[track_id]:

                            log_type = None

                            if raw_class_name_lower in ["motor", "motorcycle"]:

                                log_type = "Motor"

                            elif raw_class_name_lower in ["car", "cars"]:

                                log_type = "Cars"

                            if log_type is not None:

                                print(f"[DEBUG] Red light violation detected! ID={track_id}, Class={log_type}")

                                license_plate_box = self._find_associated_license_plate(box, license_plate_detections)

                                target_box = license_plate_box if license_plate_box is not None else box

                                extra_motors = motor_detections if log_type == "Motor" else []

                                self._handle_violation(frame, target_box, track_id, "Beating the red light", None, log_type, extra_motors, license_plate_box is not None)

                                self.counted_ids[track_id].add("Beating the red light")

        violation_type = None; violation_key = None; is_component = False
```

```
        if raw_class_name_lower == "no helmet":
            violation_type = "No helmet"; violation_key = 'no_helmet'; is_component = True
        elif raw_class_name_lower == "short":
            violation_type = "Wearing Short"; violation_key = 'wearing_shorts'; is_component = True
        elif raw_class_name_lower == "slippers":
            violation_type = "Wearing Slippers"; violation_key = 'wearing_slippers'; is_component = True
        if is_component:
            associated_motor_id = self._find_associated_motor(box, motor_detections)
            if associated_motor_id is not None:
                motor_id = associated_motor_id
                if not self.motor_violation_status[motor_id].get(violation_key, False):
                    motor_box = None
                    for motor_det in motor_detections:
                        if motor_det['track_id'] == motor_id:
                            motor_box = motor_det['box']
                            break
                    if motor_box is not None:
                        license_plate_box = self._find_associated_license_plate(motor_box, license_plate_detections)
                        target_box = license_plate_box if license_plate_box is not None else box
                        self._handle_violation(frame, target_box, motor_id, violation_type, None, "Motor", motor_detections, license_plate_box is not None)
                        self.motor_violation_status[motor_id][violation_key] = True
                        self.motor_violation_status[motor_id]['last_violation_time'] = time.time()
        for helmet_det in helmet_detections:
            self._check_visor_violation(frame, helmet_det, motor_detections, visor_detections, license_plate_detections)
    def _find_associated_motor(self, component_box, motor_detections):
        """Find motorcycle associated with a component using IoU"""
        for motor_det in motor_detections:
            motor_id_candidate = motor_det['track_id']
            original_motor_box = motor_det['box']
            motor_x1, motor_y1, motor_x2, motor_y2 = map(int, original_motor_box)
            height = motor_y2 - motor_y1
            if height <= 0: continue
            extension_factor = 1.5
            new_motor_y1 = max(0, motor_y1 - int(extension_factor * height))
            enlarged_motor_box = (motor_x1, new_motor_y1, motor_x2, motor_y2)
            iou = self._get_iou(component_box, enlarged_motor_box)
            if iou > 0.0:
                return motor_id_candidate
        return None
    def _find_associated_license_plate(self, vehicle_box, license_plate_detections):
        """
        Find license plate associated with a vehicle.
        FIX: Uses CENTER POINT check instead of pure IoU because plates are much smaller than cars,
        causing IoU to be very low (often < 0.1) even when the plate is inside.
        """
        vx1, vy1, vx2, vy2 = vehicle_box
        for lp_det in license_plate_detections:
            lp_box = lp_det['box']
```

```
            lp_x_center = (lp_box[0]
+ lp_box[2]) / 2

            lp_y_center = (lp_box[1]
+ lp_box[3]) / 2

        if (vx1 <= lp_x_center <=
vx2) and (vy1 <= lp_y_center
<= vy2):

            if hasattr(self,
'license_plate_association_count
er'):


self.license_plate_association_co
unter += 1

                else:
self.license_plate_association_co
unter = 0

            if
self.license_plate_association_co
unter % 5 == 0:

                print(f"✅ Plate
Found via Center-Point Check!
(ID: {lp_det['track_id']})")

 return lp_box

   return None



    def
_check_visor_violation(self,
frame, helmet_det,
motor_detections,
visor_detections,
license_plate_detections):

        """Check for no visor
violation using motorcycle
association"""

        box = helmet_det['box']

        track_id =
helmet_det['track_id']

        current_time = time.time()

        associated_motor_id =
self._find_associated_motor(box
, motor_detections)

        if associated_motor_id is
None:

            return

        motor_id =
associated_motor_id

        has_visor = False

        for visor_det in
visor_detections:

            if self._get_iou(box,
visor_det['box']) > 0.1:

                has_visor = True

                break

    if not has_visor:

            if motor_id not in
self.helmet_detection_times:


self.helmet_detection_times[mot
or_id] = current_time

                start_time =
self.helmet_detection_times.get(
motor_id)

            if start_time is not None:

                if (current_time -
start_time) >=
self.HELMET_VIOLATION_D
ELAY:

                    if "No Visor" not in
self.counted_ids:


self.counted_ids["No Visor"] =
set()

                        if motor_id not
in self.counted_ids["No Visor"]:

                            motor_box =
None

                            for motor_det in
motor_detections:

                                if
motor_det['track_id'] ==
motor_id:

                                    motor_box =
motor_det['box']

                                    break

                            if motor_box
is not None:


license_plate_box =
self._find_associated_license_pl
ate(motor_box,
license_plate_detections)

                target_box =
license_plate_box if
license_plate_box is not None
else motor_box


self._handle_violation(frame,
target_box, motor_id, "No
Visor", None, "Motor",
motor_detections,
license_plate_box is not None)


self.counted_ids["No
Visor"].add(motor_id)


self.no_visor_confirmed_time[m
otor_id] = current_time


self.motor_violation_status[moto
r_id]['no_visor'] = True


self.motor_violation_status[moto
r_id]['last_violation_time'] =
current_time

                    if motor_id in
self.helmet_detection_times:

                        del
self.helmet_detection_times[mot
or_id]

                else:

                    if motor_id in
self.helmet_detection_times:

                del
self.helmet_detection_times[mot
or_id]
```

```
        if motor_id in
self.no_visor_confirmed_time:

            del
self.no_visor_confirmed_time[m
otor_id]


self.motor_violation_status[moto
r_id]['no_visor'] = False

    def _handle_violation(self,
frame, box, track_id_to_log,
violation_type, speed,
vehicle_type,
extra_motors=None,
is_license_plate=False):

      """Process violation in a
thread to avoid blocking main
loop"""

      frame_copy = frame.copy()

          def violation_worker():

        try:

            x1, y1, x2, y2 =
map(int, box)

            h, w =
frame_copy.shape[:2]

            x1, y1 = max(0, x1),
max(0, y1)

            x2, y2 = min(w, x2),
min(h, y2)

          if y2 <= y1 or x2 <=
x1:

return

            print(f" [Thread]
Processing: {violation_type} |
ID: {track_id_to_log} | License
Plate: {is_license_plate}")

    plate_text, plate_img =
self._detect_license_plate(frame
_copy, box)

      clean_vehicle_type =
"Other"

            if vehicle_type ==
"Motor": clean_vehicle_type =
"Motor"

          elif vehicle_type ==
"Cars": clean_vehicle_type =
"Car"

          save_img = None;
image_type = "violation";
plate_number_for_db = None

            person_violations =
{"no helmet", "wearing short",
"wearing slippers", "no visor"}

            if is_license_plate:

              if plate_img is not
None and plate_img.size > 0:


plate_number_for_db =
plate_text if plate_text and
plate_text not in ["ERROR",
"UNREADABLE"] else
"Unreadable"

                  save_img =
plate_img

              else:

                  box_h = y2 - y1

                  box_w = x2 - x1

                  pad_x =
int(box_w * 0.10)

                  pad_y =
int(box_h * 0.15)

                  cx1 = max(0, x1 -
pad_x)

                  cy1 = max(0, y1 -
pad_y)

                  cx2 = min(w, x2
+ pad_x)

                  cy2 = min(h, y2 +
pad_y)

 save_img =
frame_copy[cy1:cy2, cx1:cx2]


plate_number_for_db =
"Unreadable"

                image_type =
"license_plate"

            print(f"  -> FORCE
SAVED license plate image for
{violation_type}")

            else:


plate_number_for_db =
"Unreadable" if plate_text ==
"UNREADABLE" else
"Unknown"

            if
violation_type.lower() in
person_violations and
clean_vehicle_type == "Motor"
and extra_motors:

          target_motor_box =
None

              for motor_det in
extra_motors:

                  if
motor_det['track_id'] ==
track_id_to_log:


target_motor_box =
motor_det['box']

                      break

            if
target_motor_box is not None:

                  motor_x1,
motor_y1, motor_x2, motor_y2
= map(int, target_motor_box)

                  motor_height =
motor_y2 - motor_y1

                  crop_x1 =
max(0, motor_x1)

                  crop_y1 =
max(0, int(motor_y1 -
motor_height * 0.2))

                  crop_x2 =
min(w, motor_x2)

                  crop_y2 =
min(h, int(motor_y2 +
motor_height * 0.1))
```

```
                    save_img = frame_copy[crop_y1:crop_y2, crop_x1:crop_x2]
                    image_type = "full_motor_bbox"
                    print(f"  -> Using full motorcycle image for {violation_type} (no license plate)")
                elif save_img is None:
                    vehicle_img = frame_copy[y1:y2, x1:x2]
                    if vehicle_img.size > 0:
                        save_img = vehicle_img
                        image_type = "vehicle_crop"
                        print(f"  -> Using vehicle crop for {violation_type} (no license plate)")
                if save_img is None or save_img.size == 0:
                    print(f"  -> WARNING: No image captured for {violation_type}")
                    return
                violation_data = {
                    "violation_type": violation_type, "vehicle_type": clean_vehicle_type,
                    "license_plate": plate_number_for_db, "speed": int(speed) if speed else None,
                    "timestamp": datetime.now().strftime('%H:%M:%S'), "date": datetime.now().strftime('%Y-%m-%d'),
                    "vehicle_track_id": int(track_id_to_log),
                    "image_type": image_type,
                }
                unique_id = uuid.uuid4().hex[:8]
                safe_violation_name = violation_type.replace(" ", "_").replace("/", "_").replace("(", "").replace(")", "")
                local_filename = f"{safe_violation_name}_ID{track_id_to_log}_{unique_id}.jpg"
                local_path = os.path.join("plates_images", local_filename)
                try:
                    cv2.imwrite(local_path, save_img)
                    print(f"  -> Saved image: {local_filename}")
                except Exception as e: print(f"[WARN] Local write failed: {e}")
                self._process_violation_async(violation_data, save_img.copy())
            except Exception as e:
                print(f"Threaded violation error: {e}")
        self.violation_count += 1
    threading.Thread(target=violation_worker, daemon=True).start()
    def _process_violation_async(self, data, image):
        """Wrapper for async violation processing"""
        if image is None or image.size == 0: return
        self.run_async(self._log_violation, data, image)
    async def _log_violation(self, violation_data, image):
        """Async violation logging to Supabase"""
        temp_path = None
        try:
            if image is None or image.size == 0: raise ValueError("Empty image")
            unique_id = uuid.uuid4().hex[:8]
            filename = f"violation_{unique_id}.jpg"
            temp_path = os.path.join(self.image_storage_dir, filename)
            cv2.imwrite(temp_path, image)
            with open(temp_path, "rb") as f:
                res = self.supabase.storage.from_(self.SUPABASE_BUCKET).upload(
                    path=f"violations/{filename}", file=f, file_options={"content-type": "image/jpeg"}
                )
            base_url = self.SUPABASE_URL.rstrip('/')
            image_url = f"{base_url}/storage/v1/object/public/{self.SUPABASE_BUCKET}/violations/{filename}"
            full_data = {**violation_data, "image_url": image_url}
            self.supabase.table("reportlogs").insert(full_data).execute()
```

```
            log_violation_type = violation_data.get('violation_type', 'UNKNOWN_TYPE')

            print(f"Successfully logged: {log_violation_type}")


        except Exception as e:

            print(f"Log error: {str(e)}")

        finally:

            if temp_path and os.path.exists(temp_path):

                try: os.remove(temp_path)

                except: pass

    def _calculate_speed(self, track_id):

        """Calculate vehicle speed in km/h"""

        data = self.vehicle_speeds[track_id]

        if 'first_seen' not in data or data['first_seen'] is None: data['first_seen'] = time.time()

        first_seen_time = data['first_seen']

        if first_seen_time is None: return 0

        current_calc_time = time.time()

        if current_calc_time - first_seen_time < 0.2: return 0

            positions = data['positions']

        times = data['times']

        if len(positions) < 3: return 0

        speeds = []

        for i in range(len(positions) - 1):

            try:

                dx = positions[i+1][0] - positions[i][0]

                dy = positions[i+1][1] - positions[i][1]

                distance_pixels = np.hypot(dx, dy)

                time_diff = times[i+1] - times[i]

                if time_diff > 0.01:

                    speed_mps = (distance_pixels * self.pixel_to_meter) / time_diff

                    speed_kmh = speed_mps * 3.6

                    if 0 < speed_kmh < 200: speeds.append(speed_kmh)

            except IndexError: continue

        if not speeds: return 0

        median_speed = np.median(speeds)

        current_speed = data.get('speed', 0)

        smoothed_speed = 0.8 * median_speed + 0.2 * current_speed if current_speed > 0 else median_speed

        data['speed'] = smoothed_speed

        return min(smoothed_speed, 200.0)

    def _point_in_polygon(self, point, poly):

        """Ray casting algorithm for point-in-polygon detection"""

        x, y = point; inside = False; n = len(poly); p1x, p1y = poly[0]

        for i in range(n+1):

            p2x, p2y = poly[i % n]

            if y > min(p1y, p2y):

                if y <= max(p1y, p2y):

                    if x <= max(p1x, p2x):

                        if p1y != p2y: xinters = (y-p1y)*(p2x-p1x)/(p2y-p1y)+p1x

                        if p1x == p2x or x <= xinters: inside = not inside

            p1x, p1y = p2x, p2y

        return inside

    def _detect_license_plate(self, frame, bbox):

        """MULTI-PASS OCR: Tries 3 different preprocessing methods to find text"""

        x1, y1, x2, y2 = bbox

        try:

            h, w = frame.shape[:2]

            x1c, x2c = max(0, int(x1)), min(w, int(x2))

            y1c, y2c = max(0, int(y1)), min(h, int(y2))

            if y2c <= y1c or x2c <= x1c: return "ERROR", None

            box_h = y2c - y1c

            box_w = x2c - x1c

            padding_x = int(box_w * 0.10)

            padding_y = int(box_h * 0.15)

            crop_x1 = max(0, x1c - padding_x)

            crop_y1 = max(0, y1c - padding_y)

            crop_x2 = min(w, x2c + padding_x)
```

```
        crop_y2 = min(h, y2c + padding_y)

        lp_roi = frame[crop_y1:crop_y2, crop_x1:crop_x2]

        if lp_roi is None or lp_roi.size == 0: return "ERROR", None

        def attempt_1_simple(img):

            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

            return cv2.resize(gray, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)

        def attempt_2_otsu(img):

            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

            resized = cv2.resize(gray, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)

            blurred = cv2.GaussianBlur(resized, (5, 5), 0)

            return cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]

        def attempt_3_adaptive(img):

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

            resized = cv2.resize(gray, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)

            filtered = cv2.bilateralFilter(resized, 11, 17, 17)

            return cv2.adaptiveThreshold(filtered, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,


cv2.THRESH_BINARY, 11, 2)

        methods = [attempt_3_adaptive, attempt_2_otsu, attempt_1_simple]

            config = '--oem 3 --psm 7 -c tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'

                for method in methods:

                processed_img = method(lp_roi)

                 processed_img = cv2.copyMakeBorder(processed_img, 10, 10, 10, 10, cv2.BORDER_CONSTANT, value=255)

                 text = pytesseract.image_to_string(processed_img, config=config)

                cleaned = ''.join(e for e in text if e.isalnum())

                  if cleaned and len(cleaned) >= 4:

                   return cleaned, lp_roi

                return "UNREADABLE", lp_roi

            except pytesseract.TesseractNotFoundError: print("[ERROR] Tesseract not found."); return "ERROR", None

        except Exception as e:
    print(f"OCR Error: {e}")

            return "ERROR", None

    def _is_light_flashing(self, frame, box, threshold_percentage=5.0, brightness_threshold=220):

        try:

            x1, y1, x2, y2 = map(int, box)

            h, w = frame.shape[:2]

            x1, y1 = max(0, x1), max(0, y1)

            x2, y2 = min(w, x2), min(h, y2)

            if x2 <= x1 or y2 <= y1:

                return False

            roi = frame[y1:y2, x1:x2]

            if roi.size == 0:

                return False

            hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

             h, s, v = cv2.split(hsv)

            bright_pixels = np.sum(v > brightness_threshold)

            total_pixels = roi.shape[0] * roi.shape[1]

            if total_pixels == 0:

                return False

           percentage = (bright_pixels / total_pixels) * 100

           if percentage >= threshold_percentage:

                return True

            else:

                 return False

    except Exception as e:
```

```
        print(f"[ERROR] Brightness check failed: {e}")
        return False

def _is_red_light_active(self):
    """Enhanced red light status check with time buffer"""
    elapsed = time.time() - self.last_change_time if self.last_change_time else 0
    red_duration = self.current_mode['red']
    if self.light_state == self.LightState.RED and self.last_change_time is not None:
        return 0 <= elapsed <= red_duration + 1.5
    return False

def _setup_flask_routes(self):
    @self.flask_app.route('/')
    def index():
        html = """
<html><head><title>Raspberry Pi - Traffic Cam</title>
<style>body{background:#282c34;display:flex;justify-content:center;align-items:center;height:100vh;margin:0}img{max-width:95%;border:3px solid #61dafb;border-radius:8px}</style>
        </head><body><img src="/video_feed"></body></html>
        """
        return render_template_string(html)

    @self.flask_app.route('/video_feed')
    def video_feed():
        return Response(self._generate_stream_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

def _draw_overlay(self, frame):
    """Draw informational overlay with dynamic colors"""
    try:
        if self.all_red_cooldown_active:
            light_color = (0, 0, 255)
            state_name = "ALL RED COOLDOWN"
        else:
            light_color = self.light_colors[self.light_state]
            state_name = self.state_names[self.light_state]
            remaining = 0.0
        if self.last_change_time is not None: remaining = max(0, self.current_duration - (time.time() - self.last_change_time))

        cv2.putText(frame, f"FPS: {self.fps_avg:.1f}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.putText(frame, f"{state_name} - {remaining:.1f}s", (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 0.7, light_color, 2)

        line_start_int = tuple(map(int, self.line_start)); line_end_int = tuple(map(int, self.line_end))
        cv2.line(frame, line_start_int, line_end_int, light_color, 2)
        cv2.putText(frame, f"{state_name} Zone", (line_start_int[0], line_start_int[1]-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, light_color, 2)
        cv2.putText(frame, f"Violations: {self.violation_count}", (10, 110), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        amb_status = "ON" if self.ambulance_override else ("READY" if self.ambulance_detected else "OFF")
        amb_color = (0, 255, 0) if self.ambulance_override else ((0, 255, 255) if self.ambulance_detected else (255, 255, 255))
        cv2.putText(frame, f"AMBULANCE OVERRIDE: {amb_status}", (10, 150), cv2.FONT_HERSHEY_SIMPLEX, 0.7, amb_color, 2)
        speed_area_int = self.speed_area.astype(np.int32)
        cv2.polylines(frame, [speed_area_int], True, (255, 0, 0), 2)
    except Exception as e: print(f"Error drawing overlay: {e}")

def _generate_stream_frames(self):
    """Generate frames for web streaming"""
    while self.running:
        with self.frame_lock:
```

```
            if self.streaming_frame is not None:
                frame = self.streaming_frame.copy()
            else:
                frame = np.zeros((480, 640, 3), dtype=np.uint8)
                cv2.putText(frame, "No frame available", (50, 240), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
            ret, jpeg = cv2.imencode('.jpg', frame)
            if ret:
                yield (b'--frame\r\n'
                       b'Content-Type: image/jpeg\r\n\r\n' + jpeg.tobytes() + b'\r\n')
            time.sleep(0.03)

    def run(self):
        """Main execution loop"""
        try:
            while self.running:
                if self.arduino and self.arduino.in_waiting > 0:
                    try:
                        incoming = self.arduino.readline().decode().strip()
                        # Detect siren signal "S"
                        if incoming == "S":

                            self.ambulance_audio_detected = True

                            self.last_siren_time = time.time()
                            print("[SIREN] Siren detected from Arduino")
                    except Exception as e:
                        print(f"Arduino read error: {e}")
                frame = self._capture_frame()
                if frame is None:
                    time.sleep(0.1); continue
                processed_frame = self.process_frame(frame)

                self.update_traffic_light()
                current_time = time.time()

                MOTOR_STATUS_TIMEOUT = 120
                expired_motors = [mid for mid, status in list(self.motor_violation_status.items()) if current_time - status.get('last_violation_time', 0) > MOTOR_STATUS_TIMEOUT]
                for mid in expired_motors:
                    if mid in self.motor_violation_status: del self.motor_violation_status[mid]

                NO_VISOR_TIMEOUT = self.HELMET_VIOLATION_DELAY + 20
                expired_helmet_timers = [tid for tid, t in list(self.helmet_detection_times.items()) if t is not None and current_time - t > NO_VISOR_TIMEOUT]
                for tid in expired_helmet_timers:
                    if tid in self.helmet_detection_times: del self.helmet_detection_times[tid]
                    if tid in self.no_visor_confirmed_time: del self.no_visor_confirmed_time[tid]

                TRACK_HISTORY_TIMEOUT = 60
                expired_tracks = [tid for tid, data in list(self.vehicle_speeds.items()) if data.get('times') and len(data['times']) > 0 and current_time - data['times'][-1] > TRACK_HISTORY_TIMEOUT]
                for tid in expired_tracks:
                    if tid in self.vehicle_speeds: del self.vehicle_speeds[tid]
                    if tid in self.track_history: del self.track_history[tid]
                    if tid in self.crossing_records: del self.crossing_records[tid]
                if processed_frame is not None:
                    try:
                        resized_frame = cv2.resize(processed_frame, self.display_size)

                        self._draw_overlay(resized_frame)

                        with self.frame_lock:

                            self.streaming_frame = resized_frame.copy()
```

```
        cv2.imshow("Traffic Monitoring System", resized_frame)
            except cv2.error as e: print(f"[ERROR] OpenCV display error: {e}")
    key = cv2.waitKey(1) & 0xFF
            if key == ord('q') or key == 27: print("Quit key pressed."); self.running = False; break
        except KeyboardInterrupt: print("Ctrl+C pressed. Shutting down..."); self.running = False
        except Exception as e: print(f"Runtime error in main loop: {e}"); import traceback; traceback.print_exc()
        finally: print("Shutting down..."); self.shutdown()
    def shutdown(self):
        print("Executing shutdown sequence...")
        self.running = False; time.sleep(1.0)
        if self.picam2 is not None and PICAMERA2_AVAILABLE:
            try: self.picam2.stop()
            except: pass
        if hasattr(self, 'cap') and self.cap is not None and self.cap.isOpened():
            try: self.cap.release()
            except: pass
        try:
cv2.destroyAllWindows()
            for _ in range(5): cv2.waitKey(1)
        except: pass
        if self.arduino and hasattr(self.arduino, 'is_open') and self.arduino.is_open:
            try: self.arduino.close()
            except: pass
        print("System shutdown complete")
if __name__ == "__main__":
    system = TrafficMonitoringSystem()
    system.run()
```

APPENDIX L. DATASHEET

Arduino Uno

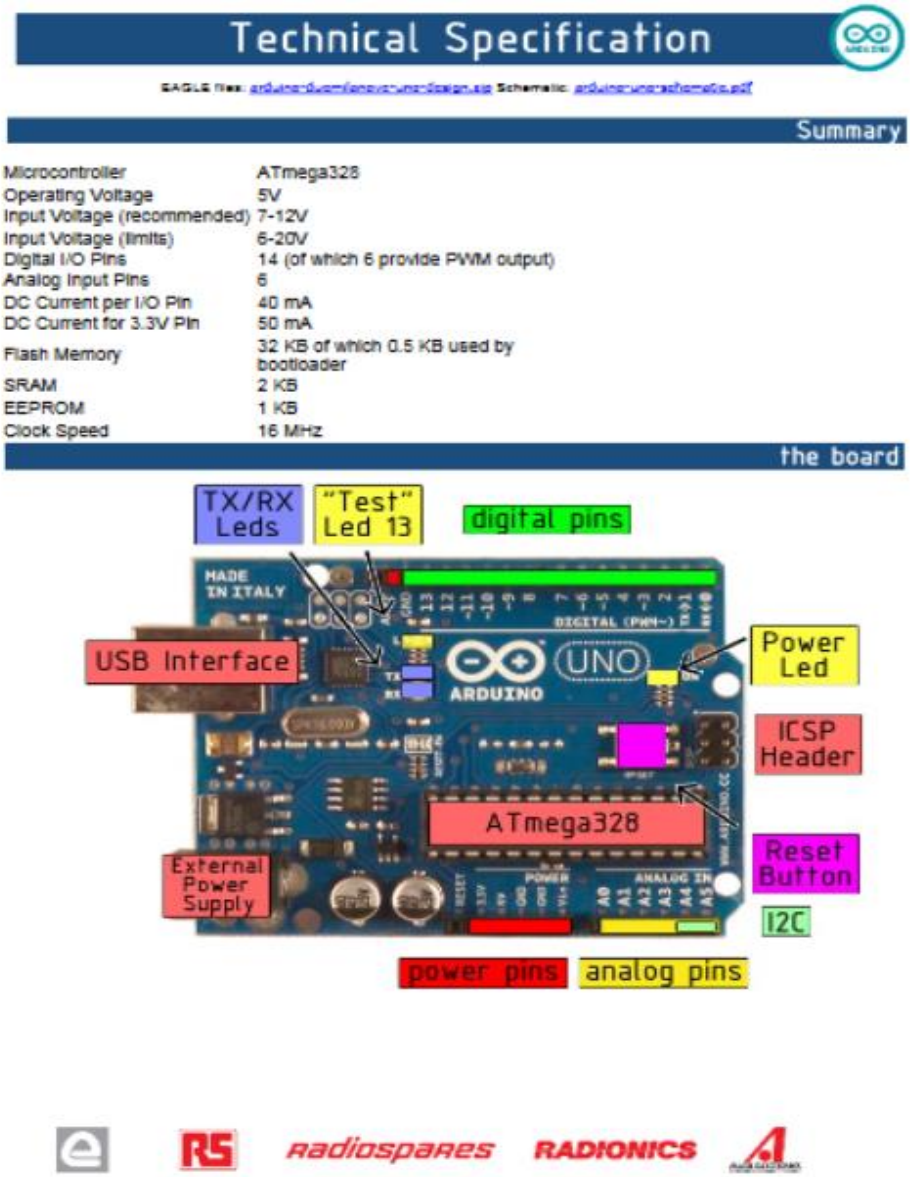


Figure 80. *Arduino Datasheet*

Raspberry Pi 5

- 2.4GHz quad-core, 64-bit Arm Cortex-A76 CPU, with 512KB L2 caches and a 2MB shared L3 cache
- VideoCore VII GPU
- LPDDR4X-4267 SDRAM (4GB and 8GB SKUs available at launch)
- 2.4 GHz and 5.0 GHz 802.11ac Wi-Fi
- Bluetooth 5.0 / Bluetooth Low Energy (BLE)
- Micro SD card slot, with support for high-speed SDR104 mode
- 2 × USB 3.0 ports, supporting simultaneous 5Gbps operation
- 2 × USB 2.0 ports
- Gigabit Ethernet, with PoE+ support (requires PoE+ HAT)
- 2 × 4 lane MIPI camera/display transceivers
- PCIe 2.0 x1 interface for fast peripherals
- 5V/5A DC power (PD enabled)
- Raspberry Pi standard 40-pin header
- Real-Time Clock (RTC), powered from external battery
- On-board power button!!

Raspberry Pi 5 Power Supply 27W USB-C PD (Power Delivery)

The new 27W USB-C PD (Power Delivery) Power Supply will deliver a maximum of 5.1V, 5A enabling the Raspberry Pi 5 to power a wider range of peripherals. The total power drawn from the four USB ports on the Raspberry Pi 5 is limited by default to a nominal 600mA; this limit is automatically increased to a nominal 1.6A when the USB-C PD Power Supply is detected.
The USB-C PD Power Supply is also capable of delivering 3A @ 9V, 2.25A @ 12V, and 1.8A @ 15V to PD-compatible products, making it a cost-effective power supply for many general-purpose, non-Raspberry Pi use cases.

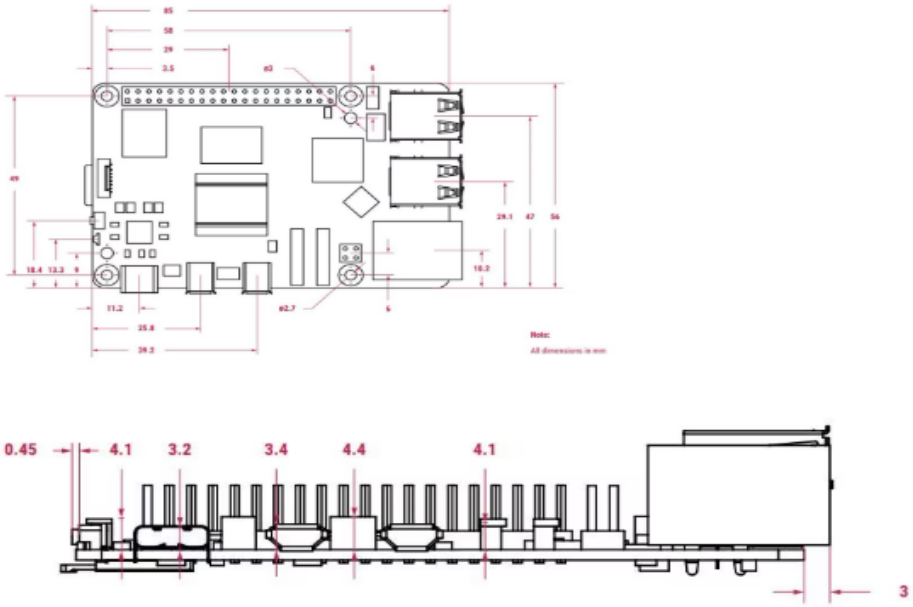


Figure 81. *Raspberry Pi Datasheet*

Portable Power Station 200w/220v

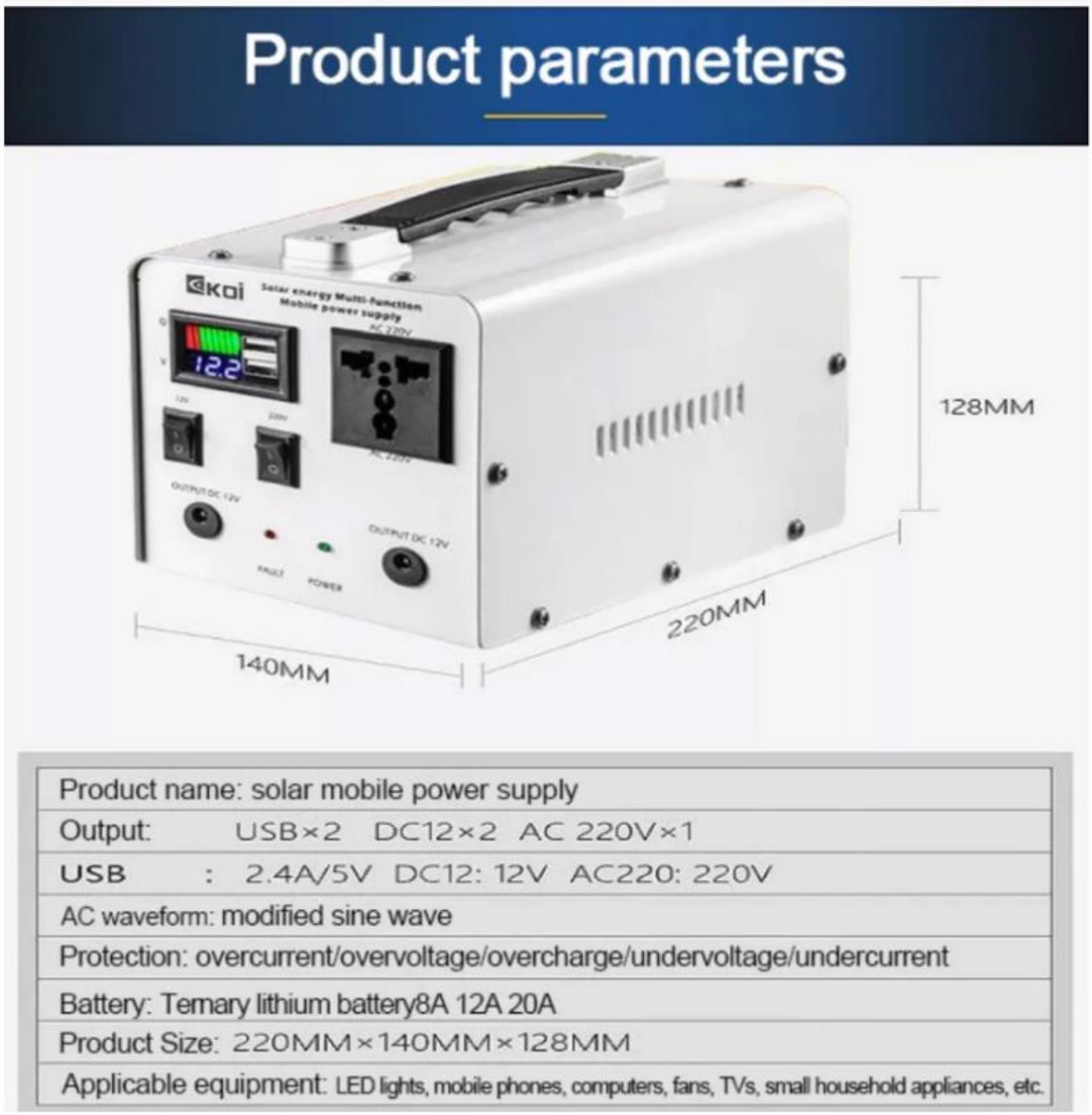


Figure 82. *Power Station Datasheet*

1 Channel 5V 10A Relay Module

Features:

•One normally closed contact and one normally open contact

•Triode drive, increasing relay coil

•High impedance controller pin

•Pull-down circuit for avoidance of malfunction

•Power supply indicator lamp

•Control indicator lamp

Specification:

•Channel: 1 Channel

•Relay Voltage: 5V

•Dimensions: 56.5mm x 17mm x 20mm

•Weight: 16.5g

*2 Channel 5V 10A Relay Module with Optocoupler

Specs:

- 250VAC 10A, 28VDC 10A

- Indicator LEDs

- 5V control voltage

Figure 83: *Relay Datasheet*

Arducam 12.3MP HQ Camera

Camera Specs	B0240
Sensor	Sony IMX477 Exmor RS sensor
Sensor Size	1/2.3"
Resolution	12.3MP 4056 × 3040
Video	2028 × 1080p50, 2028 × 1520p40 and 1332 × 990p120
Focus Type	Manual Focus
Lens Type	CS lens
Pixel Size	1.55μm
Focal Length	6mm
F.NO	F1.2
Field of View (FOV)	65°(H)
Back Focal Length	7.53mm
Lens Dimension	Φ30×31mm
IR Sensitivity	Integral IR-cut Filter, visible light only
Dimension	38mm × 38mm
Hole Pitch	Compatible with 29mm, 30mm, 34mm
Peaking Current	300mA
Supported Platforms	Raspberry Pi 4, 3B+, 3, 2, B+, B, A and Pi Zero

Figure 84. *Arducam Datasheet*

Philips LED Downlight

- Brand: **Philips**
- Item: **Recessed Type LED Downlight**
- Model/Series: **Eridani DL190B**
- Input: **220-240V 50/60Hz**
- Power Factor: **0.5**
- Wattage: **14W / 23W**
- Color Temperature: **Warmwhite 3000K / Cool White 4000K / Daylight 6500K**
- IP Rate: **IP20 (Indoor and dry place use only!)**

Philips Eridani Led Downlight DL190B

Wattage	Color Temperature	Lumen	Hole Cut Size
14W	Daylight 6500K	1200LM	ø 150MM 5.9"
14W	Cool White 4000K	1200LM	ø 150MM 5.9"
14W	Warm White 3000K	1150LM	ø 150MM 5.9"
23W	Daylight 6500K	1900LM	ø 200MM 7.9"
23W	Cool White 4000K	1900LM	ø 200MM 7.9"

Figure 85. *Philips LED Downlight Datasheet*

1 Features

- NEW LM393B and LM2903B
- Improved specifications of B-version
  - Maximum rating: up to 38V
  - ESD rating (HBM): 2kV
  - Low input offset: 0.37mV
  - Low input bias current: 3.5nA
  - Low supply-current: 200µA per comparator
  - Faster response time of 1µsec
  - Extended temperature range for LM393B
  - Available in tiny 2 x 2mm WSON package
- B-version is drop-in replacement for LM293, LM393 and LM2903, A and V versions
- Common-mode input voltage range includes ground
- Differential input voltage range equal to maximum-rated supply voltage: ±38 V
- Low output saturation voltage
- Output compatible with TTL, MOS, and CMOS

2 Applications

- Vacuum robot
- Single phase UPS
- Server PSU
- Cordless power tool
- Wireless infrastructure
- Applicances
- Building automation
- Factory automation & control
- Motor drives
- Infotainment & cluster

3 Description

The LM393B and LM2903B devices are the next generation versions of the industry-standard LM393 and LM2903 comparator family. These next generation B-version comparators feature lower offset voltage, higher supply voltage capability, lower supply current, lower input bias current, lower propagation delay, and improved 2 kV ESD performance and input ruggedness through dedicated ESD clamps. The LM393B and LM2903B can drop-in replace the LM293, LM393 and LM2903, for both "A" and "V" grades.

All devices consist of two independent voltage comparators that are designed to operate from a single power supply over a wide range of voltages. Quiescent current is independent of the supply voltage.

Device Information

PART NUMBER	PACKAGE(1)	BODY SIZE (NOM)
LM393B, LM2903B, LM193, LM293, LM293A, LM393, LM393A, LM2903, LM2903V, LM2903AV	SOIC (8)	4.90mm x 3.91mm
LM393B, LM2903B, LM293, LM293A, LM393, LM393A, LM2903	VSSOP (8)	3.00mm x 3.00mm
LM293, LM393, LM393A, LM2903	PDIP (8)	9.81mm × 6.35mm
LM393, LM393A, LM2903	SO (8)	6.20mm x 5.30mm
LM393B, LM2903B, LM393, LM393A, LM2903, LM2903V, LM2903AV	TSSOP (8)	3.00mm x 4.40mm
LM393B, LM2903B	SOT-23 (8)	2.90mm x 1.60mm
LM393B, LM2903B	WSON (8)	2.00mm × 2.00mm

(1) For all available packages, see the orderable addendum at the end of the data sheet.

Family Comparison Table

Specification	LM393B	LM2903B	LM393 LM393A	LM2903	LM2903V LM2903AV	LM193	LM293 LM293A	Units
Supply Voltage	2 to 36	2 to 36	2 to 30	2 to 30	2 to 32	2 to 30	2 to 30	V
Total Supply Current (5V to 36V max)	0.6 to 0.8	0.6 to 0.8	1 to 2.5	1 to 2.5	1 to 2.5	1 to 2.5	1 to 2.5	mA
Temperature Range	−40 to 85	−40 to 125	0 to 70	−40 to 125	−40 to 125	−55 to 125	−25 to 85	°C
ESD (HBM)	2000	2000	2000	2000	2000	2000	2000	V
Offset Voltage (Max over temp)	± 4	± 4	± 9 ± 4	± 15	± 15 ± 4	± 9	± 9 ± 4	mV
Input Bias Current (typ / max)	3.5 / 25	3.5 / 25	25 / 250	25 / 250	25 / 250	25 / 100	25 / 250	nA
Response Time (typ)	1	1	1.3	1.3	1.3	1.3	1.3	µsec

Figure 86. *Sound Sensor Datasheet*

Adaptor 12V 2A

Connector(s)	Output Connectors	1 - M-Type Barrel (Inner Diameter: 2.1 mm, Outer Diameter: 5.5 mm) Plug Output
Power	Center Tip Polarity	Positive
	Input Current	0.6 A
	Input Voltage	100 ~ 240 AC
	Input Voltage Frequency	50/60Hz
	Output Current	2 A
	Output Voltage	12 DC
	Plug Type	M
	Power Consumption (In Watts)	24
	Power Source	AC Adapter Included
Physical Characteristics	Cable Length	1219 mm [48 in]
	Product Height	43 mm [1.7 in]
	Product Length	75 mm [3 in]
	Product Weight	137 g [4.8 oz]
	Product Width	40 mm [1.6 in]
Packaging Information	Shipping (Package) Weight	0.2 kg [0.5 lb]

Figure 87. ***Adaptor Datasheet***

APPENDIX M. BUDGETARY EXPENSES

THESIS EXPENSES

Quantity	Specifics	Approximate Cost	Actual Cost
1pc	Raspberry pi 8gb ram	₱8,000	₱6,200
1pc	Arducam Wide lens 6mm with Rapsberry pi camera module	₱6,000	₱8,330
3pcs	pcs Philips LED Downlight 1200 lumen, 14w LED	₱1,000	₱807
9pcs	9 pcs Philips LED Downlight, 10w LED	₱1,700	₱1,550
3pcs	Electrical tape	₱200	₱144
4pcs	Cellophane Color red	₱150	₱109
4pcs	Cellophane Color green	₱150	₱109
4pcs	Cellophane Color yellow	₱150	₱109
1pc	Soldering Iron	₱150	₱118
1pc	Portable Power Station 200w/220v	₱3,000	₱2,959
2pcs	Plywood	₱1,700	₱1,600
1pc	Stikwel	₱50	₱45
1kg	Nails	₱50	₱40

50pcs	Wires 20cm Male-Male	₱70	₱80
50pcs	Wires 10cm Female-Female	₱40	₱70
1pc	Type C to Type C 100W PD Fast Charging	₱200	₱160
1pc	Original Type C fast charging Cable 66W 6a	₱170	₱160
100pcs	Steel nail clips	₱60	₱59
1pc	Blue and red solar strobe lights road traffic safety	₱850	₱799
1pc	20 meters Hypertech TW solid Wire	₱580	₱540
14pcs	Relay 5v 1 channel	₱510	₱504
4pcs	Door hinge	₱25	₱20
16pcs	Screw	₱40	₱30
1pc	Metro	₱100	₱99
1pc	Pole	₱600	₱588
2pcs	Steel	₱3,000	₱2,230
	Labor cost	₱700	₱650
1pc	Essager 6A USB Type C	₱180	₱158
	Colab Pro +	₱3,000	₱2,904.95

1pc	BAVIN PC008 22.5W Power Charging PowerBank	₱3,000	₱2,425
1Ltr	B-690 QDE Black	₱300	₱202
1/4Ltr	B-630 QDE Silv. Gray	₱100	₱73
2pcs	P. Brush no.1 (FSL)	₱50	₱42
1pc	P. Brush no. 4(Legend)	₱150	₱125
1pc	Plainsheet 3x8 non-colored	₱280	₱240
2pcs	Ordinary hinges no. 1 1/2	₱30	₱24
1pc	Ordinary hinges no. 2	₱15	₱15
1set	Safety hasps no.2	₱20	₱15
1pc	Scissor K-012	₱100	₱60
1pc	Driver set 775mm	₱50	₱39
1set	Nails	₱60	₱50
Total			₱34,481.95.

Table 16. *List of Materials Bought*

Prepared by:

Duane Benedict A. Mahinay

Harvey G. Malicse

Alvin Ken G. Modina

Juliana Paula P. Baloy

Noted By:

Azaneth A. Mahinay

Lilibeth G. Gregorio

Amorsolo V. Modina

Sarah Joy P. Pagunsan

Approved by:

Sherwin M. Ostria

APPENDIX N. CURRICULUM VITAE



Curriculum Vitae of

JULIANA PAULA P. BALOY

4 Apple St. New Lower Bicutan, Taguig City
Jpbaloyyy@gmail.com
09763271612

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022-Present	STI College Global City
TechVoc	month year	N/A
High School	2016-2020	Saint Francis of Assisi College
Elementary	2010-2016	Ricardo P. Cruz Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
November 2024	Children's Halloween Feeding Program	Samahang Pampalakasan ng STIers (SAMPAS)

AFFILIATIONS

Inclusive Dates	Name of Organization	Position

SKILLS

SKILLS	Level of Competency	Date Acquired
Communication	7/10	2019
Leadership	7/10	2019
Organization	8/10	2022
Adaptability	8/10	2017

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop
2024	Student Leadership Summit
2023	Student Leadership Summit



Curriculum Vitae of

DUANE BENEDICT A. MAHINAY

#21 9th St. Zone-1 North Signal Taguig City
Duanemahinay@gmail.com
09682204305

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022-Present	STI College Global City
TechVoc	N/A	N/A
High School	2016-2020	Hansarang Christian Academy
Elementary	2010-2016	Hansarang Christian Academy

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization

AFFILIATIONS

Inclusive Dates	Name of Organization	Position

SKILLS

SKILLS	Level of Competency	Date Acquired
Communication	8/10	2017

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop



Curriculum Vitae of

HARVEY G. MALICSE

#4 Tindalo St., North Signal Village, Taguig City
malicseharveyg@gmail.com
0927-018-6677

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022-Present	STI College, Global City
TechVoc	N/A	N/A
High School	2016-2020	Hansarang Christian Academy Inc.
Elementary	2010-2016	EM'S Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization

AFFILIATIONS

Inclusive Dates	Name of Organization	Position

SKILLS

SKILLS	Level of Competency	Date Acquired

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop



Curriculum Vitae of

ALVIN KEN G. MODINA

Blk 39 Lot 5 Phase 2 Pinagsama Village , Taguig City
kkenr2c@gmail.com
+639618773469

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022-Present	STI College, Global City
TechVoc	N/A	N/A
High School	2016-2020	KERIS
Elementary	2010-2016	KERIS

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization

AFFILIATIONS

Inclusive Dates	Name of Organization	Position

SKILLS

SKILLS	Level of Competency	Date Acquired
Communication	8/10	2020
Problem Solving	7/10	2016

TRAININGS, SEMINARS OR WORKSHOP ATTENDED

Inclusive Dates	Title of Training, Seminar or Workshop